SlopNinja

Writing transformation and
AI detection on Bittensor

SlopNinja Research

November 11, 2026
Draft 0.65

Abstract

SlopNinja proposes an adversarial training subnet on Bittensor, pairing mechanism *A*'s AI detection with mechanism *B*'s writing transformation. *B* is the main product objective: private models revise authorized text toward the author's intended voice while preserving meaning, readability, and tone. *A* learns to detect AI involvement, including in *B*'s revisions; *B* learns to remove unwanted formulaic prose and evade *A*'s detection. Public *A* models supply training feedback for *B*, while authorized *B* revisions provide challenging examples for future *A* training after evaluation retirement. This competition is intended to improve both mechanisms; gains must hold on fresh evaluation data. *A* is also intended to supply author-fit feedback against a private writer profile built from a user's own samples, once a learned profile matcher qualifies. A lower detector score neither establishes better writing nor changes a document's production history.

The subnet launches with *A* alone as the initial evaluator and detection service; *B* activates after its editing, private-serving, and reward requirements are qualified. *A* is one text model with a shared encoder. It reports estimated AI-written or rewritten text share and confidence in any AI involvement as separate percentages, attributes human and model contributors, and locates AI-written spans of at least 24 words, with per-sentence share and confidence; shorter edits still count in both document percentages. AI-involvement Brier score is the only reward quantity; share, attribution, and spans are qualification gates, and once a cohort template enforces it, every entry must also pass a span gate that compares its spans with SlopNinja's first span package. On a sealed, pre-registered Test of passages whose human documents are excerpts of 150 to 450 words, the *A* baseline's document model, the first reference, was noninferior to Pangram 4.1, with balanced Brier 0.0518 against 0.0551 after both detectors' outputs pass through maps fitted by the same method on a separate Calibration split, but the pre-registered verdict failed because its three-seed recipe missed one domain slice. SlopNinja's first span package leaves those document outputs unchanged and adds a span head; retrospectively, on the consumed Test, it reaches character F1 0.544 on spans of at least 24 words and puts a span on 2.5% of human documents. *A* reads raw text and uses no context-free-grammar (CFG) features, which failed their first real-data test. Private writer profiles use *A*'s encoder plus stylometric features; no profile tested so far is a reliable author-fit signal.

B's proposed private adapters run in miner-owned Cloudflare accounts: miners pay their model bills but must not receive customer prompts or responses. Cloudflare and SlopNinja remain trusted operators. *B*'s editing pipeline is planned to use a shared English grammar for typed word, phrase, and clause edits and for revision guidance. Mechanism *C* credits accepted reusable contributions to that grammar once. It activates by its own decision, which may come before *B*'s so the grammar can grow first, and is not required for the *A* launch.

One lead validator, operated by SlopNinja, serves as the coordinator. The protocol below states requirements, and Section 14.2 records which parts run today: the coordinator is specified to run entirely serverlessly on Cloudflare Workers but remains unfinished, and testnet cohorts still run in a separate evaluation service. Miners commit artifact digests on chain and upload their model files straight to the coordinator's private storage. Each cohort, at least a day long, scores every entrant and the current and previous reference models (when a previous one exists) on a fresh hidden batch of paired cases, a fifth of them human-only. On mainnet its human text comes from Common Crawl pages captured before ChatGPT's public release; testnet uses a smaller public stock, and

every testnet cohort so far has drawn only documents already exposed. Entrant models are published, licensed CC BY-SA 4.0, when scoring starts, and SlopNinja publishes its own reference models on the same terms, without evaluation text. Exact scores never leave the coordinator. It releases noisy quantized scores under a per-cohort privacy budget and derives the native weight row from them by exact rules ported from subnet 78's geometric score bands: detected copies of published models earn nothing, and near-duplicate entries share one credit. Until an entrant first clears the reference by a margin, miner rewards burn. Released scores are noisy enough that an entrant merely equal to the reference clears that margin by chance about a sixth to a third of the time on testnet (Section 5.9); at mainnet's planned size the current privacy budget would make rewards mostly noise, so it, the batch size, or the budget split must change before mainnet. The reference changes when such an entrant also passes the promotion checks, or by a recorded operator decision. The coordinator signs the row; every other validator verifies it, commits it every tempo through native commit–reveal without seeing held-out data, and keeps committing it through coordinator silence, since the authorization it commits under never expires. Slow work and outages preserve the cohort, no record depends on a chain runtime version, and no rule locks the operator out: it can abandon a cohort, refuse to pay one, set the references, or restart the record, each by a journaled decision with a signed public record, the high-stakes ones under a separate offline key.

Testnet runs for several months before the mainnet subnet is registered. On November 7, 2026 its admission opened to every registered hotkey, and a cohort with an outside entrant has since run end to end there. Mainnet will open a fresh record of cohorts with the reference testnet then holds. During testnet, SlopNinja plans to earn revenue from a free public detection page served by A, with no advertising on the page that holds pasted text, and from other products. Their payouts go into a publicly visible pot of native TAO in a published two-of-three multisignature account with an independent third signer, which lists every payout, conversion, purchase, burn, and claim. Once the pot holds about 1.5 times the cost of a mainnet registration, SlopNinja registers the mainnet subnet with it. After the subnet launches, the rest buys its alpha in daily tranches bound to that registration: half is burned, and half is distributed through a claim contract to testnet miners in proportion to the testnet-era revenue earned by their winning models, which must first have beaten the reference; any part of that half left undistributed is burned. From the launch on, all revenue buys and burns alpha; if SlopNinja abandons mainnet, the pot's TAO is burned. Customers may also fund quoted paid detection jobs by card, TAO, or subnet alpha, whose principal converts to alpha and is burned. Pricing, any emissions assigned to paid use, and the mainnet split of emissions among owner, validators, and miners are not selected.

1 Purpose and scope

SlopNinja's primary goal is to transform writing so it sounds natural and matches the author's intended voice. Mechanism *B* serves that goal by revising authorized text to remove unwanted formulaic prose and reduce detectable model-writing patterns while preserving meaning, readability, and tone. Mechanism *A* supplies detection and attribution signals that support *B*'s revision search and evaluation, alongside separate preservation and editorial checks. Together they form an adversarial training subnet: *B* trains against public *A* detectors, and authorized *B* revisions challenge future *A* training after evaluation retirement. *A* seeks to identify AI involvement even when *B* succeeds at reducing its visible signatures. The subnet launches with *A* alone; this development sequence establishes a supporting evaluator and initial detection service before the main writing product is qualified. Protocol sections state requirements; Section 14.2 records which parts are implemented today. A model, path, or component is *qualified* once recorded measurements or tests show that it meets its declared requirements; qualification precedes activation.

A's origin detection estimates whether a document has human-only, model-only, or mixed production history. Model-only and mixed work both count as AI involvement under the declared labeling policy. The detector reports confidence in any AI involvement and estimated AI-written or rewritten text share as separate percentages, and attributes human and model contributors where supported. It also locates AI-written spans of at least 24 words, with per-sentence share and confidence (Section 3.2); shorter edits count toward both document percentages but are not localized. The scorer serves span outputs, and once the standing cohort template enforces a span gate, measured against SlopNinja's first span package on the same batch, every entry must emit spans and pass it, while the reward stays document-level. Explanations tied to particular words or constructions are later extensions.

Human writers have recurring habits of expression. Language models can also exhibit characteristic choices in vocabulary, syntax, rhythm, and document structure, shaped by training, tuning, and prompting. The proposed detector learns these writing signatures through a shared origin-and-attribution model. The hypothesis is that learning whose writing a text resembles helps determine whether AI was involved. Signatures can overlap and change with prompts or editing; they are statistical evidence, not unique identifiers. Their value for AI detection must be measured on unseen writers, generators, and workflows.

A learns these signatures from raw text and uses no grammar features. A shared English grammar is planned as a later input for *B*, where it locates typed word, phrase, and clause edits and guides revision. Detector evaluation can begin before transformation, private serving, or grammar incentives are ready.

The three logical mechanisms have distinct jobs:

- *A*, AI detection and attribution: one text model estimates AI involvement from production origin, matches human or AI-model profiles, and locates AI spans of at least 24 words. Entrant models are published, licensed CC BY-SA 4.0, when each daily evaluation cohort's scoring starts. Launch includes *A* alone.
- *B*, authorized transformation: a private model proposes revisions that improve fit to a writer's brief and reduce detector scores while preserving meaning, readability, and intended tone. It activates separately.
- *C*, grammar improvement for *B*: accepted reusable lexical and structural contributions to the grammar that *B* uses for edits earn credit once, after a separate acceptance and reward protocol is qualified. It activates by its own decision, possibly before *B*.

These names describe workstreams, not three available native chain mechanisms. The launch uses one native mechanism; later allocation must fit the chain's supported mechanism interface (Section 5).

1.1 What a prediction means

Origin is a production-history claim. Human-only, model-only, and mixed labels refer to how a document was made under an explicit labeling policy. Attribution instead asks which registered human or model profiles it resembles. A known writer may use AI assistance; an unfamiliar model may produce text with no good gallery match. Neither a matching human profile nor a low detector score proves human-only origin. An unavailable prediction must remain an error or abstention, never a default human label.

The launch evaluates English documents within declared length and resource bounds. The evidence so far comes from passages whose human documents are whole-paragraph excerpts of 150 to 450 words; other lengths need

their own measurements. Span outputs rely on retained exact span labels; multilingual detection and manuscript-scale support each require suitable data and measurements. Handling an embedded foreign phrase, equation, or code block does not establish detection quality for that language or content type. Predictions must retain uncertainty, calibration context, and the limitations of the measured population.

We use *slop* for defects in prose: vague or formulaic phrasing, needless repetition, filler, and other patterns that weaken the piece. This is an editorial definition, separate from production origin. Informal language, unusual grammar, or a writer's recurring verbal habit may be appropriate to a brief. The shared grammar planned for *B*'s edits likewise describes English as used, including errors and nonstandard forms, rather than enforcing a prescriptive ideal.

1.2 The grammar's role in later editing

A resolved parse exposes lexical choices, grammatical relations, and document structure in a reusable representation. *B* is planned to use the shared English grammar to divide a source into typed word, phrase, and clause edit units and to compare each revision with its source (Section 10). *A* does not use it. The first real-data test of a CFG-only detector failed, and adding CFG features to text gave only a small, uncertain gain, so *A*'s input contract is raw text (Section 3.3). A head-to-head test of writer profiles found CFG writer vectors below *A*'s encoder on writer verification, by -0.034 in AUC, simple stylometric features above both, and only $+0.0015$ from adding CFG to the encoder plus stylometry, so private writer profiles use the encoder and stylometric features (Section 14.1). For *B*, parsing costs money and time, and resolution can be wrong. Whether grammar-typed edits help *B*, whether *C*'s contributions improve them, and any later explanations each need their own evidence.

1.3 Commercial thesis and users

The first paid product is hosted AI detection for applications, publishers, researchers, and document workflows that need to assess AI involvement in writing, with origin probabilities and attribution as additional detail. *A* artifacts become freely executable when their cohort's scoring starts. Paid access, through the detection API or quoted jobs, offers maintained attribution profiles, bounded execution of a qualified artifact, private delivery, and integration convenience. An *A* detection job runs the raw-text model alone, with no grammar parse. Publishing a model does not establish that users will pay for hosting it; the commercial pilot must measure repeat use, willingness to pay, false-positive disputes, and delivery cost.

Testnet runs for several months before the mainnet subnet is registered. During that period SlopNinja plans to earn revenue from free, advertising-supported public detection pages served by *A* and from other products, accruing in a publicly visible pot (Section 8.1). The pot pays the mainnet subnet's registration once it holds about 1.5 times its cost. After the launch, the rest buys the subnet's alpha: half is burned, and half is distributed to testnet miners in proportion to the revenue earned by their winning models, those that beat the reference, with any part left undistributed burned; all later revenue buys and burns alpha. The detection page itself carries no advertising and stores no text; ads run on the site's content pages, and a paid detection API follows once the served package is published. The pot is a published multisignature TAO account with a public ledger of every payout, conversion, purchase, burn, and claim. The coordinator decides when measured quality is sufficient to open sales and which workloads may be sold. Benchmark eligibility alone does not enable checkout. The initial price hypothesis aims below Pangram's standalone API rate, subject to an independently evaluated quality comparison and measured costs. The principal of a quoted paid job buys and burns alpha, while the detection API's revenue joins the pot. Spendable operating funds and refund reserves must come from an explicitly funded treasury, independently of the burn (Section 8.6).

The main intended writing product, activated after the initial detection service, serves users who supply samples of their own writing. A private writer profile summarizes those samples from *A*'s encoder and stylometric features; *A* compares successive revisions with that profile, and *B* uses its author-fit signal to guide editing toward the requested voice. No profile tested so far ranks rewrites toward a writer reliably, so this workflow waits for a learned matcher qualified on fresh writers (Section 3.1). It aims to preserve every argument, qualification, attribution, and necessary detail while improving voice and removing unwanted formulaic prose. Editorial quality and production history remain separate: a revision of model-generated writing retains that history even if readers prefer it or detectors miss it. Incremental improvement and attainment of a strict detector target are also separate claims (Section 10.3).

The intended adversarial training loop uses public detectors during private revision search and reuses authorized revisions after evaluation retirement. It must measure whether adaptive editing improves prose or merely exploits the current judges. Writer consent and source rights limit training reuse; customer jobs do not become benchmark material by default. The same tools can be used to obscure provenance or impersonate a writer. Product claims must preserve that risk and must not represent detector evasion as evidence of human authorship.

2 Participants and execution

An *A* miner trains a versioned text model for origin, share, span, and attribution outputs. The artifact contains the weights, tokenizer, calibration, and execution contract needed to run every output it declares. The miner commits only its digest on chain and uploads the artifact's files straight to the coordinator's private R2 storage through presigned URLs, never through a SlopNinja Worker; the coordinator publishes the artifact when the cohort's scoring starts. A published artifact, the miner's own included, earns nothing when entered again, unless its cohort was abandoned after its scoring started (Section 6.6), so a miner earns again only with a model that differs measurably from every published one or improves on its source by half the promotion margin (Section 5.10). SlopNinja executes submitted artifacts for the pilot's hidden evaluations and published ones for customer requests. Neither step requires the miner to operate a serving endpoint or Cloudflare account. Miners pay for their training and development; the operator pays for centrally served *A* inference. Benchmark rewards depend on measured performance, not a claim about training expenditure.

SlopNinja is the subnet *operator*. The *subnet owner* is the account recognized on chain; SlopNinja may operate through ownership or a lease. A lease must secure an owner-authorized execution path for required hyperparameter changes, including enabling commit–reveal, and any owner-only burn operations. Operating the service alone grants neither those powers nor the owner's emission entitlement.

One lead validator, operated by SlopNinja, serves as the *coordinator*. This validator is the serverless service: it administers the daily frozen evaluation cohorts, runs their models and the reference models, and alone computes the reward outcome. It signs a private score package of exact measurements, which never leaves its storage, and a public quantized result from which anyone can recompute the native weight row. It derives that row, signs it as a row record, and publishes the record off chain as soon as it is signed. It also signs, submits, and recovers its own native weight commitments, then certifies its finalized row. These tasks all run within the serverless deployment. Every other validator verifies the signed row record and its authorization under pinned keys and commits that row, using its own key, nonce, and recovery journal to commit and reveal it in parallel with the others. Its only change to the row uses its own finalized snapshot: the units the row burns through the coordinator's recorded owner hotkey, and the units of any registration that snapshot no longer holds, go to the UID of that snapshot's owner hotkey, which burns them. No validator waits for the coordinator's reveal or finalized row before committing. Held-out documents, labels, individual predictions, exact scores, receipts, customer text, and private model weights never go to miners, other validators, or the public; held-out passages leave the coordinator only in its generation requests to hosted model providers (Section 5.2). Other validators do not reproduce its hidden model scoring. A validator may recompute the row from the public result, check models in its own way, and publish its own scores; those checks are advisory, never change the row it commits, and never pause its commitments. Section 6 defines the row record, native submission rules, and per-validator recovery requirements.

This pilot trusts the coordinator to execute, score, and derive rows honestly. Recomputing the row from the public quantized result checks reward arithmetic over the released scores; it cannot establish that the reported predictions are real. After its own row finalizes, the coordinator publishes a certificate for audit against the same score package and row. Its certificate authenticates those bindings without creating a validator quorum or a submission prerequisite. The coordinator also decides when measured quality justifies offering paid inference, and which workloads the service may accept. Commercial activation does not revise a cohort's scores.

Dependence on other validators is confined to their timely native delivery of the coordinator's weight row. Scoring, cohort advancement, and customer service require no validator quorum and do not depend on other validators' acknowledgments, availability, software updates, or migration completion. Each validator submits and recovers independently; delayed validator rows affect native reward allocation without holding evaluation or service work open. The coordinator's own chain delivery and certificate publication also do not gate cohorts or other validators.

Curators maintain authorized evaluation collections, production histories, hidden labels, and source-group splits. Access to these materials follows the collection's permissions. Customers authorize processing of submitted documents and reference writing under their declared rights for a quoted job; buying inference gives no automatic permission to add either to training or benchmarks. Author references and derived profiles receive the same content protection as the query document.

Later *B* miners register private transformation pipelines, with language-model adapters served by Cloudflare in their own accounts and numeric rankers executed by SlopNinja. These miners maintain their delegated access, capacity, and billing, and pay for their model calls, including required evaluations. Customer payments follow the burn policy; no direct per-job reimbursement is promised. Later *C* contributors submit public grammar and lexical improvements to the grammar *B* uses for edits. Accepted contributions receive credit once under a separately activated contribution contract (Section 11). Neither *B* service obligations nor *C* rewards apply to the *A*-only launch.

2.1 Rust applications and serverless execution

All SlopNinja production application logic is written in Rust, including native miner and non-coordinator validator clients, update and migration tools, and the serverless coordinator, challenge, and inference logic. A *challenge* is a coordinator-internal evaluation task, such as generating cohort documents or running an admitted artifact on them; no challenge is sent to a miner or validator. Native participants use `rtao` for chain integration [1]; Linux and macOS releases require explicit platform qualification. Research ML backends remain separate from this production contract. Native participants retain their own hotkeys. The coordinator's validator hotkey is controlled within the serverless service, with durable signing ownership and transaction recovery. Its authorization and receipt keys remain distinct from that hotkey; other validators' keys never enter the service. High-stakes operator decisions, such as declining a cohort's result or restarting the record, are signed with a further operator-decision key that is kept offline (Section 6.6).

Miner and validator clients are to install signed updates automatically and independently. A participant that has missed several releases is to follow the supported sequence of resumable schema migrations before resuming its work. Updates must preserve unfinished obligations and reconcile chain submissions before retrying them. No participant waits for another to update or migrate. The release, migration, and conditional rollback requirements appear in Section 6.10; automatic recovery must satisfy its compatibility and journal checks before new signing is allowed. This updater is planned and not yet built (Section 14.2); until it exists, participants update by hand.

The coordinator is entirely serverless on Cloudflare Workers and Durable Objects [2,3]. Shared Rust code defines canonical messages, artifact identities, scoring, and state transitions; supported modules compile to Wasm with the required platform bindings and generated deployment glue. The full native `rtao` runtime is not assumed to compile into a Worker. Workers code follows a bounded-memory rule: it allocates only inside a fixed arena sized from measured peaks, checks every declared length against that budget before allocating, and streams large objects instead of buffering or re-reading them. The coordinator needs qualified Workers-compatible chain transport, encoding, signing, and durable reconciliation, including its validator commitments and reveal obligations. Both coordinator and native participants use the `rtao-rpc` service, which provides SlopNinja RPC access at no service charge [4]. Chain fees and execution costs remain separately funded.

Live `rtao-rpc` webhooks, offered for both Finney and testnet, deliver every finalized block. That per-block delivery is the coordinator's primary heartbeat and clock, and each delivery is its own Worker invocation [5]. Durable Objects retain the finalized block cursor, and RPC reads backfill gaps and resolve finality. A one-minute Cron Trigger [6] is the only fallback: when deliveries stall, including after the provider pauses a subscription whose retries are exhausted, it processes blocks from public OpenTensor RPC through the same per-block path. No permanently running subscriber is required. A webhook is a notification, not evidence of finality or permission to sign. Section 6.3 specifies ingress, the cursor, fallback, and the operator-resumed pause, a stated limit; the webhook receiver and the serverless chain path remain integration work.

Component	Responsibility
API Workers	Enrollment, authenticated requests and block-webhook ingress, status, and result retrieval.
Durable Objects	Authoritative cohort checkpoints, admission, reservations, accounting, finalized-block cursors, and the coordinator's chain journal and signing ownership. Each record has one transactional owner.
Lane Durable Objects	Execute fan-out lanes of bulk evaluation work. Each lane claims tasks under the cohort object's durable leases, runs in its own invocation, drives its own Container instance for model runs, and persists outcomes before releasing its claim.
Queues	Carry lane wakeups holding identifiers and generations only; durable lane records reconstruct any lost, duplicated, or expired message.
R2 storage	Versioned artifacts, evidence, manifests, and encrypted private job objects, bound to their durable records by identity and digest.
Cron watchdog	Detect stalled deliveries or cursor lag, then process finalized blocks from public RPC through the per-block path until webhooks resume.

Table 1: The serverless coordinator. Queue delivery, lane invocations, and wakeups do not own protocol state.

Task identities and dispatch intent are persisted before queue delivery, and a lane's assignment is recorded in the cohort object before its wakeup is sent. Messages carry identifiers and checkpoint generations, never customer content or credentials. A lane checks the current authorization and claims the recorded task before acting. Duplicate delivery cannot create a second accounting outcome. External execution can still repeat after an uncertain timeout; bounded retries retain every attempt and apply a declared selection rule. Queue retention is finite, so durable records must reconstruct pending dispatches after a long pause [7]. Cross-object work uses persisted idempotent messages; a transaction in one Durable Object cannot atomically update another [3].

All production inference, challenge generation and execution, and scoring run serverlessly. Model execution, and later B parsing, take place outside the objects that serialize admission and accounting. Bounded Rust/Wasm computation runs in Workers; a qualified Cloudflare serverless Container under SlopNinja's control supplies bounded computation beyond their limits, including every A model run. Workers AI supplies supported language-model and adapter formats; it cannot host the A encoder (Section 14). Bulk evaluation work runs as fan-out jobs: the cohort's Durable Object divides challenge generation, paired scoring, receipt bundling, and per-model score summaries into lanes, at least one per scored artifact and more as the batch grows (Section 6.2). Each scoring lane drives its own Container instance, and generation lanes share one rate limiter. Lane count changes elapsed time and per-lane start cost, never a document's outcome. Every workload needs measured CPU, memory, bundle, and numerical-runtime limits before admission; these targets do not imply support for arbitrary uploaded architectures. Miner-supplied executable plugins are excluded. Coordinator state and scheduling remain in Workers and Durable Objects, and production serving never depends on a miner's or validator's local machine being online.

The native adapter uses durable `rtao` journal and weight/reveal recovery APIs. It starts paused, reconciles finalized history, and requires explicit resumption with application compatibility checks. This is an integration component, not a deployed validator. Its tested scope and current limitations are recorded in the "Implementation boundary" paragraph in Section 14.2; unsupported paths require completion and qualification before deployment. The coordinator's Durable Object journal must implement the same intent, attempt, and finalized recovery requirements as the native validators' local journals; the native file-backed implementation does not supply that storage adapter. The cohort recovery contract in Section 6 applies to both deployments, including the evidence needed to resume after runtime upgrades or missed tempo boundaries.

3 Model interfaces and qualification

Launch admits one A model per entry. The model predicts AI share, production history, author identity, and AI spans in one invocation. All output heads must consume a common document encoding produced by a

shared learned encoder; each may also have task-specific layers. This requirement enforces one shared encoding interface; whether the heads use common predictive features requires measurement. Admission checks this structure in the submitted computation graph, together with one manifest and input/output contract. Packaging two independent predictors behind one API does not qualify. This checks inference structure, not whether the components were trained jointly. Known AI models have predefined author profiles alongside human writers. Detecting AI involvement is the primary purpose of the origin output and the only reward quantity; the other outputs are qualification gates. The transformation mechanism B is deferred to Section 10; C's proposed contributions to the shared grammar that B uses appear in Section 11.

The shared encoder is intended to let the two tasks improve each other. Learning author- and generator-specific writing patterns can help identify model involvement. Production-history supervision can help attribution distinguish a writer's habits from patterns introduced by generation or editing. Both objectives can therefore shape features useful to the other. This expected mutual benefit requires measurement: sharing an encoder alone does not establish it. Section 15 requires each new architecture to be tested in both directions against models trained for each task alone. The requirement deliberately gives detection, attribution, and later author-fit feedback one reusable learned representation. Graph inspection cannot establish useful feature sharing; disjoint slices could satisfy the structural rule. Measured harm would prompt an explicit revision of the design before launch, rather than an undocumented exception to admission.

An A entrant submits the weights and all data needed for the declared inference calculation. The artifact stays private until its cohort's scoring starts and is published then (Section 3.4). Miner submissions are licensed CC BY-SA 4.0, so anyone may obtain a published artifact free of charge and execute it locally, and later B development is expected to train against published detectors. A published artifact stays public; the operator removes one only on legal grounds, as a recorded removal (Section 3.4). The coordinator deletes its private admission copy once the cohort closes and publication is recorded, and keeps the current and previous reference artifacts hosted for evaluation while they are references (Section 6.8). Hosted execution through the paid APIs is charged separately. Miners may keep their training recipes and permitted training corpora private.

Project code and documentation are licensed MIT OR Apache-2.0, except files and datasets that carry their own licence or notice. Reference weights are licensed CC BY-SA 4.0, like miner submissions. Every allow-listed graph class is built on ModernBERT-base, licensed Apache-2.0, and an artifact carries its tokenizer and, when trained from its weights, a modified copy of them, so that licence's text and notice are to be published beside the artifacts. SlopNinja publishes its own references, R_0 and any later reference it trains (Section 5.6), on the same terms. A public reference package carries no evaluation text: its replay cases are synthetic rather than development documents, and the package passes a scan for corpus text before release. R_0 is published once its package has been re-exported in this way. Miners can also start from a public training recipe [8]: it fine-tunes ModernBERT-base on about 4,200 Train rows from five redistributable dataset snapshots and packs and checks an admissible artifact with the miner client. It reproduces R_0's pipeline and artifact format, not its V6 starting point, data, or weights, and is not a strong detector. An entrant artifact that becomes the reference was already published with its cohort. SlopNinja executes submitted artifacts for the pilot benchmark and published, qualified artifacts for customer service; an A miner needs no private serving account or delegated inference credential.

3.1 AI detection through origin and attribution

The origin output is a calibrated probability distribution over three recorded production histories: human-only (p_{human}), model-only (p_{model}), and mixed human/model work (p_{mixed}). The artifact combines model-only and mixed probabilities to answer the primary AI-involvement question. It reports a continuous probability, displayed as a percentage:

$$p_{\text{AI}} = p_{\text{model}} + p_{\text{mixed}}, \qquad \texttt{ai_confidence_percent} = 100\, p_{\text{AI}}.$$

Binary training labels are zero for human-only documents as the frozen labeling policy declares them, whether documented human-only work or a historical-human proxy, and one for model-only or mixed work. The model learns a probability from those labels; it does not convert a hard decision into a percentage. Documented three-way histories additionally supervise the origin components. The three probabilities sum to one, and their calibration method and workload profile are frozen with the artifact. Every completed inference returns numeric share and confidence scores. An unavailable prediction returns an explicit failure status, never zero scores. Fixed thresholds remain useful for benchmark operating points but are not required public outputs.

The second percentage is estimated AI share: the expected fraction of final words in model-written or model-rewritten spans under the provenance convention in Section 4. Write that fraction as f_{AI} and its predicted expectation as $\hat{f}_{\mathrm{AI}}$:

$$\texttt{ai_share_percent} = 100\,\hat{f}_{\mathrm{AI}}, \qquad 0 \le p_{\mathrm{model}} \le \hat{f}_{\mathrm{AI}} \le p_{\mathrm{AI}} \le 1.$$

These bounds follow because human-only work has zero share and model-only work has unit share under this convention. One implementation uses $\hat{f}_{\mathrm{AI}} = p_{\mathrm{model}} + p_{\mathrm{mixed}}\mu_{\mathrm{mixed}}$, where $\mu_{\mathrm{mixed}} \in [0, 1]$ is the learned expected share conditional on mixed history and the document. Binary labels supervise confidence; measured fractional labels additionally supervise share. A result can therefore report 20% estimated share with 99% confidence that AI contributed. Confidence is the probability of involvement, not certainty that the share estimate is exact; uncertainty intervals for share require separate validation. Both percentages must qualify on their declared workloads before paid service.

Attribution reports conditional results for both human and AI-model roles. Each result names supported gallery matches, or returns *Other human* (`other_human`) or *Other AI* (`other_ai`) when no named profile for that role satisfies the frozen matching rule. The question is which contributor fits *if that role contributed*; a conditional match or fallback does not itself assert that the role is present. Report its involvement probability alongside it:

$$p_{\text{human involved}} = p_{\mathrm{human}} + p_{\mathrm{mixed}}, \qquad p_{\text{AI involved}} = p_{\mathrm{AI}}.$$

These probabilities may sum above one because mixed work includes both roles; they are not shares of the text. No presence cutoff suppresses a role's attribution. Both fallbacks can therefore appear for mixed work, including when neither contributor has a named match. Multiple contributors need not be forced into one identity.

The fallback categories are outcomes, not enrolled profiles or assertions that an identity is absent from the gallery. A listed writer or model may be unidentifiable from the available text. There is no separate unknown or attribution-abstention label in a completed result; failure remains separate. Supported named-model matches remain visible even at low AI-involvement probability, without changing that probability or proving authorship. The output contract fixes these meanings and their calibration before evaluation.

Origin and attribution answer related but different questions. An unseen generator may receive high AI confidence and *Other AI* attribution. Failure to match a known model does not establish human origin. A known writer's style may survive AI assistance, so a human-profile match cannot override the origin estimate. When a model rewrites human-only text, the resulting work has mixed history. When one model rewrites another model's draft, it remains model-only under the initial label policy, with attribution potentially unresolved. The outputs estimate the recorded workflow and its contributors; they do not prove authorship.

Independent reference works form the gallery. Its construction, pooling, reference/query roles, and version are part of the inference contract. The interface should permit enrollment of a new human author from independent samples without fitting another permanent identity classifier. Profile IDs identify output matches and gallery entries; they are excluded from query features.

The main intended human-attribution workflow is personal: a customer supplies their own writing samples to construct a private writer profile. A qualifying match returns *Provided writer* (`provided_writer`), bound to that customer's selected reference profile. This is a named match under the normal matching rule; if no human profile qualifies, return *Other human*. It can accompany AI involvement and identifies resemblance to the supplied writing, without authenticating the customer's identity or proving authorship. The profile is scoped to the request or customer account and is never implicitly enrolled in the public gallery. Operator-maintained human profiles may also use documented public or licensed references under a published enrollment policy recording provenance, the identity basis, and permitted uses. Public availability alone does not settle those uses.

In this later workflow, trusted preprocessing represents each reference work by A's encoder output and by stylometric features, function-word frequencies and character n-grams, then pools them into one writer profile under a qualified, versioned pooling contract. A normalized weighted mean is the initial candidate; the contract fixes feature scaling, per-work weights, missing-channel handling, and the compatible encoder, feature, and pooling versions. The matcher receives the profile instead of raw reference prose. A pooled summary need not represent any single work, so qualification must test whether averaging loses useful variation across topics or registers and whether sample length or an outlier dominates the result. Raw samples are still read during trusted preprocessing. Both samples and profiles receive customer-content protection; a profile is not an anonymization method.

The same fixed profile is the intended target for later B revision using A's continuous author-fit feedback (Section 10.1).

No profile type yet qualifies to back *Provided writer* or that feedback. In an October 31, 2026 comparison on writers neither system had seen, unlearned profiles from R_0's encoder beat CFG writer vectors on verification, stylometric features beat both, and adding CFG to the encoder plus stylometry gained only $+0.0015$ in AUC, so CFG writer vectors are not a profile channel. Encoder plus stylometry was the strongest profile, yet at five references 42% of impostor trials still passed when 90% of genuine ones did, and no profile ranked rewrites toward a target writer much above chance (Section 14.1). The next qualification is a learned matcher trained on fresh authors, with its acceptance thresholds measured on withheld writers and on real style-transfer data. Until a profile type passes it, neither *Provided writer* nor B's author-fit feedback is offered.

Open-set rejection is evaluated on withheld writers and generators, separately from choosing among known profiles. Rejection means a completed attribution decision that accepts no named match for the evaluated writer or generator role, including *Other human* and *Other AI*. Evaluate each documented contributor role, including both roles in mixed work, regardless of its predicted probability. Credit successful rejection only when the ground-truth identity is absent from the gallery and the corresponding fallback accepts no named match for that role. A wrong named match is an attribution error; fallback on a listed identity is not successful open-set rejection. Missing role results and execution failures remain in the assigned population and earn no rejection credit. Do not score conditional identities for roles absent from the documented history; evaluate mistaken involvement probabilities through origin calibration and false-positive measurements. This role-level rejection rule covers one contributor or a role whose contributors are all withheld. A role containing both known and withheld contributors needs a frozen set-valued attribution metric before earning rejection credit; retain such cases in coverage and report them separately. Launch qualification covers English documents. Multilingual claims and additional document-length classes need their own data and workload evidence.

3.2 AI span localization

The goal is to locate AI-written text, not only to score whole documents. Sentences are one view of that location. The output contract therefore adds a token-level head on the shared encoder. It returns an AI probability for every token, decodes those probabilities into AI spans with exact character offsets into the canonical text and a confidence for each span, and derives per-sentence AI share and AI-involvement confidence from the same token probabilities. The document-level outputs keep their meanings, and the span outputs are made to agree with them by construction. In the runtime's emission, a word's probability is the mean probability of its tokens, and a share tilt shifts every word's logit by one common bias so that the mean word probability equals the emitted document share. A sentence's share is the mean tilted probability of its words, so sentence shares aggregate to the document share by final-word counts over complete, nonoverlapping partitions. Document involvement confidence cannot be assembled that way, because neighboring sentences are dependent: neither averaging sentence confidences nor multiplying their complements yields a document probability. It remains the document head's calibrated output, and sentence and span confidences are capped at it.

Span localization covers AI-written spans of at least 24 words. A shorter edit, such as a replaced word, a clause, or one short sentence, carries too little signal to locate reliably: the release's initial frozen-encoder span head of October 31 reached character F1 of only 0.05 on clause edits and 0.17 on single-sentence edits (Section 14.1). Under the current rule, spans under 24 words are therefore not trained on, evaluated, or decoded; the first span package's weights predate the rule, as described below. Training masks the tokens of a shorter model-produced span out of the span loss rather than relabelling them human. Evaluation removes shorter gold spans from the recall denominator, counts their characters neither as true nor as false positives, and ignores an unmatched decoded run that lies mostly over such words. The decoder never emits a span under 24 words. A shorter edit is still AI involvement: every model-produced word counts in the document's share and involvement labels, so the floor changes only what the span output must locate. The floor is a versioned rule bound into the training protocol, the evaluation, and the decoder.

The training labels already exist: retained exact model-produced spans for mixed documents, all-AI labels for model-only documents, and all-human labels for human-only documents (Section 4). Qualification measures, under the floor, word-level Brier score, character precision, recall, and F1, span precision and recall, with decoded runs matched one to one to gold spans at an intersection over union of at least 0.5, recall in span-length bands of 24–49, 50–99, and 100 or more words, the number of human documents that carry a decoded span, and the

calibration of span and sentence confidences. Agreement with the document outputs needs no gate, because the emission guarantees it; the gap between mean word probability and document share before the tilt is reported as a diagnostic, since the tilt costs word-level accuracy wherever the document share is the less accurate estimate. The span gate includes the in-context false-positive rate, the share of human characters inside mixed documents that decoded spans cover, counted separately from the pure-human document gate, because identical human words score higher next to AI text than alone (Section 14.1).

Spans are part of every entry. From the first cohort manifest that enforces the span gate, every entrant carries a token-level head whose decoder never emits a run under 24 words: artifact spec v5 refuses any other artifact at admission, and a manifest that enforces the gate under an earlier spec fails such an entrant at scoring. The scorer requests spans for every document it scores in such a cohort, and an entrant that fails the gate earns no credit (Section 5.9). The gate compares each entrant with a fixed anchor, SlopNinja's first span package, on the same cohort batch under the 24-word floor. Over mixed documents whose model-produced span has at least 24 words, the entrant's mean character F1 may be at most a fixed margin below the anchor's, and its in-context false-positive rate at most a fixed margin above it; over the batch's human-only documents, the number on which it decodes a span may exceed the anchor's by at most a fixed margin. A document whose prediction or span emission fails counts as decoding nothing in a mixed document and as carrying a span in a human one. Character F1 alone does not separate localization from marking whole documents: on spliced documents whose rewritten share is large, one span over every involved document reaches a high F1 while covering all the human text beside it, so the gate checks the in-context rate with it.

The margins are fixed after measurement on the sealed batches of closed testnet cohorts. The design target is that floor-compliant heads as good as SlopNinja's own, trained with other seeds, with another recipe, or on public material only, fail in at most about one cohort in 365×256, a modelled rate rather than a guarantee, and that heads placing spans by position or document share alone do not pass; the margins are rounded in the miner's favour, so the gate is a floor against unusable outputs, not an improvement target. The manifest binds the margins with the anchor's identity. If the anchor fails to emit spans or to separate from such trivial heads on a cohort's batch, or the cohort's population differs from the one measured, the measured conditions decide no entrant in that cohort, and its public record says so; the operator may also void the gate for one cohort, with a recorded reason, before its packaging starts. Neither waiver holds a package, and no miner can trigger one. Before any manifest enforces the gate, it runs in report mode, measured and published without failing anyone, on at least three live cohorts, at least two of which complete after the tools miners need to check their heads against the anchor are public. Span outputs carry no reward weight: document involvement Brier remains the only reward quantity, and enforcing the gate does not by itself change the reference (Section 5.6).

SlopNinja's first span package under the 24-word floor, which replaced an earlier package with the same S2c weights and a decoder that allowed 10-word spans, keeps R_0's encoder and document heads frozen, so on the same CPU class its document outputs are bit-identical to R_0's, and confidence, share, and reward scoring do not change. Its token head, S2c, reads a trained copy of the encoder's last four layers and final norm, about 20 million parameters (Section 14.1). Its weights were fitted before the floor; a retrain that masked shorter spans matched them within seed noise and was not adopted. Its decoder emits only runs of at least 24 words, and its span confidence map is fitted on the Calibration split under the floor. Retrospectively, on the consumed Test, it reaches character F1 0.544 on mixed documents that contain a span of at least 24 words, recovers 36%, 44%, and 56% of gold spans in the three length bands, and puts a span on 151 of 6,132 human documents, 2.5%; on held-out spliced rewrites its character F1 is 0.639 (Section 14.1). The tower adds 18–20% of the encoder's CPU time, and its graph forms a separate allow-listed class with its own parameter bound (Section 3.4). Since November 9, 2026 it is SlopNinja's span-serving package, the model its public detection pages are to run first (Section 8.1). Because its document outputs are R_0's, this package is to take R_0's position on testnet before the first cohort that enforces the gate, by a signed reference decision (Section 6.6) that leaves the record without a predecessor and changes no score, margin, or cutoff; it is not the planned replacement by a retrained reference. Before that decision it runs once as an ordinary cohort entry, so that, as for any reference, its artifact is published, its content digest enters the published-artifact index, and its probe fingerprint is recorded. The artifact spec that testnet cohorts name today checks only that a decoder's minimum span length is at least one word; artifact spec v5 refuses an artifact without a token-level head or with a decoder whose minimum span is under 24 words. Spec v5 and the gate's report mode are built but not yet deployed, so no cohort has yet evaluated spans (Section 14.2).

A further extension could identify words and constructions that materially influence a score, starting from the token-level outputs. Test explanations by removing or changing the cited text and measuring the model's response with meaning and context controls. Require stability across innocuous formatting changes. Explanations in terms of grammatical constructions belong to later *B* revision guidance, which uses the shared grammar. An influence on a model score is evidence about its calculation, not proof that a word or construction was produced by AI. Attention weights or a generated explanation alone do not establish that influence. These explanations are a separate future qualification, not an initial launch requirement.

3.3 *A* input contract: raw text

The *A* input is the canonical document text. A miner artifact is a text model: an ONNX computation graph, a fixed tokenizer, the output heads, and their calibration. The coordinator tokenizes each document with the allow-listed tokenizer, which the artifact's own must match byte for byte, and runs the graph on CPU in a SlopNinja-controlled Cloudflare Container under the cohort's declared length and resource limits; an input beyond the artifact's limit returns an explicit unsupported-length result. Cohort batches stay within the declared profile. Until a manifest enforces the span gate, an entrant with document heads only is admissible; from that manifest on, every entrant carries the token-level head (Section 3.2). The reference R_0 is a ModernBERT-base encoder [9] of about 150M parameters with a 2,048-token limit, fine-tuned from the earlier V6 detector on raw tokens, with origin, share, and attribution heads (Section 14); the release package adds the token-level span head. Its architecture, with or without a token-level span head, defines the graphs that admission allows (Section 3.4).

A uses no features derived from a resolved context-free-grammar (CFG) parse. A CFG-only detector failed its first real-data test: on an October 26, 2026 research Test of 102 documents (splits as in Section 4.1), CFG-only shared models had AI-involvement Brier 0.126–0.134 against 0.096 for a Calibration-mapped Pangram comparator on the same documents, and flagged 67–87% of human documents at the 0.5 cutoff [10]. Adding CFG coordinates to frozen text features gave only a small, uncertain Development gain, 0.048–0.057 against 0.052–0.058 for text alone [11]. Those tests used pooled grammatical-leaf counts; ordered and tree-aware CFG inputs remain untested. Parsing and ambiguity resolution also add latency and cost, and numeric features carry information about the source, so they provide no confidentiality guarantee.

The grammar's planned uses are *B*'s edit representation, which types edits and places a revision alongside its source (Section 10), and *C*'s contributions to it (Section 11); neither depends on *A*'s input contract. Any change to that contract, including grammar features, would need a new paired comparison against raw-text models, including its added cost, before admission (Section 15).

3.4 Artifact submission and execution policy

An execution manifest fixes the whole calculation: weights, input/output schema, feature meanings and order, normalization, availability masks, serialization, calibration, reference pooling, numerical precision, rounding and tolerances, admitted operators, dependencies, resource bounds, and reference outputs. It also fixes the tokenizer and the canonical text profile. Names, handles, and other identity-bearing material follow the profile's declared policy. Explicit author IDs, miner UIDs, assignment metadata, hidden labels, and source/candidate-role flags are excluded from learned query inputs.

Models within a comparison receive the same accepted text. A change to the input profile begins a new comparison and requires requalification; it cannot silently alter an unfinished cohort. Weight, calibration, or other inference-semantic changes create a new artifact version. Compatible operational repairs may preserve those semantics under the cohort rules in Section 6. In particular, replacing an execution host must preserve the artifact and calculation rather than substitute a different detector.

The coordinator caches the cohort batch, executes qualified artifacts under equal limits, and retains their predictions and failures. Centrally executing submitted artifacts keeps hidden benchmark inputs out of miner request paths and limits direct benchmark harvesting. Model isolation and operator access controls remain necessary. Publishing each executable when its cohort's scoring starts permits inspection and independent experiments. Validator duties follow Section 2; the scoring trust assumption is stated in Section 13.

The manifest's allow-list names three graph classes, each with matching operators and shapes and integer arithmetic on token identifiers only in the embedding lookup: the reference's ModernBERT-base architecture; the

same architecture exposing its final-norm token states to a token-level span head; and a tower class that adds trained copies of encoder layers 18–21 and the final norm for that head (Section 3.2). Each class has its own parameter bound, derived from its structure: 160 million parameters for the first two classes and 173.5 million for the tower class, against 150.3 million in R_0's package and 170.6 million in the first span package. An artifact's tokenizer must match the allow-listed tokenizer byte for byte. Its calibration, contract, and other declared files must validate against fixed schemas for their roles, such as a monotone piecewise-linear calibration map whose knot count the schema bounds, and a file without a declared role is rejected. A class fixes its graph file's length, about 601 MB for the first two classes and 681.5 MB for the tower class, and every other file is bounded by its schema or by the runtime that reads it, so no separate byte cap applies; R_0's artifact object is 610.6 MB. The scorer serves all three classes, and testnet cohorts admit them under the current artifact spec [12]. Where a limit is needed it derives in this way from structure, such as the allow-list, a schema, or a measured bound, and is versioned in the manifest rather than hard-wired, because a fixed limit that later models outgrow can strand a subnet. The allow-list widens only through a versioned manifest. The bound limits what an artifact can memorize, an estimated 40–72 MB of learned capacity at 160 million parameters and up to about 78 MB in the tower class, and an embedded index or lookup table of text fails admission (Section 5.4). The scorer has no network access, online learning, cross-job prompt cache, or undeclared mutable state. Qualification exercises resource bounds and numerical boundary cases on the actual admitted runtime. Linux and macOS tooling may inspect and prepare artifacts, but outputs from another backend become interchangeable only after numerical qualification.

Qualification uses independently labelled hidden material and includes formatting variants, identity and topic leakage, targeted triggers, chosen-input failures, exact or near-copy submissions, and batch invariance: each document's outputs must be identical whether it is scored alone or among its batch. Canonicalization does not replace these controls. A publicly reproducible calculation can still exploit a benchmark shortcut or favor a collaborator. These checks inform qualification and continuing evaluation without establishing universal robustness.

Artifact objects. An artifact object is a length-prefixed header of at most 1 MiB, listing each file's path, role, SHA-256, and length, followed by the files in path order. The graph, weights, tokenizer, calibration, and output-schema files form its inference content; reference test vectors, owner metadata, and publisher signatures do not. The inference-content digest covers the frozen contract and each inference file's role, digest, and length, so renaming or re-signing an artifact cannot create new inference content. Two inference files with the same role, digest, and length are refused. The object is the miner's local file and its published form, but it is uploaded and published as the header and each distinct file separately, from which anyone can rebuild it. The miner hashes each file when it packs the artifact, and R2 computes each object's SHA-256 as it arrives to check the declared digest; no SlopNinja code hashes a model byte after packing: the graph's class is checked by comparing its structure, the graph file without its weight values, byte for byte with the allow-listed class's pinned reference bytes [12].

Private submission and duplicate credit. A miner commits on chain only a digest. Through the chain's commitment call, it publishes a domain-separated commitment binding the chain and subnet, the cohort, its hotkey and registration generation, the inference-content digest, and a fresh secret 256-bit nonce. The commitment binds the inference content, not the object's bytes, so objects that differ only in excluded files open the same commitment. Before the admission cutoff the miner opens a signed admission session with its commitment opening and the header's length and SHA-256, then uploads the header and each file straight to the coordinator's private R2 admission storage, one presigned upload per object [13]. Each upload URL binds the object's key, its declared length and SHA-256, and create-only semantics, so R2 verifies the bytes as they arrive and stores nothing on a mismatch; no upload passes through a SlopNinja Worker. The header's length and SHA-256 are declared in the signed session request, and each file's are listed in the header. File URLs are issued only after the header has arrived and the coordinator has recomputed the inference-content digest from it and checked it against the committed one, so the commitment pins each inference file's digest and length; the header's own digest and its entries for excluded files are declared, not committed. Admission, publication, and scoring then trust R2's stored checksums and sizes and never hash model bytes again. TLS and the bucket's access controls protect an artifact until publication; no further encryption layer is applied, because R2 can verify only the bytes it receives. The coordinator checks that the opening opens a finalized commitment for this cohort on this chain, made by the hotkey and registration generation the opening names, registered no later than the commitment block, that R2's stored digests match the header, and that the artifact qualifies. That commitment's finalized block and extrinsic index fix the entrant's admission order. A commitment without a matching upload reserves nothing. Artifacts stay private evaluation material until the cohort's scoring starts, after its roster is frozen and its batch sealed, so

no other participant can download, run, or copy a model before it is entered; a detected copy entered in a later cohort earns nothing (Section 5.10). When scoring starts the coordinator publishes every admitted artifact, header and files each named by its SHA-256 and copied inside R2 or by a stream that R2 verifies. The commitment openings are published when the cohort closes. Other validators need none of these objects to commit: they need not download or run the models.

A published artifact stays public. The operator removes published material, meaning an artifact's header or files or a reference package and never a result, opening, or record, only on legal grounds, such as a valid copyright notice, a court or authority order, or a personal-data request that applies to it, and never silently: each removal is a recorded operator action with a content-free public record of the date, the material's digests, what was taken down, and the legal basis by type and jurisdiction. A removal deletes only the hosted public copies and cannot reach copies already downloaded. It changes no commitment, opening, score, quantized result, or signed row, so the target in force and every reward paid or still due under it are unchanged. The published-artifact index keeps the removed artifact's digest, so an entry with the same inference content is still never executed; if such an entry is published again, it is removed the same way, with its own record. The coordinator keeps its private evaluation copy of a removed reference. If the legal basis also bars running a removed artifact that is a current or previous reference, or the package the detection service serves, the operator stops using it through the paths of Section 6.6: a reference decision for later cohorts, and an abandonment of a running cohort that cannot finish without it; the detection service stops serving that package.

Within a cohort, identical inference content, or predictions within the near-copy distance on the cohort batch, forms one cluster that receives one credit split equally among its members (Section 5.9). The commitment nonce is excluded from content identity. The finalized commitment block and extrinsic index, the admission order, choose which member heads a cluster, whose quantized score then places the whole cluster, and break ties between equal quantized scores, and so choose which of the tied cluster heads can become the next reference. This order assigns no authorship: it cannot establish who originally trained a model. Already-published artifacts may enter but earn nothing (Section 5.10), apart from those of a cohort abandoned after its scoring started (Section 6.6). Permitted foundation-model components may be used in new models, and a weight soup of published models, which fits the allowed graph, is a new model.

4 Training data and evaluation separation

The data model separates authorship from production history. A third record type supports later transformation research:

Record	Material	Supervision
Authorship	Independent works and reference/query roles	Writer or model identity, work family, date, genre, label strength
Production	Originals, generations, and revision chains	Workflow, generator version, parent links, assistance order
Revision preferences	Sources, briefs, edits, and reviews	Preservation, target voice, editorial preference, ties

The first two records supervise the shared A model. A production label comes from documented creation and editing history. A publication date alone, a detector's opinion, or a request to imitate an author does not supply that label. Model-only text written in a named human's style remains model-only; style resemblance supplies no authorship label. Mixed histories retain the order and type of assistance. Uncertain historical-human proxies remain identified as such in each evaluation, rather than becoming verified human examples through repeated reuse.

Controlled partial rewrites can also supply a measured extent label. Select sections of a documented human-only source, replace only those sections with recorded model outputs, and retain the remaining text byte for byte. Under a fixed word-counting rule, divide the final words in model-produced replacement spans by all final-document words. Record exact span boundaries, original and replacement lengths, model versions, prompts, and the assembly procedure. The realized fraction can differ from the requested fraction when a rewrite changes

length. Words copied within a model-produced span count under this provenance convention; the label does not measure semantic dependence or editing effort. It is distinct from the binary label for any AI involvement. A complete model rewrite of human text can have 100% replacement-span share while retaining mixed production history. An unchanged copied passage can also have different recorded histories, so exact provenance labels need not be recoverable from the final text alone. Documented untouched human examples supply zero-share targets; complete generation records with every final word retained from model output supply one-share targets. Intermediate shares require the span evidence above. The same exact spans supervise and evaluate span localization where they reach 24 words; shorter ones still count in the share and involvement labels but are masked out of span training and scoring (Section 3.2).

Vary replacement size, position, topic, generator, and editing intensity, including dispersed edits and complete rewrites. All variants of one source remain in the same split. Control for splice artifacts and formatting cues, and evaluate on unseen sources, generators, and rewrite procedures. These examples could supervise an extent estimate; they do not establish that it generalizes to arbitrary human/AI revision histories.

4.1 Splits and exposure

Training, public development, private evaluation, and customer references are separate collections. Experiments use four fixed splits: Train fits weights, Development chooses recipes and settings, Calibration (CAL) fits only the map from raw outputs to calibrated probabilities, and a held-out Test stays sealed until the final comparison. Assign splits before fitting vocabulary, feature selection, calibration, or model weights. Group authors and source families before assigning their works. Every excerpt, translation, revision, and generated descendant of a source remains in its declared split. A common prompt, source document, or editorial pipeline can create dependence even when the resulting texts differ; source manifests record those relationships for group-aware evaluation.

Evaluation writers have independent reference and query works. Claims of chronological transfer require earlier references and later queries with usable composition dates. Held-out writers test gallery enrollment and open-set rejection. Origin testing also holds out generator and prompt families, and attribution testing includes previously unseen generators. Match subject, genre, length, and assignment format across origins. Cross-topic comparisons test whether an apparent writer signature survives a change of subject rather than merely identifying the material that writer discussed.

For *B*'s grammar-based pipeline, grammar development and parse-selection tuning count as exposure. Documents used to choose productions, select derivations, or tune a resolver cannot also serve as untouched final-test evidence for that representation. Freeze the parser, resolver, projection, and feature catalog before the held-out test. The same principle applies to held-out contribution tests for *C*. Reusable lexical resources do not exempt a fitted pipeline from recording what evaluation material influenced its choices.

The gallery has its own version and exposure controls. Reference-profile IDs identify the candidates to match, not privileged coordinates in a query. Names, handles, quotations, shared templates, and editorial signatures need the declared identity policy and leakage controls. Distinguish an original writer from a translator, shared account, coauthor, or quoted source before assigning a single-author label. Author uncertainty can coexist with a usable production-history label, and the dataset should preserve that distinction.

4.2 Evaluation and reuse

The coordinator qualifies each submitted artifact on independently labelled hidden material. Origin and attribution measurements use their declared populations, including identities absent from the gallery. Thresholds, comparison tolerances, source-group weights, and workload limits are fitted or chosen outside the final test. Retain failed assigned items in the evaluation accounting and report coverage for the requested population. A common input failure must receive the same treatment across models rather than produce a different successful subset for each entrant. Each cohort's batch is fresh. A passage of fresh human stock is scored as a pure-human negative at most once; otherwise it may serve only as a rewrite source, in its own cohort or later ones, under the reuse rule of Section 5.2. Testnet cohorts without a working outside entrant instead redraw already-exposed documents (Section 5.3).

Formatting controls are necessary even when inputs have a canonical byte profile. The V6 leading-space probe reported in Appendix A used exposed training material and found a formatting shortcut. Include paired variations that preserve the claimed label, and test identity leakage, targeted triggers, and source-format confounds. Upstream preprocessing, and for later *B* parsing and resolution, see prose and need their own contamination checks. Hiding assignment metadata from the numeric head does not establish that upstream processing was blind to benchmark cues.

Active evaluation material has no general training release. Retirement and source authorization precede reuse, including reuse of later *B* candidates or adversarial examples. A low detector score does not turn a model-assisted revision into a human-only example. Preserve its complete production history and include difficult human examples as well. Store exact provider observations and query counts, including failures, separately from the independent labels. Agreement with Pangram cannot supply ground truth.

Customer documents, references, profiles, and outputs are never reused automatically for training or benchmarks. Each use needs separate permission. Permission for a bounded inference job does not authorize a training release. An artifact license also grants no rights to its training texts. Source records must distinguish rights in prose, model outputs, and associated data from the license on code that downloads or processes them.

4.3 Acquisition and later revision data

Global Voices is a prospective source of attributed prose under its default CC BY terms, subject to writer/translator and quotation checks [14]. PLOS articles offer factual material and citation constraints; coauthorship limits their use as single-writer examples [15]. Acquisition qualifies rights and labels item by item. Existing Blog Authorship Corpus experiments remain local, noncommercial research and supply no commercial starter data. Restricted private research material is excluded from releases. Project policy permits publishing Pangram row-level results, hosted and open-weight model outputs, and fifteen Creative Commons-licensed collections, each with its licence notice. FineWeb page text, Reddit, Amazon reviews, Ubuntu IRC, RAID, and SBCSAE text are not published, and Common Crawl text is never redistributed. The acquisition plan [16] records screened sources and outstanding qualification.

The lookup canary funds about 1,000 freshly written anchor documents a year from paid writers on a keystroke-logged editor, one source group per writer, plus PACER purchases of US-government court filings, within a human-data budget below $10,000 a year (Section 5.4). Anchors stay private evaluation material and are never scored in a cohort. A separate stronger-label collection would commission 50 writers, each providing twelve independent pieces and two edits of supplied drafts. Shared briefs enable comparisons on similar topics; multiple topics per writer test transfer. Collection would record collaboration and AI assistance and obtain separate rights for training, private evaluation, and redistribution. A larger 500-writer target would allocate 300 to training, 100 to development, and 100 to private evaluation, with further withheld writers needed for stronger open-set rejection evidence. These two collections are unfunded options. Neither has been commissioned, and launch does not depend on their counts.

Permitted sources can also produce model-only and mixed examples while retaining exact prompts where known, model revisions, outputs, and human editing records. Later *B* research adds linked source, brief, candidate, and review records. Synthetic restoration pairs require a reviewer to check that the generic draft retains the information needed to reconstruct the original; otherwise the target would reward invention. Keep this weak supervision separate from writers' own edits. Any later training tickets, adversarial revision releases, or transformation benchmarks must preserve these splits, production labels, and permissions.

5 Benchmark scoring and native rewards

The launch rewards submitted *A* artifacts under coordinator-led scoring. SlopNinja runs each artifact, the current reference model, and, when one exists, the previous reference model on the same fresh hidden batch, retains every prediction and failure, and keeps every exact score inside its evaluation service. It releases only quantized scores (Section 5.7), derives the canonical native row from them under the frozen `reward-v2` rules below, or under their successor, which changes only how a cluster is placed (Section 5.9), signs that row, and publishes it off chain. Every other validator verifies the signed row and submits it through its own journaled commit–reveal, in parallel with the other validators (Section 6). Held-out documents, labels, individual predictions, and exact scores never

go to miners, other validators, or the public; only the hosted generation providers of Section 5.2 receive held-out passages. Other validators trust the coordinator's execution, labels, and arithmetic; anyone can recompute the row from the public quantized result, but that check is advisory and never changes a committed row. Native consensus determines the eventual payouts [17, 18].

`reward-v2` ports the band mechanism of Bittensor subnet 78 (Umi) [19]: geometric weights across score bands and within each band, a baseline that ratchets only on improvement, and a largest-remainder integer row. Its principal departures are these: artifacts stay private until the cohort's scoring starts; the cutoffs, bands, credit, and the row compare quantized scores released under a per-cohort privacy budget (gates and cluster membership use private statistics), equal quantized scores share credit, and subnet 78's near-equal drop is not used; eligibility requires a margin over the reference rather than subnet 78's 1% grace below the incumbent; a fallback cutoff, a SlopNinja rule with no subnet 78 counterpart, can keep rewards flowing when nobody reaches that margin; a detected copy of any earlier published artifact earns nothing, and near-duplicate entrants share one credit equally; and no record, identity, or gate carries a chain runtime version. Its predecessor `reward-v1`, which the rehearsal cohorts of October 31 to November 1, 2026 ran (Section 14.2), applied the same band rule to exact scores and published them; the testnet record restart of November 6 installed `reward-v2` (Section 6.6). The shared protocol crate implements these rules as exact computation [20].

5.1 Freeze the comparison before execution

A cohort manifest binds its roster of artifact commitments, the raw-text input profile and output contract, the reference gallery, the reference models, the batch design, the human pool, qualification gates, numerical tolerances, the parameter bounds and compute limits (Section 3.4), retry policy, the cohort schedule (Section 6), and the `reward-v2` and `quant-v1` parameters, including the privacy and bit budgets. The batch design fixes n, the mix, the domain×origin strata, and the sealed batch digest. The human pool enters through its frozen pool manifest and, for the mainnet stock, its stratum share caps and a commitment to its secret sampling key (Section 5.3). The manifest also binds the index of published artifacts and the fingerprint probe versions in use (Section 5.10). Every model in the comparison, including both references, receives the same canonical text for each document. Hidden labels remain outside model inputs.

One model version per miner enters a cohort. On-chain digest commitments and private uploads have an admission cutoff (Section 3.4); accepted cohorts have no completion expiry. The credited identity is the original hotkey and its registration generation, which survives UID changes. Renaming metadata cannot create new inference content.

The coordinator caches admitted artifacts and the cohort batch before execution. Runs have no online learning, network access, or undeclared mutable state, and each document is scored independently of the others in its batch. Every assigned item receives an outcome record. Infrastructure errors, model errors, invalid outputs, and unsupported inputs remain distinguishable. The comparison cannot select the easiest successful subset for each entrant or its best answer across retries.

5.2 The cohort batch

Each cohort draws a fresh hidden batch, and every entrant and both reference models score the same documents, so every comparison is paired. The batch has about 1,000 cases on testnet and 4,000 on mainnet, with 2,000 as the mainnet floor. Its mix is 20% human-only, 55% mixed, and 25% model-only documents. Fresh AI-side text, both partial rewrites and model-only mirrors, is generated for each cohort by hosted models, under a per-cohort budget on testnet, by parallel generation lanes that share the providers' published rate limits [21] (Section 6.2). Half of the generated text comes from frontier models, to which the coordinator has access through configured credentials, with requests spread equally across the effort or reasoning levels each model offers, and half from a spread of open models on Workers AI. Each request draws a voice profile from a versioned library or, for a quarter of requests, none, leaving the model's default voice. Model, span length, edit intensity, effort, and voice rotate per source, and each document records them. Held-out passages may be sent to these providers under their own data policies; miners, validators, and the public never receive them.

Whole source groups are sampled, and the design is self-weighting over domain×origin cells: every document carries equal weight, and no Kish design loss arises. The mix is therefore part of the estimand. Under 20/55/25, human-only documents carry about 21% of the reference's Brier loss, against 43% under a 40/40/20 mix, so the

human share sets the reward's weight on human false positives. Holding a class-balanced estimand fixed at this human share would instead raise the paired standard error by 17–21%. A model's *headroom* is one minus its score, its remaining Brier loss (Section 5.5); margins, band widths, and gains are stated as fractions of the reference's headroom. Three headroom figures appear in this paper, each a different estimand computed from the same R_0 predictions on the consumed Test: 0.0518 is its balanced involvement Brier under the Test's own pre-registered document weights (Table 7); about 0.053 weights documents equally within each domain×origin cell at the Test's origin mass of about 34/34/32; and about 0.062 is the self-weighting estimand of a launch batch, under 20/55/25 as under 40/40/20. Over the eighteen domains of the consumed Test and the testnet stock, a 1,000-case batch holds six to ten human-only documents per domain, minimum to median, so per-domain slices serve as diagnostics.

About 200 human-only documents in a 1,000-case batch measure a human false-positive rate at the frozen 1% operating point only to about ± 1.6 points (95%). The separate pure-human false-positive gate therefore has two tiers. Every entrant must pass a loose threshold on its own cohort's human-only documents. A candidate for promotion to reference must also pass a strict threshold measured on the thousands of human documents in the lookup canary's fresh Common Crawl draw, which is scored for every such candidate anyway (Section 5.4). Testnet has neither Common Crawl stock nor canary anchors, so there the strict tier is measured on the canary's first audit draw: separate human documents drawn from the same pools as the cohort's (Section 5.3), at least as many as its human-only documents, and at that size no more precise than the loose tier. Rates pooled over seven cohorts, about ± 0.6 points at 1,000 cases and ± 0.3 at 4,000, are reported for monitoring. Human spans inside mixed documents cannot stand in for these documents: identical human words score higher inside a mixed document than alone (Section 14.1), so their in-context false-positive rate is a separate span measurement (Section 3.2).

A passage of fresh stock is scored as a pure-human negative at most once. It may also serve as the source of model rewrites, in its own cohort or later ones, at most four times and with uses at least seven cohorts apart, with a fresh generator, seed, voice, and editing operation each time; a use in its own cohort counts toward the four. A rewrite ledger records each use's generator, voice, and operation, and the signed manifest binds the use limit, the gap, which counts cohorts that drew human text, and the set of voices and operations. A mixed document repeats its source's unreplaced words byte for byte, so a passage is no longer unseen once used. In steady state only pure-human negatives consume the human stock, about $0.2n$ per cohort: 73,000 documents a year at 1,000 cases and 292,000 at 4,000; before then, rewrite sources come from fresh stock or from the same cohort's negatives. Each document is scored independently, and qualification requires a document's outputs to be identical whether it is scored alone or among its batch, so an artifact cannot link a passage to its other appearances, in the same batch or earlier ones. The dependence that reuse adds between cohorts is negligible: the within-group correlation is about -0.03.

At 1,000 cases the standard error of a paired score difference between two independently retrained models is about 3.5% of the reference's headroom under 20/55/25; it is 2.5% at 2,000 and 1.7% at 4,000. These figures were measured on self-weighting batches drawn from the consumed Test. Separating two models that differ by 1% of headroom, the measured training-seed noise floor, would need about 126,000 documents under the sealed Test's design, where the same standard error is about 4.0% of headroom at 1,000 cases, for a two-sided 5% test at 80% power. `reward-v2` therefore sets its eligibility margin and band width at about two standard errors before quantization, whose grid is coarser still: at the batch sizes and rosters given here a band is one grid step, so the clusters in a band tie and share its credit equally, while differences below the noise still move clusters across band boundaries, where they can reverse the order of pay (Section 5.9). Container scoring costs about $0.03–$0.34 per 1,000 documents, depending on length and instance size (Section 14).

5.3 Human stock

The mainnet human stock is pre-cutoff Common Crawl [22]: pages from crawls CC-MAIN-2013-20 through CC-MAIN-2022-49 captured before November 30, 2022, the public release of ChatGPT. These pages form a historical-human proxy stratum, not documented human-only work. The mainnet estimand is therefore historical human writing against current model output. Pages pass the date gate, rights exclusions, quality rules, and versioned machine-text screens. No detector filters individual documents: at least 10% of each quarterly pool is scored with R_0 and two earlier detectors, and a domain and capture-era cell is capped or excluded when the upper 95%

bound of the excess of its later crawls' flag rate over its earliest crawls' exceeds two points on two of the three detectors or on the screen counts. Pilot measurements appear in Section 14.1.

A frozen quarterly pool manifest governs each draw. A secret 256-bit key ranks WET files, source groups, and the passage taken from each page; the manifest records the key's SHA-256, and the key is revealed when the pool retires. Random WET files fill every domain, and draws approximate uniform sampling over unique documents by weighting each crawl by its new URLs and down-weighting repeated captures. Versioned domain rules assign each excerpt its cohort domain. A fresh hand-labelled draw under the current rules found 72% [64, 79] of documents usable at the launch shares, with no domain reaching 90%; the rest are mostly human text under the wrong domain or register, and 7 of the 450 documents, 1.6%, were machine-made or machine-processed text, which counts as unusable. Domain precision is therefore a stated limit that sizes the pool, not a launch requirement (Section 14.1). Launch shares of human parents follow per-domain caps derived from each domain's measured frame: essays 51.25%, news 26.5%, forums 7.8%, reviews 4.16%, legal text 3.11%, technical text 2.39%, speech 2.3%, fiction 1.8%, and email 0.69%, almost all press releases. Each capped domain sits under its cap, and essays receive the freed points. A larger share for business prose needs its own measured frame outside guessable press hosts. The caps bound lookup in small domains (Section 5.4).

Recency and formatting are shortcuts. Text from 2020–2022 differs in topic from text from 2013–2016, model-only mirrors are newer still, and generated text carries typography and layout that human excerpts lack. Uniform sampling over unique documents puts about 44% of the mass in 2020–2022, and each model-only mirror is conditioned on its parent's topic and on what existed by the parent's capture date. A versioned cohort generation profile then puts each generated child into its parent's form: a parent-conditioned normaliser applies the parent's quote, dash, range, and ellipsis conventions and paragraph structure, a form screen rejects children whose text fails the human excerpts' prose screen, and one window of 150 to 450 words, the human excerpts' own bound, applies to the final documents of every class. Two gates are measured on a Common Crawl-parented mock batch, each passing on its point estimate with its bootstrap upper bound reported: an era-only adversary and a layout-and-typography adversary must each gain at most 5% of the reference's headroom. An adversary here is a cross-fitted logistic model that reads only its own features, and its gain is its Brier improvement over a constant prediction as a share of the reference's headroom: how much of the task that shortcut alone could do. Two signals are reported as diagnostics, without a gate: an era probe's ability to tell parents from mirrors, and character-class shares, such as case, digits, non-ASCII characters, and emoji. Character use is left out of the layout gate because it is not settled whether it is a formatting shortcut, which the generation profile should remove, or a property of model writing itself, as mean word length is. On the mock batch it gains more than the gates' bar (Section 14.1), a stated limit re-measured with the era-only gate. Both gates pass on the mock batch, but the era-only gate's upper bound reaches 5.04% (Section 14.1). It is accepted on its point estimate for the testnet phase and re-measured before mainnet live rewards on normalised, length-matched positives from the next training round, with a cohort large enough to bring the upper bound below 5%; until then the bound is a stated limit. σ, δ, w (Section 5.9), and the headroom are re-derived on Common Crawl-parented batches. Common Crawl text is evaluated privately and never redistributed. As project policy it may be sent to hosted model APIs to generate mixed rewrites.

Testnet cohorts use a separate public stock: the unassigned remainder of the evaluation-reserved ranges behind the sealed Test, about 240,000 dated public documents ranging from Wikipedia and PLOS articles to forum and blog text, all historical-human proxies (Section 14.1). At 1,000 cases it lasts about three years ignoring domain balance, but seven of its eighteen domains hold fewer than 350 documents, so domains bind first. When a domain cannot fill its share, a later manifest reweights or retires it and records the changed estimand; σ, δ, and w are then re-derived. When a stock cannot fill a batch, or the mainnet floor, no new cohort opens and the last target stays live, paying the same clusters, until new stock arrives. Every text in the public stock can be recognized (Section 5.4); this is an accepted pilot limit of testnet. It reaches mainnet's scoring only through the reference testnet hands on (Section 5.6), and it also reaches the revenue pot's distribution on mainnet, which credits testnet promotions (Section 8.1).

Testnet keeps fresh stock for cohorts with outside entrants. A cohort whose roster has no admitted outside entrant with a working artifact, such as a rehearsal with only operator or synthetic entrants, draws its human documents from already-exposed pools alone: 570 development human documents, 4,593 public documents whose only earlier roles were the consumed Test, held-out, or diagnostic sets, and any document an earlier testnet cohort drew. Fresh stock is drawn only for a cohort with such an entrant. The coordinator enforces the rule: it draws from a plan's fresh pool only when the frozen roster has such an entrant, and otherwise only from pools on its

exposed list. Every testnet cohort so far has drawn exposed documents only, including the first cohorts with an outside entrant, because testnet's standing cohort template names only an exposed pool (Section 14.2).

Exposed documents may be scored again as pure-human negatives without limit, never-drawn documents first and none twice in one cohort. Their uses as rewrite sources need not keep the limits above either. A 1,000-case cohort needs about 660 distinct parents, its 550 mixed documents plus a fifth more as spares, so under a four-use limit with a seven-cohort gap the standing template's draws would stop within seven consecutive cohorts, which need more distinct parents than its 4,593-document pool holds. A versioned testnet-only rule therefore lifts the use limit and the gap for exposed pools: a passage first seen in an earlier cohort may parent again, at most once in a cohort, after the cohort's own negatives and never-parented stock, least recently parented first, and the 570 development documents join as parents only. The gate policy that the manifest binds names the rule, so no manifest binds a limit that its draw lifted, and the coordinator refuses the rule off testnet or beside a fresh pool. As of November 11 the rule is built and awaits installation between two cohorts. A score on reused documents can rest on recognizing the text, so it shows that an artifact runs and is scored, not how it performs on unseen text, and testnet scores are not evidence of mainnet quality.

5.4 Lookup exposure and the canary

The human side of a batch is existing text, and a mixed document keeps the unreplaced parts of its source byte for byte. An artifact that memorized human text could look a passage up and compare it with the query, labelling recognized text human and altered text AI, without detecting AI involvement. Its gain is the covered documents' share of the reference's loss, roughly coverage times headroom, and more where it targets domains the reference handles worst. Admission rejects an explicit text index (Section 3.4); an artifact later found to carry one has invalid scores, and their signed correction must also address any promotion they produced. Recognition stored in the parameters, including through pretraining exposure, cannot be excluded. The protocol reduces lookup; it does not prevent it.

On testnet the exposure is large. The whole public stock fits in a 4.1 MB sentence Bloom filter, far below the learned capacity the parameter bounds allow, and its raw text in 327 MB. Recognizing one domain's documents alone, such as Wikipedia, would gain about 12% of the reference's headroom on a 40/40/20 batch, more than the 1,000-case margin. On mainnet the frame is web-scale, about 64 billion first-seen URLs, while an artifact at the cap holds an estimated 40–72 MB of learned capacity: 160 million parameters at the 2 to 3.6 bits per parameter that language models have been measured to store [23,24]; the span tower class's bound of 173.5 million parameters raises that capacity, and each gain below that the capacity bounds, by about 8%. Against that capacity, estimated gains are at most about 0.09% of headroom under uniform sampling over unique documents, 0.31–0.46% if sampling followed captures, and 2.3–3.4% if every domain had an equal share. The share caps hold the total to about 0.5% against an attacker whose coverage of each stratum stays partial; one who completes a capped stratum can reach about 1.1%. These estimates can understate coverage by up to 1.44 times, and the frame size is known only to within a factor of two. Encoders pretrained on web text may already recognize Common Crawl pages. A first red team found no membership signal in the masked-language-model loss of two small encoders pretrained on data containing Common Crawl, and a linear membership head trained on R_0's encoder recovered 8.5–14.7% of the pages it was trained on at a 0.1% false-positive rate, far below the ideal index that the caps price; the test on ModernBERT-base's own pretrained weights is still owed (Section 14.1).

The defences are the parameter bounds; breadth of sampling; keyed, secret passage choice, which multiplies the keys an attacker must store by about 5.5, the average number of valid passages per page; share caps for small or guessable domains; text first published after a manifest freezes, such as new parliamentary records or court opinions, as an option, which counts as unseen once only; and a lookup canary.

The canary uses about 1,000 freshly written anchor documents a year, produced by paid writers on a keystroke-logged editor, plus about $1,000 a year of US-government court filings bought from PACER that are not already in free public archives. Anchors stay private, form one source group per writer, are reused across audits, and are never scored in a cohort. They cost about $8,900 a year, which keeps the human-data budget under $10,000 a year. Every candidate for promotion to reference (Section 5.6) is audited: its mean paired utility difference against a frozen R_0 on a fresh 20,000-document Common Crawl draw, minus the same difference on the anchors and their rewrites, is flagged when it exceeds an honest-model offset by 3.09 standard errors. A first flag triggers an immediate re-audit in the same cohort on a fresh, independent Common Crawl draw. A cohort's candidates share one draw, so only a re-audit adds a second. At one audit a day, the draws take about 7.3 million fresh documents

a year, 25 times the human negatives of 4,000-case batches and about 1% of the frame's WET text (Section 14.1), and scoring one model on a draw costs about $0.70. A single flag is recorded only in the coordinator's private score package. Two flags block the candidate's promotion, with no automatic reward penalty: the next retained cluster head in rank order that passes the canary and every other promotion condition is promoted instead, and credits are paid as usual. A signed operator review may later clear a blocked candidate; clearing does not change a past promotion. The canary has little power: with 1,000 anchors, a close fine-tune of R_0 needs a lookup gain of about 20–23% of headroom to be flagged with 80% probability, and no achievable lookup by a very different model is flagged. Honest models are falsely flagged in about 0.3–0.6% of audits. Prevention therefore does most of the work.

5.5 AI detection quality

Let $a_i(t)$ be model i's predicted AI-involvement probability for document t, its displayed AI confidence divided by 100 and the sum of its model-only and mixed probabilities. The independent label y_t is one for either model-only or mixed production and zero for the batch's human-only documents as the frozen labeling policy declares them, including the historical-human proxies of Section 5.3, which are not documented human-only work. Binary Brier utility is

$$u_i(t) = 1 - (a_i(t) - y_t)^2. \tag{1}$$

For G source groups, let $\lambda_g \geq 0$ sum to one and let within-group weights $\omega_{t|g} \geq 0$ also sum to one. The self-weighting batch makes these weights equal per document. The cohort score and its improvement over the reference R_k in force are

$$S_i = \sum_{g=1}^{G} \lambda_g \sum_{t \in g} \omega_{t|g} u_i(t), \qquad q_i = \frac{S_i - S_{R_k}}{1 - S_{R_k}}, \quad S_{R_k} < 1, \tag{2}$$

where S_{R_k} is the reference's score on the same batch under the same outcome rules. Improvement is divided by the reference's remaining headroom; negative examples are not individually clipped away. Involvement probabilities enter in parts per billion, so each utility is exact in units of 10^{-18}, and a score is the floor of the batch mean on that grid. These exact scores stay inside the coordinator. Cutoffs, bands, credits, and the row compare the released quantized scores of Section 5.7 as exact rationals instead; only the promotion margin and the public-copy gate's score clause read exact scores, privately.

AI-involvement Brier is the only reward quantity. Share, attribution, span, and the pure-human false-positive gates decide eligibility or promotion and carry no reward weight. Evaluate share against measured fractional labels with source-balanced absolute and squared error, coverage, and slices by rewrite size and procedure. A mixed-history label does not imply 50% AI share, and a fractional share target cannot replace y_t in Equation 1 while preserving its involvement meaning. Span outputs are measured as described in Section 3.2.

Report three-class origin Brier, calibration, sensitivity at fixed human false-positive rates, and population slices separately. Attribution reports include retrieval accuracy, reciprocal rank, open-set rejection, and performance on withheld writers and generators. Rejection means declining named attribution as defined in Section 3.1, including *Other human* and *Other AI*. These comparisons use independent reference and query works. A profile match cannot override the origin output, and mixed work need not have one attributable author. Measure *Other AI* on documented generator roles and *Other human* on documented human roles, including both roles in mixed work. Neither fallback earns named-identity credit. Both outputs are conditional, so a fallback for an absent role is not itself a false attribution; mistaken involvement probabilities are scored through origin measurements. Low predicted role probability never removes a documented contributor from the attribution denominator. Gate thresholds, including both false-positive tiers, are fixed on development evidence before the cohort and bound into its signed manifest; testnet's manifests bind provisional values. After the close, a planned gate feedback record tells a failing entrant the thresholds and the coarse classes of the gates it failed (Section 5.7). Because the gates carry no reward weight, they are floors against unusable outputs rather than improvement targets. That evidence includes R_0's own mixed-document share error of 15.5 points and attribution top-1 accuracy of 0.35 (Section 14.1); the launch thresholds themselves are not yet chosen (Section 15).

5.6 The reference model

The reward baseline is the current reference model R_k. The sequence starts at R_0, the document model of the A baseline release candidate described in Section 14, to be published like every reference once its package has

been re-exported without evaluation text (Section 3). Each cohort re-runs R_k, and its predecessor R_{k-1} when one exists, on that cohort's batch. Only a cohort scored against the margin cutoff (Section 5.9) has promotion candidates: its retained cluster heads, in rank order. Each is eligible, so its released score has reached the margin cutoff, $\hat{S}_i \geq \hat{T}$. The first candidate whose exact score reaches $S_{R_k} + \delta(1 - S_{R_k})$, an improvement of δ times R_k's exact headroom, passes the strict false-positive tier (Section 5.2), and is not blocked by the lookup canary (Section 5.4) becomes R_{k+1} when the cohort closes: the baseline for the next cohort and the new reference, whose artifact was already published when its cohort's scoring started. Credits are paid as usual. In a cohort scored against the fallback cutoff or producing no new target, and when no candidate passes, the reference does not change. Promotion therefore has two bars. The first is eligibility, on released scores: at testnet's full roster $\hat{T}$ is one 0.03 grid step above $\hat{S}_{R_k}$, several times the unrounded margin (about 0.0044 at 0.062 headroom), and each released score carries release noise with a standard deviation of about 0.030. Release noise can therefore keep an entrant that is truly better by the margin from becoming a candidate, and it can also make an entrant no better than R_k eligible: by a simulation of `quant-v1`'s noise, offsets, grid, and rounded cutoff, an entrant equal to R_k reaches $\hat{T}$ about 35% of the time at testnet's full roster. Such an entrant, including an undetected perturbed copy of R_k, which takes its own noise draw, then earns credit in a margin cohort. Release noise also sets the order in which candidates are tested. The second bar is the margin itself, which promotion tests on exact scores, privately, releasing only its outcome. Testing exact rather than quantized scores keeps the release noise out of the margin test, so an entrant that noise makes eligible can earn but is not promoted by that noise. The rates in Table 2 are for the margin test alone; eligibility can only lower them. Because the margin uses the pooled standard error of seed retrains, how often it promotes an equally good model depends on how entrants differ from R_k (Table 2). Entrants that differ from it as seed retrains do share R_k's scoring noise, which lowers the chance that one of several clears the margin. A structurally different model has a much larger paired standard error against R_k: those of the other two measured recipes and of V6 against R_0 are about 13–19% of headroom at 1,000 cases, 3.7 to 5.4 times σ (Section 14.1). Those models are much weaker than R_0, so they supply only this scale, and the table's figures for structurally different entrants, taken at its midpoint of about 4.6σ under a normal approximation, are illustrative. Which kind of roster the subnet will see is not yet measured. Once structurally different models of equal quality compete, rather than seed retrains of R_k, promotion is the expected case, and the reference changes hands among equally good models (Section 5.9). The same noise can promote a worse model; the strict false-positive tier and the canary bound human false positives and lookup, not quality.

Entrants	Quality against R_k	Chance that one clears the margin
Ten, differing as seed retrains	equal	about 14%; 21% if their differences were independent
One, structurally different	equal	about a third (0.29–0.36 across the measured range)
Ten, structurally different	equal	about 80% if they share R_k's noise as seed retrains do; 98% if they share none
One, structurally different	half a margin below	about a quarter
One, structurally different	one margin below	about a fifth
One, structurally different	two margins below	about a tenth

Table 2: Chance per cohort that an entrant no better than R_k clears the shared margin test.

No rule bounds the resulting drift in the reference's true quality, but four properties limit it. The promoted model is the top-ranked entrant that passes every promotion condition, so a worse entrant is promoted only when it precedes every better qualifying entrant in rank order on that batch, through batch noise, release noise, or a tie on the grid broken by admission order. Each cohort re-runs the reference on a fresh batch, so a winning draw does not carry forward. A worse reference is easier to pass: against one a full margin below the model it displaced, a structurally different entrant as good as that model clears the margin about half the time rather than a third. And the displaced model becomes R_{k-1}, so a fallback cohort pays only entrants that outscore it. False promotion and the drift it allows are stated limits of the shared margin; their size on simulated rosters is measured before launch (Section 15.2).

Testnet opens a fresh record of cohorts and references, with R_0 as its first reference and no predecessor. Outside the promotion rule, the operator may also set the current and previous references by a signed reference decision

that records its reason (Section 6.6). The decision appears in the next signed manifest, so the sequence of signed manifests is its record, and it never changes a sealed cohort. Credit is bound to its own cohort's package and target (Section 6.1), so a reference decision moves no earned reward; it sets the margin for later cohorts and, while the launch burn lasts, the bar that ends it. Before the first cohort that enforces the span gate (Section 3.2), SlopNinja makes one span-equivalent decision on testnet: R_0's position passes to its first span package, whose document outputs are R_0's and whose probe predictions match R_0's within the measured difference between scorer hosts, so no reward input changes. SlopNinja also plans one replacement, announced in advance on the public status page: replacing R_0 by the next reference it trains, retrained and recalibrated on normalised, length-matched positives (Section 5.3), only if that model passes every required gate, the span gate included; the replacement takes R_0's position, with no predecessor. Mainnet opens its own fresh record, whose first reference is the one testnet holds at that time, with no predecessor; reference decisions remain available there under the same signed public records. A testnet promotion can rest partly on lookup, since testnet's stock is recognizable in full, its strict tier is drawn from the same pools, and its canary has no anchors. Mainnet scores that reference only on its own batches, where recognizing testnet text gains nothing, so mainnet's first bar is the reference's measured mainnet quality, not its testnet record. An inflated record therefore leaves that bar lower than the record suggests: mainnet's launch burn can end, and its first promotion occur, against a weaker model, which once displaced anchors the fallback cutoffs until the next promotion. Before mainnet, that reference is measured on Common Crawl and the anchors, and the results are reported (Section 15).

Every admitted entrant artifact is published when its cohort's scoring starts, and a detected copy of an earlier artifact, a reference or an entrant of an earlier closed cohort, earns nothing; entrants of one cohort meet each other only in its clusters (Section 5.9). An entrant whose inference content equals that of any such artifact is not executed. An executed entrant within the near-copy distance of an earlier artifact (Section 5.9) fails the public-copy gate unless its exact score is more than $w_S/2$ above that artifact's, where w_S is the unrounded band width computed from R_k's exact score. This applies to R_k and R_{k-1}, including R_0 before its package is published, and to every earlier entrant's artifact, the author's own included. A miner therefore earns again only with a model that differs measurably from everything already published, such as an independent retrain or a weight soup of published models, or that improves on its source by half the promotion margin. The gate removes exact and recalibrated copies; it sets a minimum price on a copy perturbed with deliberate noise, but cannot prevent one or bound what it earns (Section 5.9).

5.7 Quantized scores

Exact scores S_i never leave the coordinator. Every released score, for each executed entrant and for both references, is a quantized score

$$\hat{S}_i = g \, \min\Big(\lfloor 1/g \rfloor, \; \max\big(0, \; \lfloor (S_i + \eta_i + o_i)/g \rfloor\big)\Big)$$

under the versioned rule `quant-v1`, so every released score is a grid value; the readings of Section 5.8 follow their own rule. The largest is $g\lfloor 1/g \rfloor$, which is 1 only when g divides 1: on a 0.03 grid it is 0.99. The grid width g is the larger of twice R_k's sampling standard error, measured on the cohort's own batch, and the noise scale below, rounded up on a versioned ladder. The offset o_i is secret and uniform in $[0, g)$, and η_i is exact discrete-Laplace noise of scale $1/(n\varepsilon_v)$. Both are drawn from the cohort's private seed, its manifest digest, and the scored model's inference content, so identical copies under several hotkeys share one draw and no two cohorts share one. The privacy budget is $\varepsilon = 2$ per cohort, versioned, and split equally over every executed entrant's quantized score and the references': with m charged scores, each receives $\varepsilon_v = \varepsilon/m$. Replacing one batch document moves an exact score by at most $1/n$, so, given the grid width and which entrants are withheld, it changes the joint distribution of the released scores by at most a factor e^2. The budget adds across cohorts and across documents that reuse one passage. For fresh stock the reuse limits of Section 5.2 bound a passage's total; testnet's exposed documents, which may be drawn again without limit (Section 5.3), have no such bound, an accepted limit for text that is already public or consumed. Released scores are therefore coarse and noisy: they bound what a score reveals about any one document, and they hide small differences between entrants.

The rank order, cutoffs, bands, credit, and the row compare quantized scores, never exact ones. Gates and cluster membership use private statistics instead: each gate releases one bit, and clusters follow the private near-copy distance, which reads no score. Two decisions read exact scores privately and release only their outcomes: promotion's margin test (Section 5.6) and the public-copy gate's score clause (Section 5.10). A minimum-success gate withholds an entrant whose attributable failures exceed a measured, versioned limit: its quantized score

is drawn and charged but never released, so the split and the noise scale do not depend on which entrants are withheld. The gate, public-copy, withheld, cluster, and promotion outcomes and the grid width are deterministic functions of exact scores and hidden data, released without noise and outside the ε accounting. A versioned per-cohort bit budget counts each of them at full face value, the base-2 logarithm of the number of values it can take, with every released score, every reading of Section 5.8, the span-gate record of Section 5.11, and the gate feedback record below, and is checked when the plan is validated, at the seal, and at publication; a cohort over budget is held, never published or skipped. The total may not exceed nh/F bits, where h approximates the bits needed to single out one human document within its domain's pool and F is a safety factor, so the released values stay F times short of naming every batch document. Testnet sets $h = 8$, below the median of 8.6 bits that its public stock's documents take, and $F = 10$: at 1,000 cases, at most 800 bits, of which a full roster of 40 executed entrants takes at most 746 before readings and records. The bit budget caps what is released; it is not a differential-privacy guarantee.

So that a failing entrant can see what to fix, a planned gate feedback record, signed and published after the close beside the result, gives the gate thresholds that the cohort's manifest binds and, for each scored entrant, the set of gate classes it failed among share, attribution, and the loose false-positive tier. A withheld entrant gets none, and no statistic, margin, or per-document value is published. The result's gate bit follows from that set, so the record adds $\max(\gamma - 1, 0)$ bits per executed entrant, where γ is the number of classes the thresholds can fail: one bit at testnet's current thresholds. The span-gate record adds one bit per executed entrant and two per cohort, so a testnet cohort that publishes both records stays within 800 bits only up to 38 executed entrants; a template naming both needs that smaller roster or a recorded change of h or F.

One channel is not counted. The public challenge counts show each model's progress while it is scored, so an artifact that made its own arithmetic slow on documents it selects, through subnormal intermediate values, could signal a bit or two about each chunk of documents it processes through its timing. Admission refuses subnormal weights, which removes only the simplest lever, and the channel is closed before mainnet (Section 15.2).

The noise has a cost. Its standard deviation is $\sqrt{2}/(n\varepsilon_v)$. On testnet, at 1,000 cases and the full roster of 40 entrants, it is about 0.030 (a noise scale of about 0.021), against R_k's sampling standard error of at most about 0.011, so the release noise dominates the score differences between close entrants, and the grid is 0.03. With five entrants the noise is about 0.005, and the grid then follows R_k's measured standard error: it was 0.02 in testnet's first eight-entrant cohorts. At mainnet's planned 4,000 cases and 256 entrants it would be about 0.046 (a noise scale of about 0.032), rewards would be mostly noise, and the plan, which could release up to 4,917 bits where 3,200 are allowed, would be refused by the current bit budget, so that budget, n, or the split must be revisited before mainnet. These are stated limits.

5.8 Pre-finalized scores

While a cohort is scored, after its roster is frozen and its batch sealed, the status page shows how each executed entrant is doing. Nothing shown here enters a cutoff, band, credit, gate, cluster, promotion, or row, and the released scores of Section 5.7 are computed exactly as they would be without it.

An executed entrant's quantized score, or its withheld outcome, depends only on its own predictions on the batch, R_k's predictions on the batch, the cohort's private seed, and counts fixed at the seal. Once the entrant has finished every assigned document and R_k has finished the batch without a failure, the coordinator computes that value with the same code and draw that the result uses and shows it, marked pre-finalized. The references' quantized scores, and the margin cutoff that follows from R_k's, appear the same way, each once the references it depends on have finished without a failure. Showing a value earlier changes neither the value nor any term of the privacy or bit budgets. A signed correction can still replace it, and the correction keeps the original record.

Before that, a cohort can release segment readings under the versioned rule `live-v1`. The seal splits the batch, never the probe or audit documents, into L segments of $\lfloor n/L \rfloor$ or $\lceil n/L \rceil$ documents by the private task order, so the split does not depend on labels, outcomes, or timing. When a model has resolved every document of segments $1, \ldots, s$ with $s < L$, the coordinator releases its reading for segment s once: the floor of its mean utility on that segment plus exact discrete-Laplace noise and a secret offset, drawn from their own seeded streams, on a grid whose width is the noise scale rounded up on the ladder. The last segment is never released alone. Readings are released for each executed entrant and for R_k, and the status page shows the running mean of an entrant's

readings beside R_k's on the same documents, with the noise's standard deviation. An entrant whose attributable failures in the released segments already exceed the minimum-success limit receives no further reading.

Readings are charged to their own versioned privacy parameter ε_ℓ, set to 1 per cohort and split equally over the executed entrants and R_k. The task order comes from a secret seeded rank, which acts as a uniformly random permutation of the sealed documents, so replacing one batch document changes the readings of one segment per model, each by at most $1/\lfloor n/L \rfloor$ before noise. Given the grid widths and the withheld outcomes, the released scores and readings together therefore change by at most a factor $e^{\varepsilon+\varepsilon_\ell}$ per cohort, one power of e more than the released scores alone, and the budget adds across cohorts and across documents that reuse one passage. Every reading counts in the bit budget at full face value. Once the batch is sealed, L is set to the largest value, up to the plan's maximum, for which the cohort's worst-case bit budget, counting every value any rule releases, still holds at its executed roster, so a larger roster receives fewer readings and possibly none. At 1,000 cases and eight executed entrants, $L = 4$: readings at a quarter, a half, and three quarters of the batch, whose running means have noise standard deviations of about 0.053, 0.038, and 0.031, against about 0.006 for the released score. Above about 33 to 35 executed entrants, depending on the other values the cohort releases, there are none. The operator can stop and resume readings, for one cohort or all, without a deployment.

No rank, band, cutoff mode, gate, public-copy, cluster, or unit value is released before the close. Pre-finalized scores and readings give everyone an earlier view of which published models are doing well: on testnet's calendar a few tempos before the close, and on the mainnet calendar of Table 4 up to about eleven tempos, over half a day, before it, so a competitor can build on a strong published model up to one cohort sooner. A reading is released when its segment is resolved, so its timing shows roughly where the segment ends in the task order, as the challenge counts already show each model's progress. On testnet, where exposed documents are drawn again without limit, the added budget makes a passage's total grow faster. These are stated limits.

5.9 From scores to credits

An entrant is *gated* when its artifact qualification, its public-copy gate, its share and attribution gates, its minimum-success gate, and the per-entrant tier of its pure-human false-positive gate pass, together with the span gate from the first manifest that enforces it (Section 3.2). Gate failures earn no credit. The margin is

$$\delta = \max\bigl(0.01,\ 2\sigma\bigr),$$

where σ is the pooled paired standard error of q at the manifest's batch size, frozen in the manifest. Under 20/55/25, δ is about 0.070 at 1,000 cases and 0.034 at 4,000. Each manifest re-derives σ from current measurements. One frozen σ is used rather than each entrant's own paired standard error because it also sets the cohort's single band width, and because it fixes every entrant's bar before any score is seen instead of making the bar depend on how closely an entrant tracks R_k's predictions.

Table 3 lists the remaining parameters and their translation from subnet 78. Subnet 78 states its band width and duplicate tolerance in points of a normalized score. A literal 2.5-point band would span about 40–48% of the reference's headroom here, depending on the estimand, and put almost every entrant in one band. `reward-v2` therefore keeps the mechanism and the ratios between its parameters, sets the band width from the measured standard error, and rounds it up to whole grid steps, which can only widen it. The grid is at least twice R_k's own standard error, which exceeds the paired standard error between entrants, so at every batch size and roster given here the grid, not the standard error, sets the band: one step, 0.03 at testnet's full roster and 0.02 with eight entrants.

Parameter	Subnet 78	`reward-v2`
Band width w	2.5 points of score	$2\sigma(1-\hat{S}_{R_k})$ in score units, rounded up to whole grid steps; before rounding, 7% of headroom at 1,000 cases (about 0.0044 at 0.062 headroom), 3.4% at 4,000
Score grid g	—	the larger of twice R_k's standard error and the noise scale, rounded up on a versioned ladder; 0.03 at testnet's full roster
Privacy budget ε	—	2 per cohort, split equally over the quantized scores
Near-equal tolerance	1 basis point, $w/250$	none: distinct quantized scores differ by at least one grid step, and equal ones share credit
Decay r, across and within bands	0.85	0.85
Eligibility	$S \geq 0.99\, S_{\text{inc}}$, the incumbent's score	gated and $\hat{S}_i \geq \hat{T}$, the margin cutoff on the grid
Duplicates	identical content credited to the first acceptance	detected copies of published artifacts (identical content, or within d without the $w_S/2$ improvement) earn nothing; near-duplicate entrants share one credit equally
Near-copy distance d	—	0.025 after isotonic recalibration, lowerable by a later manifest; the public-copy gate adds a $w_S/2$ improvement exception on exact scores, with $w_S = 2\sigma(1-S_{R_k})$
Native row	largest-remainder split of 65,535	the same, with equal remainders in roster order

Table 3: Frozen `reward-v2` and `quant-v1` parameters, signed in each cohort manifest.

Two models are within the *near-copy distance* when, after the best rank-preserving recalibration, their involvement probabilities still differ by a root-mean-square of at most d. The recalibration is fitted both ways by isotonic regression over the documents on which both models returned a prediction, and the smaller residual counts; two models with no common prediction are not near. A temperature, bias, or any other change that preserves a model's ranking of documents is therefore at distance zero, however far it moves the probabilities. The manifest freezes $d = 0.025$. On 2,000 random 1,024-document 40/40/20 batches drawn from the consumed Test, two training seeds of one recipe never came closer than 0.034, and seeds of the launch recipe never closer than 0.059. That separation has not been measured on 1,000-case batches in the 20/55/25 launch mix, a stated limit: if seeds come closer there, independent retrains would share clusters or fail the public-copy gate. A later manifest may lower d, never raise it, and each package records the smallest distances it observed (Section 5.10), so d can be lowered before independent models approach it. Clustering uses this distance alone. The public-copy gate adds one score clause, read privately: an entrant within the near-copy distance of an earlier artifact still passes when its exact score is more than $w_S/2$ above that artifact's, where $w_S = 2\sigma(1-S_{R_k})$ is the band width computed from the exact reference score, before rounding; the band width w of the table and of the calculation below uses $\hat{S}_{R_k}$ instead. A model built on a published one therefore still passes the public-copy gate when it beats it by more than $w_S/2$, which is half the promotion margin $\delta(1-S_{R_k})$ while $2\sigma \geq 0.01$; it earns only if its released score also reaches the cohort's cutoff.

The calculation proceeds in seven steps over the gated entrants. Every score it compares is a quantized score; cluster membership uses the private near-copy distance. The *roster order* is admission order: finalized commitment block, then extrinsic index. Step 1 gives the cluster placement of the successor of `reward-v2`, a separate reward rule with its own identifier in every manifest and row record. `reward-v2` itself, which testnet's current record runs, applies the cutoff first, then walks only the gated entrants at or above it, ordered by quantized score and then roster order, and places each cluster by that walk's first member, its highest-ranked one; the other steps are the same.

1. *Clusters.* Walk every gated entrant in roster order. The first entrant not yet assigned heads a cluster that also contains every later unassigned entrant with identical inference content or within the near-copy distance of the head on the batch, judged by distance alone, with no score clause; there is no chaining through other members. A cluster takes its head's quantized score, so the noise draw of its earliest-admitted member places it, and no other member's score moves it. Clusters are ordered by that score, then by their heads' roster order. This *rank order* breaks every tie below; no exact score enters it.
2. *Cutoff.* When $\hat{S}_{R_k} = 1$, which needs a grid width that divides 1, no cutoff applies, neither margin nor fallback, and the cohort produces no new target (end of this step). Otherwise thresholds are rounded up onto the grid. Let $T = \hat{S}_{R_k} + \delta(1 - \hat{S}_{R_k})$ and $\hat{T} = g\lceil T/g \rceil$; when $\hat{S}_{R_k}$ is the largest grid value and below 1, $\hat{T}$ lies above every releasable score, so no entrant reaches it. If any cluster's score $\hat{S}$ reaches $\hat{T}$, the cutoff is $\hat{T}$. Otherwise, when R_{k-1} exists and some cluster scores strictly above $\hat{S}_{R_{k-1}}$, the cohort is scored against the fallback cutoff
$$C_N = \hat{S}_{R_{k-1}} + 0.15\,(\max_N - \hat{S}_{R_{k-1}}), \qquad \hat{C}_N = g\lceil C_N/g \rceil,$$
where $\max_N$ is the top cluster score in cohort N, so $\hat{C}_N \le \max_N$. Clusters whose score reaches the cutoff are *retained*; the others earn nothing. The fallback removes the bottom 15% of the score range that would earn against the previous reference, so that rewards can flow to the current cohort; the reference does not advance. The anchor is R_{k-1} rather than R_k because no entrant's released score has reached the margin cutoff, so none is publicly shown to be better than R_k, and a cutoff at $\hat{S}_{R_k}$ would, like the margin cutoff, leave it to batch and release noise whether entrants equal to it earn. Anchoring on the last reference the subnet passed by the margin lets independent models as good as R_k, including new models from R_k's author, earn in a fallback cohort, while detected copies of R_k fail the public-copy gate (Section 5.6). On the released grid this works less well than the exact scores suggest. When R_k leads R_{k-1} by about one unrounded margin, as it does after a close promotion, the gap is a small fraction of a grid step, so $\hat{C}_N$ is almost always one step above $\hat{S}_{R_{k-1}}$ and the 15% term rarely moves it. The fallback is then a second draw against a bar about as high as the margin cutoff rather than a lower bar: it separates entrants as good as R_k little from entrants only as good as R_{k-1}, and when nobody reaches it, the previous target stays live and pays the previous cohort's winners. For example, on a 0.01 grid, if R_{k-1} scores 0.94 on the batch and the top entrant 0.96, C_N is 0.943 and the cutoff 0.95. An entrant whose released score is just short of the margin cutoff therefore earns in a fallback cohort but not when another entrant reaches the cutoff. At every roster given here the grid is wider than the unrounded margin, so the margin cutoff is one grid step above $\hat{S}_{R_k}$. When $\hat{S}_{R_k} = 1$, or when neither the margin nor the fallback cutoff applies, the cohort produces no new target: the previous target stays live, and if there has never been one the row sends the whole pool to the owner hotkey's UID, which burns it.
3. *Bands.* Round the band width up to whole grid steps, $W = g\max(1, \lceil w/g \rceil)$ with $w = 2\sigma(1 - \hat{S}_{R_k})$, and place each retained cluster in band $b = \min(\lfloor \hat{S}/W \rfloor, \lceil 1/W \rceil - 1)$ on the absolute score grid, so band edges lie on the grid. Rank occupied bands $\kappa = 0, \ldots, B-1$ from the top, and rank the clusters within band κ as $\mu = 0, \ldots, n_\kappa - 1$ in rank order.
4. *Credit.* The cluster at position μ of band κ has base credit
$$c(\kappa, \mu) = \frac{r^\kappa}{\sum_{j<B} r^j} \cdot \frac{r^\mu}{\sum_{j<n_\kappa} r^j},$$
so the credits sum to one.
5. *Ties.* Clusters whose heads have equal quantized scores form a tie group within one band. If the group occupies positions μ_s to μ_e, each of its clusters receives the mean of $c(\kappa, \mu)$ over those positions, so the roster order that lays out positions within a tie changes no tied cluster's credit.
6. *Equal split.* Each member of a cluster receives an equal share of its cluster's credit.

7. *Row.* Apportion 65,535 units across members by largest remainder: each takes the integer part of its share, and leftover units go to the largest fractional remainders, with equal remainders in roster order. At each tempo the coordinator maps each member's hotkey and registration generation to its current UID in its finalized registration snapshot and signs the resulting row. Units of a hotkey that is no longer registered under that generation go to the UID of the subnet owner hotkey in the same snapshot, which burns them (Section 5.12). The chain stores a weight row scaled so that its largest entry is 65,535, so validators commit, and compare each reveal with, the row in that form, computed exactly with ties rounded half to even; the canonical row and its digest remain the audited record.

An entrant earns in a margin cohort when its released score reaches the margin cutoff, and in a fallback cohort when it reaches $\hat{C}_N$. Because the cutoff is one grid step above $\hat{S}_{R_k}$ and every released score carries its own secret offset and noise, an entrant equal to R_k reaches it by the draws alone: about a sixth of the time when it enters alone on a 0.02 grid, which while the launch burn lasts is enough to end it (Section 6.1), about a quarter with ten entrants on that grid, and about 35% at testnet's full roster. A simulation of `quant-v1` as stated in Section 5.7, with R_k's exact score at a uniform position on the grid and 0.062 headroom, gives the cohort outcomes for rosters of entrants equal to R_k. With ten that differ from it as its seed retrains do, about a third of cohorts have no entrant at the margin cutoff and fall back where a predecessor allows, about 61% are margin cohorts without a promotion, and about 6% promote; with forty, these are about 7%, 78%, and 15%. With ten to forty structurally different equal entrants, three quarters to nine tenths of cohorts promote. The usual outcome for a roster of seed retrains is therefore a margin cohort that pays the entrants whose draws lifted them to the cutoff and keeps R_k, and an equally good entrant earns in a quarter to a third of cohorts, chosen by its draw. A detected copy of an earlier artifact earns nothing, so each artifact takes that draw once, apart from a perturbed copy that escapes detection (described below), and equally good entrants earn equally only in expectation over many artifacts. These rates leave out the strict false-positive tier and the canary, which can only lower promotion, and they are measured again under `reward-v2` on simulated rosters before launch, together with how often a fallback cohort pays an entrant equal to R_k or only as good as R_{k-1} (Section 15.2).

With $r = 0.85$, a lone cluster receives the whole pool, and two clusters with different quantized scores, which with one-step bands lie in different bands, receive 54.1% and 45.9%, which the row encodes as 35,424 and 30,111 of its 65,535 units; two with equal quantized scores receive 50% each. A crowded band divides its share equally among its tied clusters: with two clusters in the top band and one below, they receive 27.0%, 27.0%, and 45.9%. The rank split within a band applies only to a band wider than one grid step, which no parameter set given here produces. With $r = 17/20$, each credit is already a fraction in lowest terms; the private score package records every cluster's exact credit, and the public result releases band ranks and units. All arithmetic uses exact integers and rationals over the fixed-point quantized scores, so anyone can recompute the coordinator's row from the public result byte for byte. A roster holds at most 256 entrants, the subnet's UID limit.

A band's share depends on its rank among occupied bands, not on how many clusters it holds, so credit is not monotone in score across a band boundary. A cluster alone in the lower of two occupied bands receives 45.9% however many clusters share the band above, 2.55 times each of three there and 4.25 times each of five; alone in the third of three bands, it receives 28.1%. Subnet 78's 2.5-point bands seldom divide a roster, so its experience does not cover this case. Bands here often do: a band is one grid step, each released score carries its own secret offset, uniform over a grid step, and its own noise, and the band grid is fixed in absolute score, so where a roster divides depends on the batch and the draws. A score difference below the noise can therefore reverse the order of pay. The split is kept from the ported mechanism, and what it buys is the same property: a band's share does not shrink as other bands fill, so a crowd of entries in one band, such as variants just beyond the near-copy distance, cannot dilute or push down the pay of entrants in other bands, as it would under a geometric split by rank alone. Its non-monotonicity is a stated limit; the strategy it opens is described below.

Subnet 78's near-equal drop is not used. Distinct quantized scores differ by at least one grid step, far more than subnet 78's $w/250$, so a drop could act only on exact ties, which share credit instead; the content check, the public-copy gate, and clustering remove copies first. Roster order decides which entrant heads a cluster, and so which later entrants join it, since membership is measured against the head, and which quantized score places the cluster. Among equal quantized scores it also sets the order in which promotion tests tied heads, so the earliest admitted head that passes is promoted even when a later tied head has a higher exact score.

Placing a cluster by its earliest-admitted member removes a lottery. Identical copies already share one noise draw, but near copies with different content draw independently, so a cluster placed by its best member would let a

party that enters several near copies of one model take the best of their draws toward the cutoff, which an entrant equal to R_k reaches alone about 35% of the time at testnet's full roster. Under placement by the earliest-admitted member, extra near copies add no draw, and an honest entrant that shares no cluster is unaffected. Copies farther apart than the near-copy distance still draw independently, and registration cost and UID slots are the only brakes on them. As a reward-rule change, placement by the earliest-admitted member takes effect only through a fresh record under the successor rule (Section 6.6), and mainnet's first record opens under it; testnet's current record keeps `reward-v2`'s placement.

The near-copy distance removes exact and recalibrated copies and sets a minimum price on a perturbed copy; it neither prevents one nor bounds what one earns. To clear distance d, a copy must add variation that no rank-preserving recalibration explains. Variation of that size that carries no label information costs about $d^2 \approx 6 \times 10^{-4}$ of expected score, less any gain also available by recalibrating the reference, about 3.7×10^{-4} for R_0, so the guaranteed floor is about 2.6×10^{-4}. The cheapest perturbed copy of R_0, standing in for any published reference, found on the consumed Test adds noise to R_0's ranking of documents after its best recalibration. Pooling two independent measurements of 128 noise realizations each, on 40/40/20 batches, it cleared the gate on the batch and against R_0's probe fingerprint together in about 89% to 99% of 1,000-case cohorts, for an average of about 3.5×10^{-4} to 4.8×10^{-4} of expected score: 0.6% to 0.8% of the reference's headroom, or 0.07 to 0.10 of an unrounded band at that measurement's $\sigma = 0.040$, about a tenth of a band. At 4,000 cases the batch comparison alone is cleared nearer the floor, but the 1,024-document probe comparison with R_k's fingerprint (Section 5.10) then binds: clearing both in about 99% of cohorts costs about 4.7×10^{-4}, about a fifth of a band. That cheapest copy earns like an entrant equal to the reference: in a margin cohort when its draws lift it to the cutoff, about a third of the time at testnet's full roster, or in a fallback cohort when it reaches $\hat{C}_N$; and it must be rebuilt each cohort because its own artifact is published. A tolerance large enough to price an escape at a full band would be about 0.07, near the distance between independent retrains of the launch recipe, so no prediction tolerance can price it. An honest cluster that shares a band with one such copy has the same quantized score and receives half of the band's share. Clustering uses the same distance, so a sybil variant of an entrant costs the same to separate from its cluster.

Adding more noise escapes just as surely and lowers a copy's score by any chosen amount, and because pay is not monotone across bands, a lower score can pay more. A fallback cohort admits scores down to C_N, 85% of the way from the top score to $S_{R_{k-1}}$. When R_k genuinely improved on R_{k-1} by the margin, about an unrounded band in score units, that range reaches most of an unrounded band below the entrants near S_{R_k}. A copy of R_k degraded into it, with noise well beyond d, can land alone in a band below theirs and take 45.9% of the pool if they occupy one band, or 28.1% if they occupy two. In a promotion cohort, an entrant whose model clears the margin by more than a band can do the same with a weaker variant under a second hotkey, and any entrant can submit less than its best model. Such an entry aims blind: the batch, the grid's position, the secret offset, the noise, and the cutoff are unknown at commitment, and a miss earns nothing below the cutoff or an equal part of a crowded band's share. Its expected credit against realistic rosters has not been measured, and that measurement precedes launch (Section 15.2). Where it lands alone, it can earn more than each honest cluster above it; this is a stated limit of the ported split.

Weight soups of published artifacts are new models. A soup that is not within the near-copy distance of a published artifact earns normally, and the protocol does not look for one: averaging the weights of fine-tunes of one base, like further training of a published model, is legitimate work. A submission must still be one allow-listed graph, so an average of two separate encoders' outputs cannot be submitted. Such an output average is the only blend measured: on the consumed Test, a 50/50 average of two published seeds' outputs scored about 0.2 to 0.3 of a band above R_0 at 1,000 cases and cleared the gate against both sources, on the batch and on the probe, in every batch measured. A soup earns in a fallback cohort like any model at its score, and one that beats R_k by the margin can be promoted.

5.10 Published artifacts and the fingerprint probe

Exact copies are found by inference-content digest against the published-artifact index, which holds every entrant artifact of every closed cohort and every record's first reference: an entrant fails the content check exactly when its digest is in that index, and such an entrant is never executed. A cohort's entrants join the index when the cohort closes, so the entrants of a cohort abandoned after its scoring started, which never closes, stay outside it (Section 6.6). The digests are public, so anyone can repeat this check. Re-running every earlier artifact on

each batch would grow without bound, so near copies are also measured on a fixed private probe of 1,024 documents, drawn and generated like a batch. Near copies of R_k and R_{k-1} are measured on the cohort batch, where both references already run, and on the probe against their fingerprints. Near copies of every earlier artifact are measured on the probe. A near copy of R_k is recorded before one of R_{k-1}, and either before a probe match.

Every executed entrant also runs on the probe once, whatever its score and other gates. No score in an entrant's own cohort bounds what a copy of it can earn: R_{k-1} can outscore R_k on a batch, and a copy meets new documents in a later cohort. The entrant's probe predictions are compared with the recorded probe predictions of every earlier published artifact, R_k and R_{k-1} included, and become its own fingerprint when its cohort closes. The private score package records every entrant's exact comparisons with both references' fingerprints, and the coordinator checks that a near one fails the gate before it signs, so a copy of a reference must escape both the batch and the probe; only the gate's bit is released. When entrants with identical inference content return different probe predictions, the earliest-committed entrant's become the fingerprint. Each artifact therefore runs on the probe only in its own cohort. An artifact that was never executed carries no fingerprint and is protected by its content digest alone.

The probe carries no reward weight. Its labels serve only the $w_S/2$ exception against fingerprinted artifacts. Its human texts are withdrawn from the hidden stock and never scored. The probe and the fingerprints stay private, and each manifest binds at least one probe version, of at most 65,536 documents, by version and digest. A model that could recognize probe documents could look different there alone, so the probe follows the batch design. A new version is drawn when that design changes materially or the probe may have been disclosed. A replaced version stays in use, and entrants run on it too, while it holds the fingerprint of any artifact not fingerprinted again on a later version. Both references are fingerprinted on every version in use. A disclosed version leaves use at once, and earlier artifacts recorded only on it keep exact-digest protection only.

The probe adds 1,024 inferences per executed entrant and probe version in use. That roughly doubles an entrant's scoring at 1,000 cases and adds a quarter at 4,000. The private package also records, as exact integers, the smallest distance between two cluster heads, between R_k and any gated entrant on the batch, and between any gated entrant and any fingerprinted artifact on the probe, so a later manifest can lower d before independent models approach it.

5.11 Failures and the result package

An attributable failed or missing model result has zero utility and retains its assignment weight. A Container crash or out-of-memory failure is an infrastructure error until it reproduces on a fresh instance, in single-document isolation, under the frozen resource limits; a crash that reproduces counts against the artifact, and those documents score zero utility for that entrant only. A task whose every attempt ended in an infrastructure error blocks the package: the cohort pauses for reconciliation under the same manifest, and the previous target stays live. Operator downtime never becomes a miner failure. If either reference model fails to return a prediction, the package is held for a signed correction rather than scored against a damaged baseline. Documents outside the input profile never enter the batch, so no entrant can gain from a smaller successful subset. Later *B* training-service obligations are absent from an *A*-only launch; no fictitious requests or *B* availability score is needed to make an *A* entrant eligible.

The coordinator signs two records for each cohort. The private score package binds the cohort and revision, the commitments with their finalized block and extrinsic index, the artifact digests, credited hotkeys and registration generations, coverage and outcome accounting, each failed gate with its reason code, exact per-miner scores, the reference scores S_{R_k} and $S_{R_{k-1}}$, the cluster assignments, each scored entrant's public-copy result, with its exact near-copy statistics against R_k and R_{k-1} on the batch and against their fingerprints on the probe, and the matched published artifact of any failure, the statistics behind each distance-based cluster membership, the published-artifact index root and count, the distance monitor, the standard-error sums and quantization draws, every `reward-v2` and `quant-v1` parameter, the cutoff mode, each retained cluster's exact credit, every member's integer units, the R2 receipt index's root and count, and the digest of the public result. It never leaves the coordinator's storage. The public quantized result names the chain, cohort, and manifest, the rules and their public parameters, the batch size, the grid and band widths, the cutoff mode and cutoff, each reference's identity and quantized score, the promoted model if any, the occupied bands, and, per roster entry in roster order, its identity and outcome: not executed (as a copy of a published artifact or for failing qualification), withheld, or its quantized score, gate and public-copy bits, and cluster, with its band rank and units. It also records each term

of the cohort's bit budget, and it carries no exact score, standard error, offset, noise draw, statistic, probe value, attempt count, or receipt value. Each quantized score and withheld outcome in it is shown on the status page, marked pre-finalized, as soon as it is fixed (Section 5.8); its remaining allow-listed values are published when the cohort closes, on the public status host and the status page at `slop.ninja/testnet/status` for testnet; the artifacts were published when scoring started, and the commitment openings follow at the close. A separate signed record gives, for each scored entrant that is not withheld, whether it passed the span gate, and for the cohort whether the gate applied, was waived, or was voided (Section 3.2); it states these bits beside the result's own budget, and the plan-time bit budget counts them. Per-entrant span metrics stay private. The planned gate feedback record of Section 5.7 is to be signed and published the same way, beside the result, for a cohort whose plan names it. An entrant that fails artifact qualification is never executed and carries no score. Compact numerical inputs and calculated rows survive the detailed receipts' retention window.

The frozen calculation specifies exact arithmetic, operation order, tie-breaking, rounding, normalization, and integer chain encoding. It binds the registration identities used to map earned credit to UIDs; no chain runtime version is an input. Identical inputs must produce identical canonical rows on Linux and macOS; implementation-dependent floating-point drift is not an accepted tolerance. A registration change that requires a different conversion needs an authenticated compatible delivery revision; a chain runtime upgrade never does. Neither can silently change the scores or let a validator improvise its own normalization.

Released values are quantized aggregates and gate outcomes, never individual document predictions, exact scores, or hidden labels. Cluster assignments and near-copy statistics are coordinator measurements on hidden predictions that nobody outside the coordinator can check. Before it signs, the coordinator checks that each recorded gate and cluster decision follows from them and from the scores, that every entrant's probe comparison includes both references, that an entrant fails the content check exactly when its content digest is in the published-artifact index, that each closed cohort adds one fingerprint for every executed entrant, and that the row follows from the public result. Anyone can recompute the row from the public result and compare it with the signed row and the revealed weights; that check is advisory. Retain private evidence under access controls sufficient for authorized audit; public record verification does not prove that the coordinator executed the evaluation honestly. Corrections retain the original record and account for already-submitted commitments and weights. The signed row is published off chain as soon as it is signed, before any reveal. Native commit–reveal therefore supplies the chain's submission protocol without preventing off-chain copying of the row.

5.12 Mechanism mapping and unallocated rewards

The documented chain supports at most two native mechanisms with a shared UID space, and under Yuma Consensus the UID limit times the mechanism count may not exceed 256 [25]. Launch uses one for A. If C later activates, a candidate mapping uses native mechanism 0 for A quality and any paid-use credit, and mechanism 1 for grammar contributions, which halves the subnet's UID slots to 128. B needs a revised mapping before activation; three logical workstreams do not create a third native slot. C may activate before B; B's revised mapping must then also place C's contribution credit.

Let c be the fraction of total miner emissions assigned to C and f the total fraction assigned to paid A use, with $0 \le c < 1$, $f \ge 0$, and $c + f \le 1$. For separately normalized quality, usage, and new grammar-credit distributions Q_i, I_i, K_i, the intended allocations before reward caps and the handling of zero-work usage pools are

$$W_i^A = (1 - c - f)Q_i + fI_i, \tag{3}$$

$$W_i^C = cK_i. \tag{4}$$

Here Q_i is the `reward-v2` credit. The native A mechanism has share $1 - c$; its submitted distribution divides W_i^A by $1 - c$. Launch sets $c = 0$ and $f = 0$. A nonzero f requires the attribution and abuse controls in Section 8.

Unallocated credit goes to the UID of the subnet owner hotkey, which the chain burns. The coordinator reads the owner hotkey and its UID from the finalized snapshot it uses for registrations, at every tempo, and records that hotkey in the signed row. No manifest, package, or target names the owner hotkey, so an owner-hotkey rotation moves the burn destination at the next signed row without a new target or authorization. A validator that commits the row burns through the UID that the owner hotkey of its own finalized snapshot holds: it sends there both the row's burn entry, recorded against the coordinator's owner hotkey, and the units of any registration its snapshot no longer holds. An owner-hotkey rotation during coordinator silence therefore moves the burn at

each validator's next commitment, and a UID that changed hands while the coordinator was silent is never paid to its new holder. If units must burn while the finalized owner hotkey holds no UID, that tempo's row is deferred rather than sent to any other UID; a row with nothing to burn is still derived.

The burn has a documented native cost: before normalizing across subnets and applying its emission gate, the chain multiplies each subnet's TAO-side emission share by one minus the fraction of the last tempo's miner incentive that went to owner hotkeys, recycled or burned [18]. A partial burn, such as a departed hotkey's units, shrinks the share accordingly. While the row sends the whole pool to the owner hotkey's UID, as at launch, the subnet receives no TAO-side emission and hence no TAO or alpha pool injection. Under the same documented rules its participant-side alpha emission continues to accrue and is divided by the emission weight vector described below: the miner slot follows the row to the owner hotkey's UID and burns, while the validator and owner slots are paid as configured. The burn regime therefore removes miner pay and the TAO-side share, not validator or owner alpha. The native-chain rehearsal (Section 15.2) must confirm these effects. Off-chain allocations cannot guarantee precise realized payouts, freeze native emissions, or claw back rewards already distributed.

The split of subnet emissions among owner, validators, and miners is the subnet's emission weight vector. The subnet owner may change it at most once a week, and root at any time, through the chain's subnet-emission-weights call, within chain bounds: the owner slot at most 33% (and at least 18% for a subnet created through the chain's lease mechanism), the miner slot at least 33%, and the validator slot at most 50% and above root stakers' baseline share [26]. The mainnet split is not selected. The testnet subnet (netuid 601) uses an operator configuration that places the validator share just above that baseline, its chain minimum, to exercise the owner's path; it implies nothing about mainnet.

6 Cohorts, transitions, and recovery

A cohort is a frozen roster of model versions scored on one fresh hidden batch. It lasts at least K subnet tempos, frozen in its manifest. The launch value is $K = 20$. A tempo is the subnet's epoch length, 360 blocks; at about 12 seconds per block it lasts about 72 minutes [27], so a cohort takes about 24 hours and the network completes one cohort per day. On mainnet, batch model APIs could generate a 4,000-case batch more cheaply, but they queue requests with no stated completion time. Until an authorized phase for generating ahead exists (Section 15.2), a cohort that uses them may run longer than a day; generating at standard prices instead costs more. Rehearsal manifests may use $K = 4$ so that deletion and recovery tests do not take days. The tempo itself stays at 360 blocks: a longer tempo would delay every native commit–reveal by at least a full epoch and stretch the owner's hyperparameter rate limit.

Each phase advances at a tempo boundary once its work is durable. Admission and the transition also have frozen minimums, and the whole cohort cannot end before its K tempos. Generation, scoring, and publication have no minimum: they advance on completion, in order, and may share a boundary. If work takes longer, the same cohort advances at a later boundary [27]; nothing is skipped or discarded. Evaluation and publication are coordinator-owned work; each validator's submission and finalization record is separate. Other validators' acknowledgments, stake adoption, and software versions never gate cohort progress. This includes the coordinator's own chain delivery: its serverless evaluation and scheduling can continue while a weight submission remains unresolved.

Table 4: Phases of a cohort with $K = 20$ on the mainnet calendar; the first standing testnet cohort gave 16 of its 20 tempos to admission (Section 14.2). Tempo positions are nominal targets counted from the cohort's opening boundary.

Phase	Tempos	Permitted work	Advancement
Admission	0–1	Accept finalized digest commitments and presigned R2 uploads of artifact files; freeze the roster.	At least one tempo.

Phase	Tempos	Permitted work	Advancement
Generation	1–3	Draw human texts under the frozen pool manifest, generate fresh AI-side text in parallel lanes, and seal the batch and roster digests.	On completion.
Scoring	3–15	Publish every admitted artifact as scoring starts; score every entrant, R_k, and R_{k-1} in pairs in parallel lanes, then each executed entrant's probe run; work is sized to at most half of this window; release segment readings, and show pre-finalized quantized scores as models finish.	On completion.
Publication	15–16	Bundle receipts; seal and sign the private score package, the receipt index, and the public quantized result.	On completion.
Transition	16–20	Issue no new challenges. Validators process their chain obligations independently. At the closing boundary, publish the openings and the quantized result's remaining values.	At least two tempos and the cohort minimum.
Paused	any	Retain state and receive evidence from already dispatched work. Start no new challenge and advance no cohort.	Reconcile the saved state and chain, then authorize resumption of the same cohort.

The signed cohort closure records the epochs at which admission closed, publication completed, and the cohort closed. The coordinator checks them against the manifest's frozen admission, transition, and cohort minimums before the cohort's target activates, and the publisher checks them again before it lists the cohort. Every manifest, rehearsals included, freezes an admission minimum of at least one tempo and a transition minimum of at least two, together no more than K, and a reward-intent renew-by hint of at least $2K$ tempos, which never pauses anything. A package finished by tempo 18 keeps the daily cadence. A later one moves the next cohort back by whole tempos. Boundaries are finalized increments of the chain's epoch index, not block numbers modulo 360, because epochs can slip. A block handler need not run at an exact block. A late or missed delivery delays progress; it does not invalidate work, and a wall-clock timer cannot itself advance a phase. The next cohort requires a fresh coordinator start record after the current transition completes. Future submissions may queue without replacing an artifact in the frozen roster. Validator delivery can continue across later cohorts without extending the transition.

6.1 Lagged payout

When cohort N's package produces a new signed calculation target T_N, a *cohort target*, it activates at the first eligible tempo boundary after its package and retained receipts are durable: the boundary that closes cohort N, nominally the opening of cohort $N + 1$. Cohort N is therefore paid during the next day while cohort $N + 1$ is scored. From activation, at every tempo, the coordinator derives T_N's row against a fresh finalized registration snapshot and signs a new row record whenever the paid entries change, and each validator commits the current signed row under a per-tempo operation identity, with native commit–reveal enabled. That is 20 commitments per validator per day, well inside the current rate limit, and a validator that misses a few tempos stays active. Commitments made before activation reveal under their original rules. There is no per-validator activation gate: no validator must prove coverage of T_{N-1} before T_N applies.

During coordinator silence no target changes; validators keep committing the last signed row of the active target for as long as the silence lasts, since no reward-intent authorization expires, and cohorts are delayed, never

skipped. A new miner's first payout arrives about 25–49 hours after its commitment, so the subnet's immunity period must exceed that interval. That is necessary but not sufficient: when pruning a non-immune neuron would leave too few non-immune UIDs, the chain prunes the lowest-emission immune neuron instead [28], and under the burn row every miner's emission is zero.

A package that produces no cohort target carries forward the target already active (Section 6.4). Until a cohort target activates, that is the burn row, which sends the whole pool to the owner hotkey's UID. A launch authorization, which names no manifest or package and can authorize only the burn row, lets validators commit it from the first tempo of the first cohort, before any manifest is sealed. Like every reward-intent authorization it never expires, and it is accepted only while the burn row is the active target. While it is, each package without a cohort target also authorizes the burn row, as the target already active, and the launch authorization covers the tempos after the package that produces the first cohort target is sealed and before that target activates. From that activation every validator refuses the launch authorization, at each tempo's commitment as well as when checking it, so it is never accepted again; later packages without a cohort target carry forward the activated target, so within a record nothing authorizes the burn row again. Only an operator's switch to burn issues a new launch authorization, for a fresh record (Section 6.6).

Validators commit the burn row every tempo until some cohort first has a gated entrant whose released score reaches the margin cutoff. If that cohort also promotes an entrant, R_0 becomes the predecessor that later fallback cutoffs use (Sections 5.6 and 5.9). If it promotes none, its target pays its retained clusters, R_0 still has no predecessor, and each later cohort in which no released score reaches the margin cutoff produces no new target, so that first target stays live until a later cohort reaches the margin cutoff. Native validators commit the burn row too: their adapter records an explicit, journaled policy for submitting a row whose only positive entry is the owner hotkey's UID, used for the burn row under the launch authorization or a burn target, and only when the chain's weight limits and the validator's own permit allow it; otherwise that tempo defers. At the current `rtao` pin every other row whose only positive entries are the owner's or the validator's own UID is held instead, as for a cohort target whose credited miners have all left: the validator defers each such tempo, commits nothing, and leaves its previous weights on chain. Those weights can still pay a UID's new occupant, which the no-transfer rule below forbids, so committing such a row natively is a launch requirement (Section 15.2). That regime has no bounded duration; no miner earns during it, and the burn withholds the subnet's TAO-side emission, while under the documented rules validators and the owner still receive their participant-side alpha (Section 5.12). Once the subnet's UID slots are full, a new registration can therefore replace any miner past its immunity period; the credit of its entries still awaiting payout then burns (Section 5.9), and re-registering starts a new registration generation.

6.2 Fan-out evaluation work

Bulk evaluation work runs as fan-out jobs: challenge generation, paired scoring and probe runs, receipt bundling, and the per-model summaries that packaging needs. The cohort's Durable Object remains the only authority. It records each lane's assignment and holds the task leases, and lanes claim and commit work through short, idempotent requests to it. Each lane runs in its own Durable Object invocation with its own platform limits, and each scoring lane drives its own Container instance. Lanes number at least one per scored artifact, and more as the batch grows, within the account's Container limits [29], the providers' model rate limits through one shared limiter [21], and, on testnet, the cohort's budget. Lane count changes elapsed time and per-lane start cost, never a document's outcome: on one CPU class the scorer's outputs are bitwise identical across instance sizes and session settings (Section 14). Live scorer hosts fall into two arithmetic classes whose outputs differ in their last bits, at most 8.4×10^{-7} on the probability scale on the texts probed, with labels and span offsets exact, and a lane does not choose its host's class. Such a difference can move a document's utility only in digits far below the score grid and the release noise. Lanes never sign; signing stays in the cohort object's phase transitions.

Containers start on demand, and nothing runs warm at rest; scoring instances may start just in time during the tail of generation, and cold starts are accepted. A Container crash or out-of-memory failure counts against an artifact only when it reproduces in single-document isolation (Section 5.11). The testnet roster is capped at 40 entrants. Mainnet has no roster cap beyond the subnet's 256 UIDs and no spending cap. Providers publish budget alerts but no hard spending cap, so on testnet lane caps and per-cohort budgets bound spending.

6.3 Block notifications and finalized recovery

Live `rtao-rpc` webhooks on the finalized stream deliver every finalized block, each to its own Worker invocation. That delivery is the coordinator's primary heartbeat and clock. Deliveries are signed, at-least-once, and retried within a bounded provider journal. The provider retries a delivery that receives no success response ten times, with backoff, over about 5 hours 49 minutes, and then pauses the subscription; nothing more is delivered until it is resumed. The coordinator's handling of authentication, retries, ordering, and retention still requires qualification. Authenticate ingress and persist the requested work before acknowledging delivery. Duplicate, delayed, or reordered notifications must not repeat a transition, sign another attempt, reset a deadline, or skip a cohort. A notification neither authorizes an action nor establishes that its block is finalized.

Retain a finalized cursor with height and hash, plus completed event identities, in Durable Object storage. Reconcile chain history through RPC, checking hash continuity and processing missing finalized blocks in order. Advance the cursor only after the dependent state changes are durable. Best-head notifications may be reorganized; they cannot advance finalized state. Any conflict with the saved finalized history pauses affected actions for diagnosis. After the cursor advances, an invocation claims due work through a durable lease with an idempotent operation identity, holder, and expiry; overlapping invocations skip claimed work, and later blocks re-claim stale claims and continue resumable steps.

A Cron Trigger [6] every minute is the only fallback and the only timer. It is a no-op while valid deliveries arrive. If none arrives for a configurable threshold, five minutes by default, or the cursor lags the finalized head by more than that threshold, the coordinator enters a durable fallback mode. Each tick then reads the finalized head from public OpenTensor RPC and processes blocks from the cursor through the same per-block path and deduplication, bounded per tick within public rate limits. Delivery resumption returns it to primary mode automatically. A paused subscription keeps the coordinator in fallback mode: the watchdog reports the fallback state and the time since the last valid delivery for an operator alert, and an operator resumes the subscription. Automatic resumption is not yet implemented, because it needs a subscription-management credential in the Worker or a provider resume path; until then a pause lasts until an operator acts, a stated limit. The nominal wait is one finalized block plus delivery latency, or up to the threshold plus one tick in fallback. Actual observation delay also includes scheduler delay, RPC and finality latency, and catch-up work; this is no hard recovery bound during an outage. Bound each invocation and persist its continuation, so a long outage needs no resident subscriber or unbounded catch-up request. Resume scheduling at a later eligible tempo boundary after recovery, preserving the current cohort and its remaining active-time budgets. Pending chain obligations have separate recovery records; advancing the block cursor does not require their completion.

6.4 Authorization and durable dispatch

Coordinator-owned deadlines use a durable active-time clock. Bounded progress intervals advance that clock only when recorded in an authoritative checkpoint; missed intervals are not charged retroactively after recovery. Silence therefore preserves the remaining budgets for phase timing and coordinator-set submission windows without requiring a pause write during the outage. Recovery publishes the remaining budgets and revised eligible tempo boundaries. Extending an open window cannot reopen a frozen roster or a finalized outcome.

Permission to act is separate from that clock. New cohort work requires a signed authorization bound to cohort, phase, and checkpoint generation. It has no expiry: it stays valid until a newer generation supersedes it or a signed revocation withdraws it, and it fences coordinator work only. Weight commitments use a reward-intent authorization instead, a separate record signed under the authorization key. Each package is sealed with one that names the cohort's manifest and package by digest and the target active after it: the package's new target, whose predecessor must be the target active when the package is sealed, or, when the cohort produced none, the target already active. Validators receive the authorization, published beside the coordinator's row records, but never the package. Until the first cohort target activates, a launch authorization covers the burn row (Section 6.1). No reward-intent authorization expires: it yields commitments from its issue epoch for as long as its target is the validator's active target, and stops only when another target activates or a signed revocation withdraws it. A coordinator outage of any length therefore leaves validators committing the last signed row. Each record carries a renew-by hint, frozen in the manifest at no less than $2K$ tempos, that never refuses or pauses anything; a renewal only refreshes that hint, and none is required. Revocation is explicit: a signed revocation names a target and withdraws every authorization of it issued at or before its epoch, and every validator journals the revocations it has checked.

Validators verify each authorization, the coordinator's row record under it, and the revocation set under the pinned authorization key before using them. They check that the row names the authorization and a target that is their active one or activates later, and that the authorization's issue epoch is no earlier than the cohort's opening. Each validator, the coordinator included, keeps its own durable record of the target it holds active, written only forward. A validator other than the coordinator accepts a row only for that target, with the same activation epoch, or for a target whose activation epoch, as the signed row records it, is later than the held target's, and then records the row's target as active. One that missed several activations therefore moves straight to the current row's target, without the rows or packages in between. The coordinator moves its record only through each cohort's checked closure, so each target it activates names the target it held as its predecessor, except at an operator's restart (Section 6.6). A launch authorization is accepted only while no cohort target of its record has activated: the first one before any cohort target, or a restart's, issued at the restart floor after the target the validator holds. At every tempo a new commitment also requires the authorized target to be the one the validator holds active and the epoch to be at or after the authorization's issue, so an authorization stops yielding commitments once another target activates. Existing chain obligations retain their recorded authority and native deadlines. If a work authorization is superseded or revoked, or its checkpoint cannot be verified, no new coordinator work begins until the live coordinator issues another; this never ends or skips a cohort and never touches validators' commitments. Acting under a work authorization requires a fresh finalized-chain view, not a cached height; staleness pauses that action without ending the authorization. One non-coordinator validator's silence never stops the shared clock.

Before dependent actions, retain the manifest, roster, artifact and credited identity bindings, task and attempt identities, completed and outstanding work, remaining active-time budgets, scores, and chain receipts. Keep referenced artifacts and incomplete-cohort records without automatic expiry. Durable recovery exports and restore rehearsals must cover pauses longer than any provider backup window.

A lane's assignment is persisted with its state transition before its wakeup is sent. The lane claims each task under a lease, checks current authorization and generation, then retains its result before releasing the claim. Wakeups are notifications; authoritative records reconstruct dispatch after redelivery, loss, or retention expiry. The coordinator's state does not depend on a surviving message, lane invocation, Worker process, or always-on machine.

Task leases and compute attempts have separate identities. Lease expiry fences an old worker from new actions but does not erase its evidence. Reconcile an uncertain external call before starting another attempt. Reclaiming a task or recording an already durable answer consumes no compute retry. Starting another computation does. A frozen attempt-order rule selects one canonical outcome; never choose the best answer returned across retries. The late result of an old worker cannot advance a newer checkpoint or reopen a finalized score.

6.5 What pauses and what continues

Coordinator downtime suspends protocol progress and cohort scheduling. It does not remove unfinished tasks from the denominator, turn them into miner failures, reroll challenges, or start a replacement cohort. Already dispatched work may finish and persist evidence, but its completion cannot authorize another task. Resume only after reconciling in-flight attempts and late results. If scoring was final before the outage, resume publication or the transition without rescoring. Missing evidence leaves the cohort paused until a reviewed resolution is possible or the operator abandons it (Section 6.6).

A per-attempt compute limit remains the bound frozen for that attempt. Queue delay, operator suspension, and unresolved infrastructure faults are not measured model compute. A limit violation already established remains an outcome. Customer delivery/refund terms, credential expiry, transaction mortality, and native chain reveal windows retain their own clocks. Extending a coordinator-owned deadline cannot extend those obligations or renew an expired credential.

6.6 Operator decisions

No precondition in the evaluation service, the coordinator, the publisher, or a validator may lock the operator out. Each has a journaled operator path past it, so the operator can always decide to start a new cohort, change the references, install or pause a cohort template, or restart the record without first changing code. Moving forward never requires reading retired or superseded state, and a paused cohort calendar stays paused, across a template

install, until the operator resumes it. These paths keep the protocol's other rules: no cohort is skipped or expired because time passed, cohort numbers are never reused, each validator signs at most one commitment per chain, subnet registration, hotkey, and epoch and never resubmits blindly, validators keep committing the last signed row, and no decision recreates a lock it removes.

Each decision is journaled before it takes effect and leaves a signed, content-free public record that states its reason. Decisions that decline a result, change reward inputs, or restart the record are also signed with a separate operator-decision key, kept offline with a recorded backup. That key is distinct from the coordinator's hotkey and from its authorization and receipt keys, and only the coordinator and the evaluation service verify it; validators keep verifying rows, authorizations, and revocations under the pinned authorization key alone. Rotating the operator-decision key is a recorded configuration change of those two services that needs no signature under the old key, so losing it locks nothing. The paths are these:

- *Abandon.* The operator may end any cohort that has not closed, including a queued cohort or one with no fault. The abandonment record names the cohort's phase and digests, the cohort produces no target, and its number is never reused. An abandonment once scoring has started, when pre-finalized scores may already be public and the outcome may already be known, declines that outcome and needs the operator-decision key; a cohort stopped by a fault can still be abandoned for that fault without it. An abandoned cohort never closes. If its scoring had started, the coordinator queues every admission copy of its entrants for deletion as soon as it is abandoned, without waiting for publication, which runs entrant by entrant after scoring starts: an artifact already published stays published, and one not yet copied is never published. Its entrants do not join the published-artifact index, gain no probe fingerprint, and have no commitment opening published (Section 5.10), so a later entry of the same inference content, by its author or anyone else, is executed and scored like a new model. What such entrants should keep, and whether an abandonment should first let their publication finish, is settled before mainnet (Section 15.2).
- *Refusal to pay.* The operator may refuse to pay a finished cohort by declining its closure, by abandoning it late, or by naming it in a switch to burn, each under the operator-decision key. A declined result is never installed. Later targets chain from each cohort's target, so continuing after a refusal goes through a switch to burn and a fresh record.
- *Start over.* To stop paying the current record, the operator switches the coordinator to burn. With signing paused, the coordinator sets a restart floor one epoch after the last finalized one, installs the burn row from the floor under a launch authorization issued at that epoch, and names in the signed restart record every cohort that opened before the floor without being handled, so no cohort is dropped silently. The operator then opens a fresh record. Its first manifest names its first reference, with no predecessor: R_0 itself, or, once the span-equivalent decision of Section 5.6 has passed R_0's position to SlopNinja's first span package, that package. This holds on mainnet too, where a restart therefore returns to R_0's position, whether R_0 or that span package occupies it, rather than to the reference that mainnet's first record opened with; a reference decision with a recorded reason can then set another. Winners of the earlier record are never paid again. Validators accept the restart through that launch authorization, which was issued after the target they hold, and refuse rows from before its floor; none needs a manual reset.
- *Reward-rule change.* A new reward rule takes effect only through a fresh record under that rule, reached by the same switch to burn; no record carries across reward rules.
- *Reference decision.* The operator may set the current and previous references directly, with a recorded reason, outside a fresh record (Section 5.6).
- *Re-registration.* If the subnet is registered again on the same chain, the restart also rotates the authorization key, so no authorization or row signed for the earlier registration verifies under the new one; validators receive the new key through a signed release or trust update.

These paths remain available on mainnet, with the same signatures and public records. No outage triggers one: coordinator silence pauses the cohort (Section 6.5), and only an explicit decision ends, declines, or restarts. A decision changes only future work and payment. Commitments already submitted keep their recorded authority and reveal under their original rules, and rewards already paid are never reassigned. The decisions extend the trust the pilot already places in the operator (Section 13). A legal removal of published material (Section 3.4) is a recorded operator action outside these paths: it changes no reward input and needs no operator-decision key.

6.7 Coordinator rows and commit–reveal

The coordinator alone measures entrants, computes the reward outcome, and derives the native weight row. Its private score package (Section 5.11) never leaves its storage; its public quantized result carries everything needed to recompute the row. Once a tempo, from its own finalized registration snapshot and independently of its own commitment, the coordinator derives the active target's row and signs a new row record whenever the paid entries change. The record binds the deployment and chain, the reward rule, the cohort with its manifest digest and public result, the target and the authorization it is committed under, its epochs and derivation block, the owner hotkey, and, per paid UID, its hotkey, registration block, and integer weight. No chain runtime version is bound. The record is published off chain beside the authorization and the signed revocation set as soon as it is signed, before the coordinator's own hotkey signs anything, and no validator waits for the coordinator's commitment. The calculation fixes numeric precision, ordering, tie handling, and native quantization, so the same public result and registration bindings produce identical canonical UID/weight entries. Held-out texts, labels, per-document predictions, exact scores, receipts, customer content, and unpublished model weights stay coordinator-private, except that held-out passages go to the hosted generation providers of Section 5.2. This disclosure boundary has the scoring trust assumption stated in Section 13. No zero-knowledge proof is required.

Each other validator verifies the row record, its authorization, and the revocation set under its pinned keys, checks that the record names the authorization and a target it holds active or that activates later, journals the revocations, and commits the record's entries. Its only change to the row uses its own finalized snapshot: the units the row burns through the coordinator's recorded owner hotkey, and the units of any registration that snapshot no longer holds, go to the UID of that snapshot's owner hotkey, which burns them. When units must go there and that hotkey holds no UID, the validator defers that tempo's commitment; it never sends them to another UID. It needs nothing else to commit: it receives no score package, receipt, document, label, or exact score, recomputes nothing in order to commit, and downloads no model. Apart from the owner-UID deferral above, which skips only that tempo, two events about the row pause its new commitments: a signed revocation of its authorization, which it journals, until a newer authorization is installed; and an installed record whose encoding its release does not support, until a supported record is installed. A newly published record that fails the verification above, including one that names an earlier target, raises an alert and is never installed, so the validator keeps committing its last verified row while no valid newer row arrives. The commit–reveal flag check below, journal recovery, and the migrations of Section 6.10 can also pause it, for reasons unrelated to the row. Under the off-chain contract, a conforming validator does not invent a score, omit an entry, or apply an undeclared weighting rule; native consensus does not enforce it. Frozen reward policy remains common to all validators.

Anyone, a validator included, can recompute the row from the public quantized result and compare it with the signed row and the revealed weights. That checks the coordinator's reward arithmetic, not hidden scoring, and its result is advisory: a mismatch it finds never changes the row a validator commits and never pauses its commitments. The mismatch can be published as evidence; the coordinator corrects a defect it confirms only through a signed revocation or a newer signed row, which validators verify as above. A validator may also check models in its own way and publish its own scores, equally advisory. The operator prefers to keep native weight commit–reveal enabled, viewing its use as a positive signal within Bittensor and accepting the reveal delay. It adds no copy protection, since the signed row is published off chain before any reveal. The enabled flag check below enforces that configured submission policy. The recognized subnet owner sets `commit_reveal_weights_enabled = true`, through the agreed owner-authorized execution path under a lease; the operator verifies the effective setting before live submission [30,31]. Validators check that setting against finalized state at startup and fresh submission state before signing. An unexpected disabled flag pauses new weight commitments rather than silently selecting plaintext submission. Model-artifact digest commitments are separate admission records.

Validators, including the serverless coordinator, commit the coordinator's signed row under their own keys, nonces, journals, and timelock material. They target each eligible native commit window from the published schedule without waiting for another validator to commit, reveal, or finalize. Use the qualified native timelocked path, whose reveal is performed by the chain. A submitted commitment may therefore reveal during a coordinator outage or before a later audit completes. The client cannot assume it can cancel that reveal by withholding a second transaction.

Parallel commitments avoid serial reveal stages between validators; they do not remove native reveal latency or guarantee a same-tempo or atomic switch. A missed window defers only the affected validator's delivery. Reconcile its old attempts and current authorized target before scheduling another eligible window. Native submission and reveal clocks do not extend when coordinator deadlines pause.

The coordinator seals each complete evaluation's private score package and calculation target, and durably publishes the public result, row record, and authorization. Cohort completion records publication, not validator adoption or miner payment. At the next permitted boundary after the transition minimum, the coordinator may start another cohort even if every external validator is offline, outdated, or refusing to submit. Evaluation, service, updates, and recovery require no other validator's acknowledgment or cooperation. A newer target supersedes the active one only when the coordinator holds its complete authorized package and a verified receipt index, and only at its activation boundary. This activation needs no coordinator chain transaction. Revocation and identity changes still apply; the authorization itself never lapses.

After its own row is revealed and finalized, the coordinator publishes an observation certificate binding the score-package digest, coordinator identity and hotkey generation, canonical row, and finalized block reference. Anyone can compare this row and observed peer rows with the signed row records. Coordinator finalization is a later consistency audit, never a prerequisite for another validator's commitment. A missing or incorrect coordinator chain row is a delivery discrepancy; it cannot invalidate an otherwise valid signed row record or stop other validators. A shared row or encoding defect pauses another validator's new commitments only through the two row events listed above, a signed revocation or an installed encoding its release does not support, while correction and recovery proceed. Retain and reconcile commitments already submitted. Authorized coordinator hotkey rotation updates the chain identity while retaining score-signing authority and existing obligations under their recorded keys; it imposes no adoption gate. An owner-hotkey rotation needs no new target either: validators read the owner hotkey from finalized state at each tempo (Section 5.12).

Commit–reveal hides native submissions until reveal. The coordinator's row is published off chain as soon as it is signed, before any reveal, and also follows from the public quantized result, so this design does not claim to prevent copying it or to prove that any validator checked it. Pre-finalized scores and readings (Section 5.8) show the row's likely shape earlier; they authorize nothing, and validators never read them. Native consensus still accepts each validator's own submission and does not enforce this off-chain contract.

Each validator retains the signed row records and revocations it installed, its signed attempts, commit/reveal material, and finalized outcomes. On return it reconciles every submitted or uncertain operation before acting on a newer target. A target never submitted may be superseded with its history retained; a new package cannot erase an outstanding commitment or justify blindly replaying an obsolete row. Catch-up must prevent an older pending reveal from silently overwriting a newer row. Submission-mode migrations preserve those obligations and use reconciliation before new signing. These constraints apply to that validator's delivery path, never to cohort advancement or another participant.

Subtensor keeps producing blocks and applying emissions, activity, and registration rules while this protocol is paused. Submitted validator rows do not switch atomically. The network depends on timely validator submission for the intended weight rows to influence native consensus. Missing or stale rows can change actual emissions even though they cannot delay application progress. Publish observed delivery and adoption separately from cohort completion. SlopNinja cannot force a row, freeze native emissions, or promise that a missed reward period will be paid later.

Bind each award to its original hotkey and registration generation, not only a UID. If that UID changes occupant, do not transfer the old credit; the share of a hotkey that is no longer registered under that generation burns through the owner hotkey's UID at each tempo. Preserving the cohort grants neither suspension of native emissions nor a right to replay or reassign rewards already paid. Retaining all scores and delivery records provides accounting continuity, not guaranteed deferred payment.

6.8 R2 inference receipts and retention

Each inference attempt has a content-free receipt in R2, including benchmark and probe calls, paid-service calls, retries, failures, and uncertain outcomes. The one exception is the coordinator's pre-flight, outage, and admission probes: fixed synthetic calls, such as one generation or one scoring run, that check whether a provider or the scorer answers, for example before a paused cohort resumes by itself. They carry no evaluation content, their

cost is counted with the cohort, and they write no receipt. Receipts bind the deployment, immutable invocation and attempt identities, model/version, status, usage, and private-content commitments. They expose no prompt, response, held-out label, parsed feature vector, or commitment opening. Receipts are written in bounded chunk objects. A cohort package indexes each chunk by object identity and digest and each receipt by its position; its immutable manifest fixes the cutoff, count, and commitment root. The coordinator's private score package binds that manifest. Receipts are coordinator-private audit evidence: they are never published or sent to validators, because an attempt's outcome class, which an artifact can steer, can reveal information about hidden documents. The coordinator installs a new target only after it verifies that cohort's receipt index, counts, and object digests. Receipt verification checks the package's integrity, not honest inference or hidden scoring. Cloudflare does not sign these assertions.

Evaluation receipts belong to their cohort. Paid-service receipt records enter the next open certification package under a fixed cutoff rule. An uncertain attempt remains explicitly pending; a later outcome is an appended update in a later package under the same invocation identity, never a rewrite of sealed evidence or another credited inference. Pending paid-service records cannot relax the requirement to account for every assigned evaluation outcome.

Detailed receipts and their R2 index for cohort N are deleted once cohorts $N+1$ and $N+2$ complete and, if cohort N produced a target T_N, T_N has been superseded; a cohort that produces no new target has no target to supersede, so its receipts are deleted once $N+1$ and $N+2$ complete. Completion means the coordinator's durable cohort completion, independent of any validator's weight adoption. This rolling window counts completed cohorts and superseded targets, not wall time or elapsed tempos; a coordinator pause cannot advance deletion eligibility, and a target kept live through cohorts that produce no new target keeps its receipts. Published entrant artifacts stay public, apart from recorded legal removals (Section 3.4); their admission copies are deleted once the cohort closes and publication is recorded, and the artifacts of R_k and R_{k-1} remain hosted for evaluation while they are references. Probe fingerprints are private evaluation records, retained with the probe while their probe version is in use; the published-artifact index of content digests is retained indefinitely. A durable serverless cleanup task deletes eligible objects, invalidates access, and records completion. Delete managed replicas and caches as well; do not retain a separate indefinite archive of these detailed receipts. Previously downloaded copies cannot be recalled. The testnet evaluation service runs this deletion. One gap remains as of November 11: the receipts of the cohorts run just before the testnet record restart (Section 14.2) are not yet deleted, because the step that lets the new record release them waits on an older record's store that no longer answers.

Two completed successors are a deliberate pilot retention choice: they keep recent audit evidence available while limiting the number of completed cohorts whose detailed metadata remains hosted. With daily cohorts, receipts become eligible for deletion at least about 48 hours after their cohort completes. They guarantee neither a minimum audit duration nor a byte limit, particularly during a coordinator pause. Roots and counts cannot reconstruct deleted receipts, which limits later investigations.

Retain the signed score package and calculation inputs, the public result, package root and counts, canonical row, signed row records, cohort completion record, any coordinator observation certificate, and chain/accounting recovery records. They remain after detailed receipts are deleted. A returning validator uses the current signed row for new commitments and its durable journal for earlier submitted obligations; neither needs receipts. Missing or corrupt receipts within the retention window stop only the coordinator from installing that cohort's target; validators keep committing the last signed row. Customer content, delivery, and refund records retain their separately disclosed policies; this window grants no extra content retention.

6.9 Runtime upgrades and chain intents

Freeze evaluation meaning separately from operational builds. The models, batch, labels, scoring rules, and numerical tolerances define the comparison. The current RPC connection, coordinator build, and Finney runtime are recorded execution facts; they need not remain identical for the cohort to finish. An operational fix can preserve the frozen comparison.

No component gates on a runtime version, and no cohort, package, row, or journal is certified for one; a cohort survives any number of upgrades unchanged. Validators and the coordinator keep running through every upgrade. Use finalized history for decisions and reconciliation. Before submission, refresh runtime metadata and signing parameters, and read current submission constraints and epoch state from one fresh best-head snapshot.

Resolve each call, storage item, signed extension, and runtime API by name in that metadata and type-check the supplied values against the live types; repeat at signing, since finality can lag an upgrade. A mechanically safe difference, such as a wider integer or a new trailing optional argument, is encoded adaptively and recorded. An operation with no type-correct encoding is deferred alone with a durable diagnostic and re-evaluated on every new block, while ingestion, journaling, recovery, reconciliation, scoring, and package verification continue. Never submit a payload that does not type-check against live metadata. A runtime version is recorded only as an observation; signing reads it fresh. The native adapter follows this rule at its current `rtao` pin, except that upstream still composes weight calls and reads some storage through compiled descriptors; the "Implementation boundary" paragraph in Section 14.2 records how those cases defer.

Persist a logical chain intent and its signed attempts before sending bytes. Retain the cohort/result revision, participant binding, intended effect, nonce, observed runtime, transaction hash, and commit/reveal material. A timeout means an unknown outcome. Reconcile finalized execution, pending transactions, nonce state, and reveal obligations before signing another attempt. New bytes required by a changed runtime, nonce, or mortal lifetime remain attempts at the same intent. They do not authorize a duplicate effect. Decode historical evidence using metadata for its execution context.

The native adapter integrates `rtao`'s durable journal, scheduling, semantic checks, and signer-free finalized recovery [1, 32]. Opening it and beginning external recovery persist pause; resumption is explicit. A successful recovery call may leave unresolved work. Inspect per-operation outcomes and retained reveal evidence before claiming completion. Current supported operations and implementation gaps appear in the "Implementation boundary" paragraph in Section 14.2. Adapter updates must retain the original cohort and journal. The serverless coordinator needs equivalent durable intent and attempt records in Durable Objects, exclusive signing ownership, and qualified Workers transport and cryptography. A native file journal or passing native test does not establish this integration. Recover the coordinator's submitted obligations before new signing without making that recovery a barrier to unrelated cohort work.

6.10 Automatic updates and resumable migrations

Automatic updating is selected for the pilot. Whether participants may opt out or defer installation, and how releases are staggered across installations, remain deployment-policy decisions to settle before launch. If local deferral is permitted, it cannot bypass signing compatibility or pending-obligation recovery; rollout sequencing cannot require other validators' acknowledgments or gate cohort progress on their adoption.

One Rust supervisor per native installation stages signed releases and owns process handoff and restart. Miner and validator applications automatically fetch and verify release manifests and matching platform artifacts. The manifest binds artifact digests, protocol compatibility, schema versions, migration order, and release-key generation. The updater rejects unauthorized or superseded release metadata under the published key-rotation and rollback policy. Each participant updates independently; neither installing a release nor resuming afterward requires other validators to acknowledge it. An exclusive journal lock and signer ownership permit only one active participant process to write that journal or sign with its hotkey; the supervisor fences the old process before handoff.

Retain ordered migration paths from every released persistent schema. A participant that missed many releases applies every required migration in order, even if intermediate application binaries need not run. Each step has a durable progress record, validates its input and output, and resumes safely after interruption. Pause local new work and signing, fence the old process, and preserve a recoverable copy before changing state. Never replace a journal with an empty one, discard pending commitments, or blindly restart an old binary against a migrated schema. After migration, recover finalized chain history and outstanding obligations before the updated participant creates new intents. Rollback may use only a binary compatible with the current persisted schema, or a separately qualified reverse migration. Retain the latest journal and chain evidence; restoring an older backup cannot erase newer obligations.

Coordinator deployments use the same versioned, resumable state discipline in serverless storage and fence obsolete checkpoint and signing generations. Migrations preserve its own chain journal, hotkey access, pending commitments, and finalized cursor alongside cohort state. Operational updates must preserve frozen evaluation meaning and unfinished work. A missing migration or invalid signature pauses only its dependent actions with the evidence retained. A chain runtime upgrade pauses nothing; it can only defer an operation it leaves without

a type-correct encoding. Release archives, migration compatibility, and recovery after many offline versions are tested requirements.

Native validators use local journals; the coordinator retains its chain journal alongside its application records in serverless storage. Monitor journal capacity, retain completed evidence beyond bounded diagnostic history, and qualify migration before exhaustion. Never delete operation identities or remove journaling to bypass a refusal. A published correction keeps the original result, records its authority and new revision, and accounts for already-submitted chain effects. Changes to evaluation meaning belong in a future comparison; an invalid current score requires an explicit correction policy before rewards can proceed.

7 Customer content and model privacy

7.1 Public *A* and private *B* execution

The service must keep customer prompts, responses, reference writing, and derived representations inaccessible to miners. Usage metadata, including timing, token counts, and cost, may be visible. Private *B* weights must also be inaccessible for download by the general public. Cloudflare and SlopNinja are trusted with content; either may technically access hosted model weights. The proposal therefore offers no cryptographic confidentiality from those operators, no independent customer key release into attested hardware, and no TEE execution claim.

For *A*, SlopNinja runs a selected published artifact in its own runtime. The model contributor receives neither customer traffic nor an account with access to its storage. Artifacts submitted to a cohort are private evaluation material until that cohort's scoring starts. Miners upload them straight to the coordinator's private R2 admission storage through presigned URLs, protected in transit by TLS and at rest by the bucket's access controls rather than an added encryption layer, and they are never served to customers before publication. Customers may instead download published artifacts and run them locally, subject to their dependencies and execution contract. Such a run creates no hosted job, customer charge, or inference credit.

The proposed later *B* path gives a miner control of its private LoRA adapter and Cloudflare billing account while SlopNinja controls each request. The miner privately delegates an account-scoped Workers AI credential and registers the account, base model, and adapter UUID. SlopNinja stores the credential and constructs the direct Workers AI request itself. The customer receives neither that credential nor an adapter asset URL. The numeric ranker is uploaded separately to private SlopNinja storage and loaded only by our runtime. Neither account ownership nor a private adapter gives the miner a place in the customer's request or response path.

Tests found that the Workers AI Read grant also permits adapter-asset download. The design accepts that SlopNinja can technically read those weights. A dedicated serving account can limit the other assets exposed by this grant. Public asset denial, including attempts by an unrelated authenticated account, must be qualified before private serving. Keeping weights private does not prevent behavioral imitation or extraction through purchased outputs.

7.2 Direct serving and content logging

The runtime sends synchronous calls to Cloudflare's direct Workers AI endpoint for the enrolled account and base model, selecting the registered adapter UUID. It omits AI Gateway selection, including the `cf-aig-gateway-id` header, and accepts no miner endpoint, Worker, callback, or logging destination. It forwards no caller-supplied routing headers and follows no redirects to miner destinations. Provider-side batch jobs and stored conversations are excluded because they may create additional owner-readable result surfaces. Cloudflare's published data policy is an accepted provider assumption [33]; the direct REST and gateway interfaces define different request paths [34,35].

The reported owner-session inspection on October 22, 2026 found aggregate usage and no prompt or response text for the synthetic direct requests on the inspected dashboard, audit, and observability surfaces [36]. The cited record distinguishes that operator-supplied browser report from the direct API checks, which also exposed per-request usage metadata. No AI Gateway existed during those requests. This observation does not establish that an account owner can never enable content capture. Before paid *B* activation, run a controlled comparison with gateway logging enabled: a routed positive control must appear, and a direct request without gateway selection must remain unavailable. Examine owner controls that could capture that direct request. Provider changes

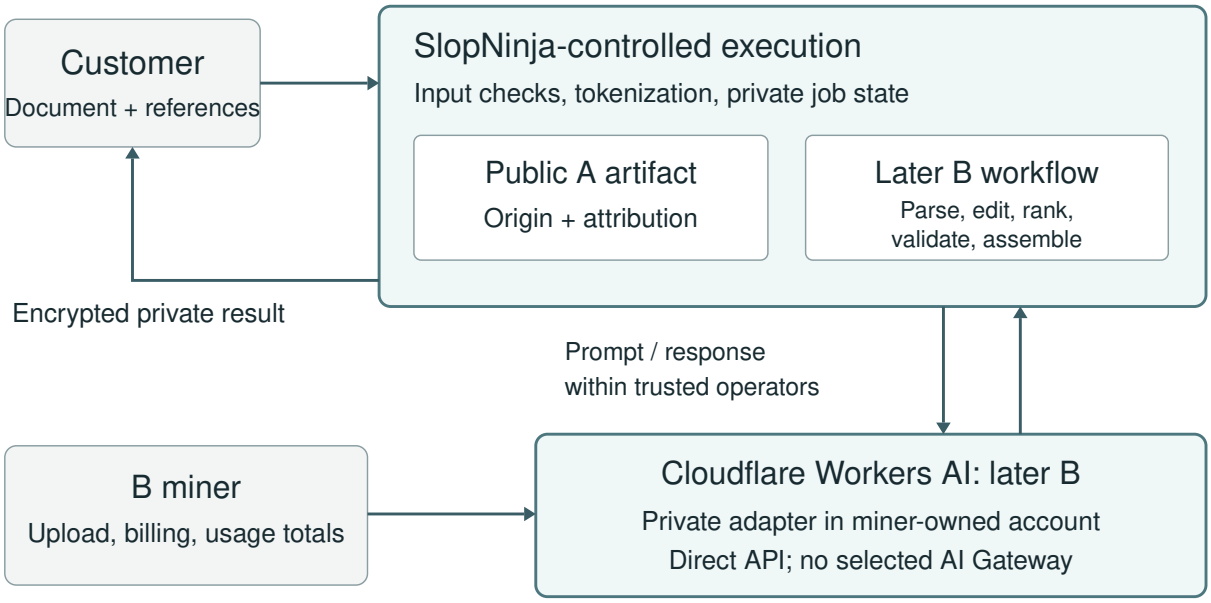


Figure 1: Proposed customer execution paths. Public *A* runs under SlopNinja's control. Later *B* uses a miner's billed Cloudflare account through direct operator requests. Content isolation from that account's owner requires the qualification described below; the diagram states the intended boundary.

require renewed qualification; absence from the surfaces inspected so far is limited evidence, not a universal privacy proof.

Returned identity associates a response with the selected adapter under Cloudflare's API behavior. It does not attest immutable weights or an unchanged hosted base model. Enrollment checks private asset digests, updates receive fresh version registrations and adapter UUIDs, and detected drift suspends service. Periodic checks cannot exclude a change-and-restore race. Receipts state the observed identity and enrolled digests without asserting a historical measurement that the provider does not supply.

7.3 Storage, isolation, and recovery

Customers submit over authenticated encrypted transport to SlopNinja. The runtime processes plaintext, then encrypts the result to the customer's bound output key for private retrieval. Private storage uses tenant authorization, encryption, and declared retention. Operational logs contain identifiers, usage, transitions, and normalized errors; provider errors must be sanitized because they can echo a prompt. Content logging is disabled in Workers, traces, and support tools. Profiles, vectors, intermediate candidates, and, for later *B*, parse trees and cached resolver outputs receive the same protection as the original text. *B*'s parsing and ambiguity resolution, including any LLM resolver, run within the same SlopNinja and Cloudflare trust boundary; miners receive neither resolver inputs nor outputs.

Each attempt binds its customer, job, assignment generation, model and pipeline version, output key, and resource limits. State from one customer must not enter another job. Recovery resumes only authorized durable state; reassignment cannot silently introduce a new content recipient. A lost key or unavailable result follows the job's failure and refund terms. Customer retention deadlines remain in force during an evaluation pause. Preserving an unfinished benchmark cohort indefinitely grants no authority to retain customer documents indefinitely.

Public receipts omit raw document hashes, filenames, titles, and report URLs. Coordinator-private R2 inference receipts, which are never published or sent to validators, also exclude prompts, responses, held-out labels, and

parsed vectors. Their index and deletion rule appear in Section 6.8; receipt access supplies no right to underlying content or commitment openings. Content commitments use canonical encoding, a fresh secret salt, and separate domains and lengths, for example:

$$C = \mathrm{SHA256}\big(\text{domain} \parallel \text{context} \parallel \text{salt} \parallel \text{length} \parallel \text{payload}\big). \tag{5}$$

Salts and openings remain private. The context binds the deployment and job; the output record also binds the customer's key and complete encrypted object's digest and length. A commitment proves consistency when opened, not execution, truth, or delivery. Result availability for the quoted retention period remains an operator service obligation.

Model outputs are untrusted data. The runtime executes no generated tools, URLs, or callbacks, and enforces schemas, budgets, and size limits. The result viewer renders inert text without remote images or automatic link previews. MCP delivery does not authorize commands embedded in a document. These controls also apply to hostile source text and grammar contributions. An independent security review before paid *B* launch must test routing, credential storage, content logging, cross-job isolation, and result access under this provider-and-operator trust model.

A malicious adapter could encode prompt information in its output length, exposing bits through owner-visible token counts or billed usage even when content logging is disabled. Review must test this active channel and assess fixed-size generation or output-size bucketing at the provider's metering boundary. Padding the returned result cannot hide counts already metered by the provider; a maximum output limit alone cannot close the channel. Ordinary metadata visibility is accepted, but a demonstrated path to recover customer content blocks paid *B* activation until addressed. Mandatory traffic padding is not otherwise part of the proposal, and passive metadata inference remains a stated limit.

7.4 Customer-directed disclosure

A dispute gives validators no automatic right to read customer text. A customer may authorize a separate disclosure naming the recipient, purpose, material, and validity window. Further recipients require further authorization. Partial evidence supports only a limited finding and cannot open a commitment to an entire document. Expiry limits subsequent authorized processing but cannot erase plaintext already received. Editorial review is advisory unless the accepted service terms specify a remedy.

External detectors, including Pangram, receive customer content only with separate authorization and funding. A service's benchmark record is not a live external-detector result for the customer's document. Customer material enters training only with separate opt-in; on-chain accounting and validator record checks create no reuse exception.

8 Paid inference, funding, and settlement

The first commercial service is hosted *A*: one admitted model returns estimated AI share, AI-involvement confidence, and author attribution within its qualified scope. SlopNinja serves a published, qualified artifact in its own infrastructure; R_0 is such an artifact once its public package is published (Section 3). Customers buy access through an API rather than operating the artifact themselves. A later *B* service would offer bounded revision through private models hosted in miners' Cloudflare accounts, under the conditions in Section 10.

The coordinator opens paid inference only after deciding that measured quality supports the advertised service. It publishes the accepted versions, supported workloads, limitations, and activation decision. Emission eligibility alone does not authorize a paid product. Each product needs its own cost, availability, privacy, and delivery qualification; book-length inputs remain outside an offer until those properties have been measured at that size.

The principal of a paid job, a quoted and reserved workload (Section 8.4), is converted to subnet alpha where needed and burned. It pays neither a direct miner fee nor an operator margin. The detection page's paid API is sold differently: its revenue joins the pot of Section 8.1, net of the fees deducted at source, before and after the subnet's launch. The economic hypothesis is that customer demand purchases alpha where conversion is required and removes alpha from supply through burns. Compared with retaining those tokens, this may support

alpha's market value for holders, including miners and other emission recipients. It guarantees neither price appreciation nor spendable proceeds sufficient to cover costs. This market-value channel is distinct from a possible reward allocation for verified paid use, which distributes emissions available under the chain's rules. Miners and the operator must fund their bills while any emissions they earn remain subject to chain allocation, market value, and liquidity.

The full-principal choice puts the entire inference price toward alpha burning, purchasing alpha when conversion is required. The operator's intended upside comes through its holdings and any owner or validator emissions it is entitled to earn. This commits the treasury to funding operations and refund reserves separately, accepting that exposure in pursuit of a benefit shared with other alpha holders. The policy's commercial viability remains to be measured.

8.1 Revenue before mainnet

Testnet runs for several months before the mainnet subnet is registered, and SlopNinja plans to earn revenue during that period. The first product is a free public detection page: a visitor pastes text without an account, and the page sets no cookie and stores no text. It runs SlopNinja's first span package (Section 3.2) in SlopNinja's own runtime, a single Cloudflare Container that sleeps while the page is idle, so submitted text reaches no miner, and no submitted text is reused for training without consent. The page opens once its serving parity, abuse controls, and wording have been checked, and it says that the package's public download is pending until it is published. Later, the detection products are to serve the reigning reference model (Section 5.6) whenever it passes serving checks that are published before the pot below receives its first deposit, so that a miner's model that has beaten the reference can earn revenue.

The page reports what A has measured and no more. Its headline is one of four evidence bands, set at the package's calibrated 1% and 0.1% human false-positive points and 50%, each shown with the rates of human and AI-involved Test documents that fell in it. In the band from 50% to the 1% point, human and AI-involved documents land at similar rates, and the page says that such a score carries little evidence. Under the band come the two document percentages, with a note that the confidence figure assumes a mix in which about two of three documents involve AI, and overstates the chance for a reader whose texts are mostly human. Human false positives were measured only on texts of 150 to 450 words, so the page scores no text under 150 words, and marks longer texts as outside the measured range until those lengths are measured. AI spans of at least 24 words are highlighted only in the two strongest bands. The page gives no human verdict, explains that web text and fiction are flagged more often than the headline rates suggest, and states that a detector output is not a determination of authorship.

No advertising or analytics script runs on the page that holds pasted text. Advertising runs only on the site's content pages, once an advertising network has approved them, with advertisers that sell text humanizing, essay writing, or detection bypass blocked, and with no ads on the pages for writers who were flagged, on the pot's page, or on policy pages. A paid detection API, priced at the hypothesis below, reached by key, and paid in advance, follows once the package is published, and other paid detection products may follow it. They are paid through a merchant-of-record card processor or directly in TAO and settle through the pot, before and after the subnet's launch, rather than one job at a time.

Everything that the detection products' advertising networks, sponsors, and payment processors pay out to SlopNinja goes into a public pot, before any of SlopNinja's own costs. SlopNinja pays those separately and never from the pot, except for the mainnet registration below. The payouts arrive net of the fees and revenue shares that networks and processors deduct at source, so, unlike the per-job route below, where SlopNinja funds processing fees and the whole price is burned, the pot receives what is actually paid out. A later product joins the pot only through a published change to its charter. The pot is held as native TAO in a published two-of-three multisignature account on the main Bittensor chain, whose signers are distinct from every subnet and operator key, one of them held by an independent person. Payouts are converted to TAO each month. A public page lists gross payouts, conversions, the balance, the registration, purchases, burns, and claims, each with its transaction, and the revenue credited to each model, together with a signed monthly statement whose hash is recorded on chain. As of November 11, 2026 the pot has received no money, so the rules below govern all of it.

The pot's money is used in three steps:

1. **Registration.** Once the pot holds about 1.5 times the cost of registering a subnet on the main chain, SlopNinja registers the mainnet subnet with that money, so that money remains for the purchases below. The subnet's owner account is SlopNinja's. Until then the pot's TAO is neither staked nor spent, and the pot waits for this point without a deadline. Any part of the registration cost later returned to the owner account, for example when the subnet is dissolved or deregistered, goes back to the pot and is listed on the pot's public page, so an abandonment burns it as well.
2. **Purchases for the testnet era.** Once the registered subnet launches, that is, once its emissions have started, the TAO left from revenue earned before the launch (testnet-era revenue) buys the subnet's alpha in daily tranches, each limit-priced and sized to the subnet's pool, and each bound to the registration by the subnet's network number, its registration block, and its owner account; if any of them no longer matches, purchases stop until SlopNinja publishes what happens next. Half of the alpha bought is burned. The other half is distributed to testnet miners on mainnet through a claim contract, in proportion to the testnet-era revenue earned by their winning models. A model counts as winning only once it has beaten the reference and taken its place (Section 5.6), so revenue earned while one of SlopNinja's own models is the reference is credited to no miner. If no miner's model was credited with any revenue, this half is burned as well. If only part of the testnet-era revenue was credited to miners' models, the attribution rule, published before the pot's first deposit, fixes how this half is divided: either the credited miners share all of it in proportion to their credited revenue, or each receives the fraction of it that its credited revenue forms of all testnet-era revenue, with the uncredited rest burned. Winning inherits the limits of a testnet promotion: it can rest partly on recognizing testnet documents or on release noise (Sections 5.4 and 5.6), so credit can reach a model no better than the one it displaced. The serving checks and the attribution rule, which credits each payout to the models that earned it, are to state what a promoted model must show before it serves and is credited; they and the claim contract's rules are published before they are used.
3. **After the launch.** All revenue from *A* and later *B* products buys and burns the subnet's alpha, with nothing distributed.

No pot money buys or burns testnet tokens. If SlopNinja publicly abandons mainnet, the TAO the pot holds, including any registration cost returned to it, is burned, and nothing is distributed. A change to what the pot's money may be used for applies only to money received after the change is published; for money already received, SlopNinja may re-plan the timing and size of purchases by a published notice, but not these uses. The charter, these rules, the attribution rule, and the wording below are reviewed by legal and tax advisers before the first deposit.

Advertising networks forbid inviting people to view or click ads, and a pledge that ties advertising income to burns or to a distribution could be read as such an invitation. So no SlopNinja page or post suggests that visiting, using, sharing, or clicking anything increases burns or what any miner receives, the detection page does not promote the pot, and traffic to pages with ads is watched for invalid activity, with ads switched off pending review when it appears.

The pot's purchases are separate from the per-job burns below. Those apply to paid jobs, the quoted and reserved workloads of Section 8.4, once the subnet's alpha exists, and the per-job card route through Coinbase Onramp remains the candidate for them after it is qualified. The distribution to testnet miners is separate from subnet emissions: it changes no score, weight row, or emission, and no validator checks it.

8.2 Pricing and cost calibration

The operator quotes a fixed price for a bounded workload using a versioned reference cost and multiplier:

$$P_{\text{USD}} = M_{\text{price}} C_{\text{model}}, \qquad C_{\text{total}} = C_{\text{model}} + C_{\text{other}}.$$

Here C_{model} is the expected cost of all model invocations per delivered job: execution of the *A* artifact and, for later *B* revision, ambiguity resolution of its parse, proposal, ranking, and required semantic checks. C_{other} includes *B*'s non-model parsing, orchestration, storage, delivery, payment and chain fees, and expected refund overhead. Numeric-model runtime is counted once. Training and mandatory benchmark research have separate budgets.

An exploratory value is $M_{\text{price}} = 25$. Detection and revision may need different values. The multiplier sizes customer principal destined for burning relative to reference model cost. Calibration must weigh the intended economic effect against competitive pricing and customer demand. It does not set an operator margin, fix the

share of emissions allocated to inference, or guarantee a miner's margin. Cost profiles bind model versions, workload limits, maximum attempts, tariff assumptions, and measurement dates. Overruns cannot increase an accepted quote. Repricing affects future reservations.

The initial detection hypothesis is $0.02 per 1,000 words, with $0.01–$0.03 as the first range to test. The following planning scenarios are not measurements of a qualified production service.

Service per 1,000 words	Model cost	Delivery subtotal	Price at $M_{\text{price}} = 25$
Raw-text detection	$0.0002875–$0.0004250	$0.0004–$0.0009	$0.007188–$0.01063
Bounded revision	$0.008289	$0.0106–$0.0112	$0.2072

An *A* detection job tokenizes the canonical text and runs the raw-text *A* artifact (Section 3.3); it has no grammar parse and no language-model call. Its model cost is the October 29, 2026 Container busy-time measurement, $0.23–$0.34 per 1,000 documents at the 2,048-token limit, used as the bound for a 1,000-word document within that limit [37]. The bounded revision scenario, dated October 22, 2026, uses published tariffs and assumed workloads for later *B*, which parses its source [38]: 8,000 input and 1,000 output tokens of batched ambiguity resolution on Llama 3.1 8B fast FP8, at $0.045/$0.384 per million input/output tokens; three proposal calls totaling 12,000 input and 4,500 output tokens at the Mistral 7B v0.1 tariff of $0.110/$0.190; an independent preservation check using 8,000 input and 600 output tokens at the Llama 3.3 70B fast FP8 tariff of $0.293/$2.253; and $0.00002 for numeric scoring and ranking, an allocation that predates the Container measurement. Both model costs include a 25% failure/retry allowance. Prices use the unrounded model-cost totals; model costs and prices are displayed to four significant figures.

The delivery subtotal includes model cost and adds $0.0001–$0.0005 per detection or $0.0002–$0.0008 per revision for orchestration, storage, and delivery. Revision also assumes sixty seconds of non-model parsing and runtime on a warm Cloudflare Container with one fully active vCPU, 6 GiB of memory, and 12 GB of disk, about $0.00215 at the published resource rates [39]. The subtotal excludes payment and chain fees, refund reserves, training, evaluation, external checks, and low-volume fixed costs. An idle Container of the standard-2 to standard-4 sizes costs $0.057–$0.113 per hour, so idle capacity and cold starts can dominate per-job cost at low volume. Private-adapter pricing remains uncertain beyond Cloudflare's advertised free LoRA beta; a base-model tariff is only a comparison point [40].

The dated comparison uses Pangram's standalone realtime rate of $0.50 per 1,000 words and its advertised bulk rate of $0.40, against the proposed $0.02 [41, 42]. Subscriptions, negotiated prices, per-item rounding, and differing detection performance limit this comparison. Prices must be checked again when an offer is published. At $M_{\text{price}} = 25$, raw-text detection prices at about $0.007–$0.011, at the bottom of the target range; a $0.02 price corresponds to M_{price} of about 47–70. Price is an independent product hypothesis, not evidence of comparable accuracy.

Calibration runs the whole admitted pipeline on authorized workloads at realistic concurrency. Record every attempt's opaque identifier, model, billed account, usage, latency, outcome, and tariff version without text. Reconcile usage with provider charges, separating cash charges, promotional credits, and undiscounted estimates. Divide the cost of all attempts by successful deliveries, and report failures and cost percentiles separately. Measure miner and operator expenses separately before selecting M_{price}, workload limits, or a paid-use emission share.

8.3 Checkout and the burn policy

The planned funding routes are:

$$\begin{aligned} \text{USD card payment} &\longrightarrow \text{TAO} \longrightarrow \text{alpha} \longrightarrow \text{burn}, \\ \text{TAO payment} &\longrightarrow \text{alpha} \longrightarrow \text{burn}, \\ \text{alpha payment} &\longrightarrow \text{burn}. \end{aligned}$$

For paid jobs, Coinbase Onramp is the selected candidate for card-to-native-TAO funding; the detection API and the products sold beside it use the merchant-of-record card processor or direct TAO payments of Section 8.1, and their proceeds join the pot. TAO.com's documented integration supports direct TAO delivery, but SlopNinja's

access, supported card types and jurisdictions, recipient wallets, limits, holds, and refunds still need qualification [43]. Credit-card support cannot be inferred from a documented debit-card flow. This intended route requires neither Ethereum nor a separate bridge. Solana is the preferred entry network for additional crypto inputs, pending a qualified route to native TAO.

The target is automated checkout. Onramp delivery, conversion, service delivery, and burning remain separate events with independent failure modes. Identity checks, finality, and liquidity can delay them. A card authorization or an expected exchange fill cannot fund execution; the adapter must confirm arrival at the authorized funding destination. Provider-hosted forms handle card details. Payment systems receive opaque job references, not customer documents or author samples. At cent-scale prices, minimum purchases and fees may require a funded balance spanning multiple jobs; that product rule remains open.

All paid-job principal follows the selected burn route. The operator funds processing, conversion, and chain fees separately, including any shortfall when a provider deducts fees from its output. Customers bear their own wallet submission fees unless the offer explicitly sponsors them. Optional external detector checks have a separate disclosed budget and require authorization to send the text. Their charges neither reduce inference principal nor earn additional inference credit.

A quote fixes the original currency and amount, conversion conditions, and slippage limits. It records an estimated alpha quantity; the final record uses actual fills and actual alpha destroyed. Native alpha's transfer and staking semantics, deployment permissions, and burn operations must be qualified against the runtime. Burning existing alpha does not itself buy TAO, and recycling is a different operation [44, 45]. If the deployed burn call is owner-only, a subnet lease must provide an authorized owner execution path or another verified route before sales open. Operating the service does not itself grant that permission. This is separate from the lease's required path for owner-controlled hyperparameter changes (Section 2). An address transfer or treasury allocation alone cannot substantiate a burn claim.

Funds remain refundable custody until valid delivery and the published pre-burn refund window have passed. Conversion may occur earlier if that custody and its refund liability remain fully funded. Burn batches allocate each job's principal uniquely and reconcile deposits, conversion fills, custody, refunds, and finalized burns. Using treasury inventory temporarily does not replace the promised purchase-and-burn accounting.

8.4 Reservation, execution, and delivery

The paid-job API and an MCP integration expose quote, reserve, submit, status, and result operations [46]. A quote binds the accepted pipeline and cost profile, price and currency, input limits, attempt budget, acceptance and delivery cutoffs, retention, and refund terms. Size and service requirements suffice for discovery. The document goes to SlopNinja only after reservation; later B miners receive no customer text through the discovery process.

1. Reserve confirmed funds, refund liquidity, and capacity durably before dispatch. A customer idempotency key identifies one job. An expired or exhausted offer cannot be reserved, and repricing cannot alter an existing reservation. Cross-ledger operations must reconcile before execution.
2. Validate the submitted document, format, size, and customer output key before acceptance. Construct the admitted input, canonical text under the launch profile, and run the selected A artifact. Check complete input coverage, schema validity, probabilities, share/confidence consistency, and attribution. An input beyond the artifact's length limit produces an explicit unsupported-length failure.
3. For later B, parse the source under the frozen grammar; an unresolved parse or exhausted ambiguity budget produces an explicit failure or a separately quoted diagnostic result. Then execute the bounded proposal and ranking procedure and check the assembled document for preservation, readability, requested tone, and any qualified author-fit objective. Local edit acceptance does not establish whole-document success. Customer-requested Pangram checks require separate disclosure authorization and funding.
4. Encrypt the result to the customer's output key, retain it in private storage, and issue an operator-signed receipt. Retrieval must remain available for the quoted period independently of a serving miner. Complete the required delivery checks before authorizing burn or inference credit.
5. After the pre-burn window, settle the principal's burn allocation once. Retain content-free accounting, return excess funds, and expire document data, derived artifacts, and caches according to the disclosed retention policy, including backup retention. Training reuse requires separate consent.

The exact delivery predicate is a paid-launch prerequisite. It must bind a durably stored encrypted object, the authorized customer/key, verified digest and length, working retrieval, completed validations, and timely delivery. Whether customer acknowledgment is required, and how acceptance interacts with the refund window, remains to be fixed in the customer terms. A response hash alone cannot authorize an irreversible burn. The offer buys a bounded computation and retrievable result; subjective quality guarantees or additional revision allowances require explicit terms.

Transactional Durable Object ledgers preserve reservations and transitions. Queues carry job identifiers; private content stays in the operator account. Persist each attempt's identity and maximum budget before dispatch. A timed-out provider call may have executed and incurred charges, so mark its outcome uncertain and reconcile before a bounded retry. Customer charges, burns, and credits remain idempotent even when provider execution repeats. Restarts do not extend accepted customer cutoffs. A cohort pause under Section 6 preserves research work but does not suspend delivery, retrieval, or refund obligations.

Switching to another model or later *B* provider requires either prior customer authorization that fixes replacement eligibility and budget, or a fresh quote and acceptance. A replacement cannot inherit the reserved model's benchmark claim. Retain the original attempt and its outcome when a qualified fallback completes the service.

8.5 Receipts and refund liabilities

A completion receipt binds the protocol and chain domain, job and acceptance, salted input commitment, customer output-key identity, enrolled model and pipeline, attempt-ledger commitment, encrypted result digest and size, deadline, and signer version. Linked payment records bind the quote, original-currency amount, conversion history, and refund state. Burn records bind finalized transactions and each job's allocation. Canonical encodings, signature verification, key rotation, and replay protection must be fixed before these records authorize funds.

Each inference attempt also has a content-free R2 receipt bound into the coordinator's signed cohort package. Section 6.8 defines its index, the coordinator's verification of it, and its deletion after two subsequent cohorts complete and the cohort's target is superseded. That rolling window applies to detailed inference receipts, separately from customer delivery terms, compact burn/accounting records, and unresolved refund obligations.

Content-free commitments may be anchored on-chain, with inclusion proofs for batches. Customer ciphertext, billing identity, report URLs, and raw document hashes stay off-chain. Use fresh secret salts for document commitments. The receipt states the operator's assertion about execution; it supplies neither a Cloudflare signature nor independent validator certification. Result availability remains an operator obligation. Later *B* receipts report the qualified identity evidence without claiming stronger historical model immutability than the provider supports.

Rejection before acceptance releases funds and capacity. Failure to deliver the agreed service requires the published refund; failed attempts still cost the billed account holder and earn no inference credit. Valid delivery after bounded retries keeps the original price. Cancellation after acceptance follows the fixed terms. A conversion or burn delay retains pending credit and follows its fixed cutoff. Reconcile finalized chain inclusion before deciding that a pending transaction failed; a local timeout cannot both return principal and leave an unresolved burn executable.

The proposed service refund restores the original amount in the original currency: USD through a qualified return path, TAO to the bound refund address, or alpha to its qualified position. Conversion losses and consumed provider fees fall to the operator. Card checkout cannot promise this until an actual USD return path is qualified; an onramp's completed crypto purchase may be irreversible. Custodial intermediate alpha is reconciled into the refund, not returned as an additional payment.

After a burn, a refund is a new operator-funded payment. Hold reserves for retention obligations, service claims, and uncovered processor disputes; reconcile chargebacks and manual refunds to avoid duplicate returns. Already paid native emissions cannot be assumed recoverable. Stop new admissions when required reserves are unavailable.

8.6 Who pays, and what paid use earns

Miners fund model preparation and training. SlopNinja pays to serve public *A* artifacts, including their Container execution, storage, coordination, and delivery. Later *B* miners pay Cloudflare for their own customer inference and bounded mandatory evaluation calls. They absorb permitted failures and retries as part of participation. Their required candidate Pangram reports use prepaid balances and a fixed budget; those deposits are separate from burned customer principal. Unused, uncommitted balances remain withdrawable.

The operator funds shared evaluation sets, source baselines, authoritative *A* execution, and independent review through named budgets. Validators fund their own record verification and native weight submissions; shared evaluation and service never depend on their funding or approval. The operator also pays conversion and burn fees, refunds, and its other service liabilities. Access to `rtao-rpc` has no service charge; transaction fees and the caller's runtime costs remain. Insufficient evaluation funding reduces new assignments rather than evidence requirements or accepted miner budgets.

These expenses require spendable operating funds. Customer principal is burned, so a high markup supplies no cash for invoices. Capital or explicitly allocated and realized owner proceeds may fund operations; future emissions are not reserves. Native owner emissions accrue under the actual ownership or lease arrangement and do not automatically fund SlopNinja or an investor [25]. Report operating cash, purchased and burned alpha, miner emissions, and separately priced enterprise services separately.

Section 5 defines the proposed quality and paid-use allocations. Inference credit requires an admitted version, acceptable delivery, and finalized allocation of paid principal to a burn. Pending burns retain pending credit. Failed attempts, free traffic, mandatory evaluations, duplicate receipts, and extra tokens earn none. Work units come from a frozen workload schedule. A published *A* artifact hosted by the operator also requires an explicit choice about model-author credit versus serving-cost credit before its calls enter a paid-use pool.

Assign usage to the epoch containing finalized burn inclusion plus a fixed protocol lag. Publish that lag, the applicable emission-share schedule, burn cutoff, and batch policy before admission. The operator cannot select a favorable reward epoch after observing its pool. Delayed or late burns follow the quoted resolution rule without creating duplicate principal dispositions or retroactive work credit.

Section 9 develops the burn hypothesis, cost-recovery conditions, and self-purchase incentives. Its sensitivity analysis separates miner participation from the operator's funding and reserve requirements.

9 Token economics and capacity

Paid demand, token value, miner rewards, and operating cash are separate quantities. Their relationship determines whether the service can expand. This section analyzes the proposed allocation before reward caps and native consensus effects. It assumes no future alpha price and makes no claim that the market will finance arbitrary inference volume. Checkout, delivery, burn finality, and refund obligations follow Section 8. The analysis applies once the mainnet subnet's alpha exists; revenue earned before the subnet launches accrues in the pot of Section 8.1, which first pays the subnet's registration. At least half of the alpha that the pot's remaining money later buys is burned, adding to the burns analyzed here, and the rest is distributed to testnet miners.

9.1 What customer demand changes

For N_{job} identical delivered jobs priced at $P_{\text{USD}} = M_{\text{price}} C_{\text{model}}$, the principal committed to burning is

$$D_{\text{burn}} = N_{\text{job}} M_{\text{price}} C_{\text{model}}. \tag{6}$$

Here D_{burn} is a USD valuation of customer principal, not operating revenue. For converted payments, if p_{buy} is the realized USD-equivalent purchase price per alpha, a planning approximation is $B_\alpha \approx D_{\text{burn}}/p_{\text{buy}}$. The ledger uses actual fills and finalized destroyed alpha. Direct alpha payments already specify the quantity to burn and need no new market purchase. Conversion fees and any shortfall are separately funded under the full-principal policy.

Purchases create demand where conversion is needed; burns remove alpha that would otherwise remain in supply. The intended benefit is support for token value, including the value held or earned by miners, validators,

and owners. This is a market hypothesis. The reduction is relative to a counterfactual without those burns. For total alpha supply under the deployed runtime, the stock-flow accounting is

$$\Delta S_\alpha = \text{gross alpha minted} - \text{all alpha burned}.$$

Any other supply-changing runtime operation must enter that reconciliation. Concurrent issuance can still increase supply. Burned alpha need not exceed emissions, and a burn does not establish net deflation, price appreciation, or immediate liquidity. Chain allocation and market responses can change together, so the analysis does not hold aggregate emissions constant as a protocol fact.

The multiplier M_{price} controls the principal removed for a given model-cost estimate. It therefore trades off the intended economic effect against the price customers will accept. It supplies no cash margin to the operator or serving miner. A usage reward separately distributes available emissions; the customer payment creates no direct claim to a share of its proceeds.

9.2 Rewards per unit of work

For one accounting period, let E_{miner} be the miner-emission allocation in alpha, f its paid-use share, and u_i participant i's verified paid-use work units. Write $U_{\text{paid}} = \sum_i u_i > 0$. The intended uncapped reward is

$$\begin{gathered} R_i^{\text{use}} = f E_{\text{miner}} \frac{u_i}{U_{\text{paid}}}, \qquad V_{\text{pool}} = f E_{\text{miner}} p, \\ \text{USD reward per work unit} = \frac{V_{\text{pool}}}{U_{\text{paid}}}. \end{gathered} \tag{7}$$

The parameter p is realizable USD per alpha after sale costs at the relevant sale size and time. It differs from the purchase price p_{buy}; liquidity, slippage, and timing can separate them. The common p is a scenario input, not a guaranteed quote available to every participant simultaneously. V_{pool} is consequently the estimated realizable value of the paid-use pool.

Equation 3 reserves c for later C, f for paid A use, and $1 - c - f$ for A quality. Increasing f reallocates this budget and can weaken the quality incentive; it does not create additional emissions. C starts at $c = 0$, and paid service can operate with $f = 0$. Applying this arithmetic to later B requires its own approved reward and native-mechanism mapping. Native normalization, consensus, and caps can change actual payouts from R_i^{use}. A zero-work usage pool, one with no paid work in its period, has no denominator; its treatment is fixed before the pool is enabled, never by assigning invented work.

For an approved B paid-use pool defined analogously, write its realizable value as V_{pool} for the following capacity calculation. For homogeneous jobs, choose one work unit per job and let C_b be the miner's variable cost per delivered job, including its permitted failed attempts. Then the pool's average payout is $V_{\text{pool}}/N_{\text{job}}$, and variable-cost coverage requires

$$\frac{V_{\text{pool}}}{N_{\text{job}}} \geq C_b, \qquad N_{\text{job}} \leq \frac{V_{\text{pool}}}{C_b}. \tag{8}$$

Here N_{job} counts jobs assigned to that reward period under the finalized-burn and fixed-lag rule. Serving costs can fall due before those rewards are realized. This is a necessary aggregate condition under proportional allocation. Heterogeneous costs and unequal routing can leave individual miners below break-even. Training, fixed capacity, mandatory evaluation, and required external reports add further costs. Quality rewards may support those costs, but an expected award is neither guaranteed reimbursement nor cash on hand.

Public A has a different payer: SlopNinja serves the artifact and pays the inference bill. Rewarding its author does not reimburse that bill. Any split between author credit and serving-cost credit must be settled before A calls enter a nonzero paid-use pool. The same job cannot receive two full units merely because its author and host differ.

9.3 Pricing sensitivity and a scale example

The planning costs in Section 8.2 are $C_{\text{model}} = \$0.0002875$–$\$0.000425$ for raw-text detection and $\$0.0082885$ for revision per 1,000 words. At $M_{\text{price}} = 25$, their prices are $\$0.0071875$–$\$0.010625$ and $\$0.2072125$. The $\$0.02$

detection target instead implies $M_{\text{price}} \approx 47$–$70$. For a chosen price ceiling $P_{\text{USD,max}}$, the general constraint is $M_{\text{price}} \leq P_{\text{USD,max}}/C_{\text{model}}$.

The comparison of raw-text detection with Pangram's dated $\$0.40$ bulk and $\$0.50$ realtime rates assumes neither equivalent accuracy nor equal terms. Subscription discounts, minimum charges, and word-block rounding still apply.

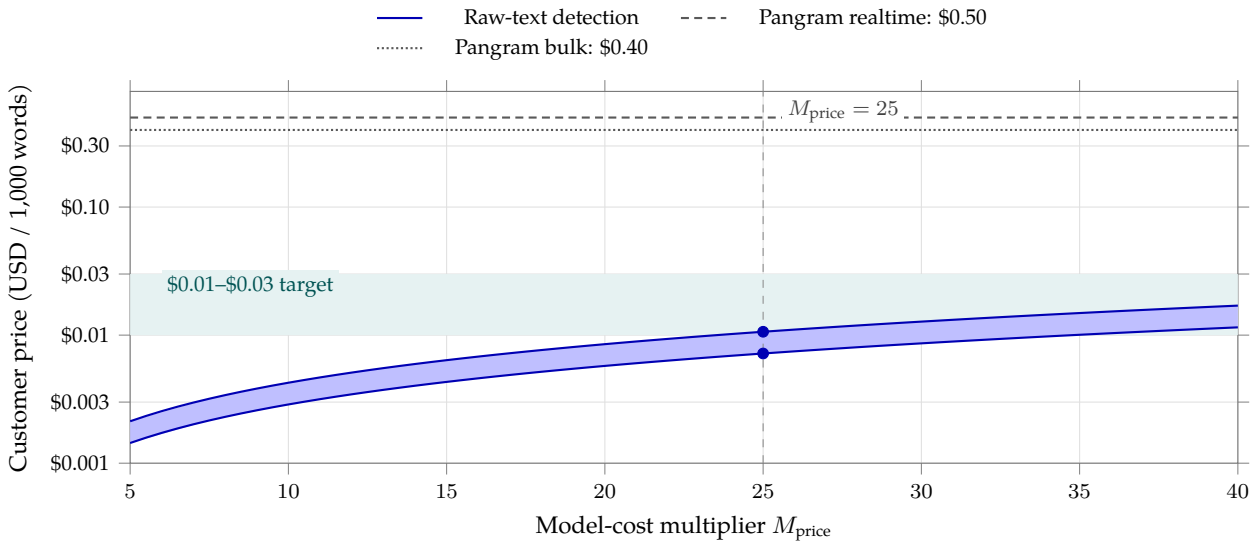


Figure 2: Price sensitivity per 1,000-word detection job. The band spans the raw-text cost of $\$0.0002875$–$\$0.000425$ from measured Container busy time, including the stated retry allowance. The shaded $\$0.01$–$\$0.03$ range is the proposed detection target; Pangram's dated bulk and realtime rates are price comparisons only. The price axis is logarithmic. No token-price assumption enters this calculation.

The revision scenario assigns $\$0.00271875$ per job to the miner's proposer calls. Operator model, compute, storage, and delivery costs are approximately $\$0.0079$–$\$0.0085$. Raw-text detection's corresponding operator subtotal is $\$0.0004$–$\$0.0009$. Table 5 scales these proxies without economies of scale. Burn principal is unavailable for either party's bills.

Revision cost is sensitive to its resolver. The batched 8B resolution budget costs $\$0.00093$ after the retry allowance; a 70B resolver using 20,000 input and 2,000 output tokens at the stated tariff would cost $\$0.0129575$ instead, raising revision model cost from $\$0.0082885$ to $\$0.020316$, about $\$0.5079$ at $M_{\text{price}} = 25$. For such workloads the escalation rate and workload bounds matter more than the multiplier.

Service	Jobs	Burn principal	Miner serving	Operator delivery
Detection	1,000	$7.19–$10.63	—	$0.39–$0.93
Detection	100,000	$719–$1,063	—	$39–$93
Revision	1,000	$207.21	$2.72	$7.90–$8.50
Revision	100,000	$20,721.25	$271.88	$790–$850

Table 5: Illustrative period totals at $M_{\text{price}} = 25$ for 1,000-word jobs, rounded for display. Serving costs use tariff proxies, the synthetic-text Container measurement, and assumed retry/runtime budgets, not production measurements. They exclude payment and chain fees, refund reserves, training, evaluation, external checks, and low-volume fixed costs. Public A has no miner serving charge in this architecture.

9.4 Capacity and operating reserves

If volume grows by a factor k while per-job cost stays unchanged, V_{pool} must also grow by at least k to preserve the same reward per job. The average log elasticity over that change must therefore be at least one. Purchase-and-burn establishes no law forcing this response: f, E_{miner}, and realizable p may each remain flat or fall. With a

fixed pool, doubling volume halves the reward per unit. Equation 8 then gives a budget-dependent serving limit, not unlimited scalability.

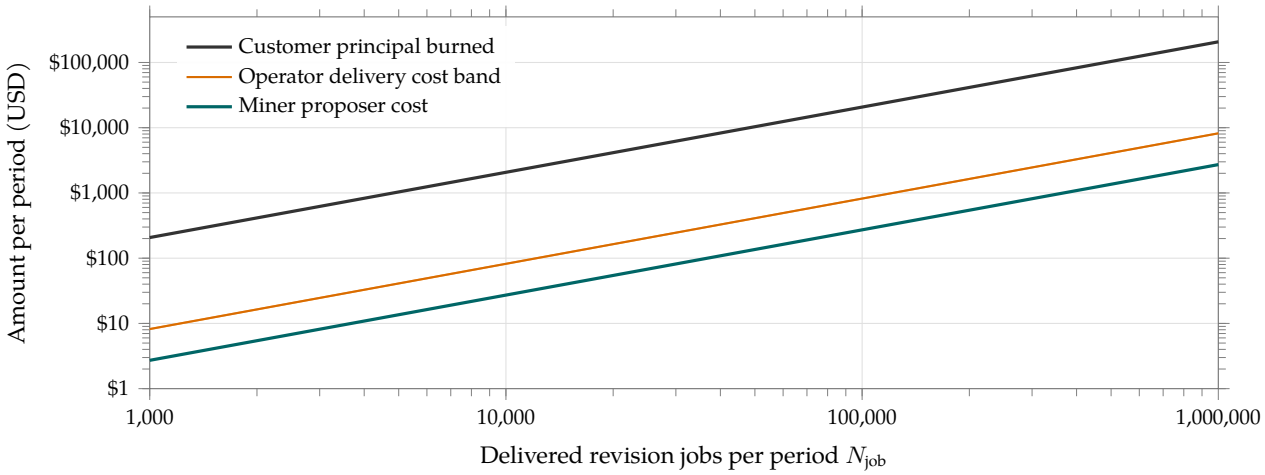


Figure 3: Revision totals versus delivered 1,000-word jobs in one period. Burn principal is $0.2072125 per job; miner proposer cost is $0.00271875; operator delivery costs span $0.0079–$0.0085. Both axes are logarithmic. The curves scale fixed planning assumptions and exclude the additional costs listed in Table 5. The burn curve is not a funding source, and no alpha price or emission trajectory is forecast.

Operator solvency needs its own constraint. Let T_{cash} be available treasury cash excluding customer custody, F_{cash} additional realized operating funding, O_{due} service and shared expenses due, L_{due} refunds and other liabilities due, and H_{reserve} the required reserve for remaining future obligations. Admission must preserve

$$T_{\text{cash}} + F_{\text{cash}} - O_{\text{due}} - L_{\text{due}} \geq H_{\text{reserve}}.$$

No term includes burned principal. Capital or explicitly allocated, realized owner proceeds can contribute to F_{cash}; future emissions cannot. Native owner entitlement follows the ownership or lease arrangement and does not itself entitle SlopNinja or an investor to those proceeds. Reserves must also match the currencies and timing of liabilities, so a USD-equivalent surplus alone does not establish that refunds can be paid.

9.5 Self-purchases and low-volume rewards

Burning gives artificial demand a cost, but a large pool divided among little legitimate work can still reward it. Let $U_{\text{honest}} > 0$ be existing legitimate work, u_{self} additional self-purchased work credited to an attacker, and k_s the attacker's incremental cost per such unit, including the full customer price and its own variable serving costs. For an attacker with no preexisting share, holding V_{pool} fixed gives the simplified gain

$$\Pi(u_{\text{self}}) = V_{\text{pool}} \frac{u_{\text{self}}}{U_{\text{honest}} + u_{\text{self}}} - k_s u_{\text{self}}. \tag{9}$$

This assumes the attacker can route the purchases to its own credited model. If it already earns $V_{\text{pool}} u_a / U_{\text{honest}}$ from legitimate work, measure incremental gain instead:

$$\Delta\Pi(u_{\text{self}}) = V_{\text{pool}} \left(\frac{u_a + u_{\text{self}}}{U_{\text{honest}} + u_{\text{self}}} - \frac{u_a}{U_{\text{honest}}} \right) - k_s u_{\text{self}}.$$

Existing rewards are not new attack proceeds. Both expressions omit caps, price feedback, quality-reward changes, and other controlled identities.

With no preexisting attacker credit, the initial marginal gain is positive when $V_{\text{pool}}/U_{\text{honest}} > k_s$. With no honest work, any positive sole credited purchase can capture the entire uncapped pool. A cap on work units or on one identity's share does not solve this after renormalization or identity splitting. Qualification needs an absolute cap on released rewards relative to attributable paid burn, applied across common control and the total pool, with

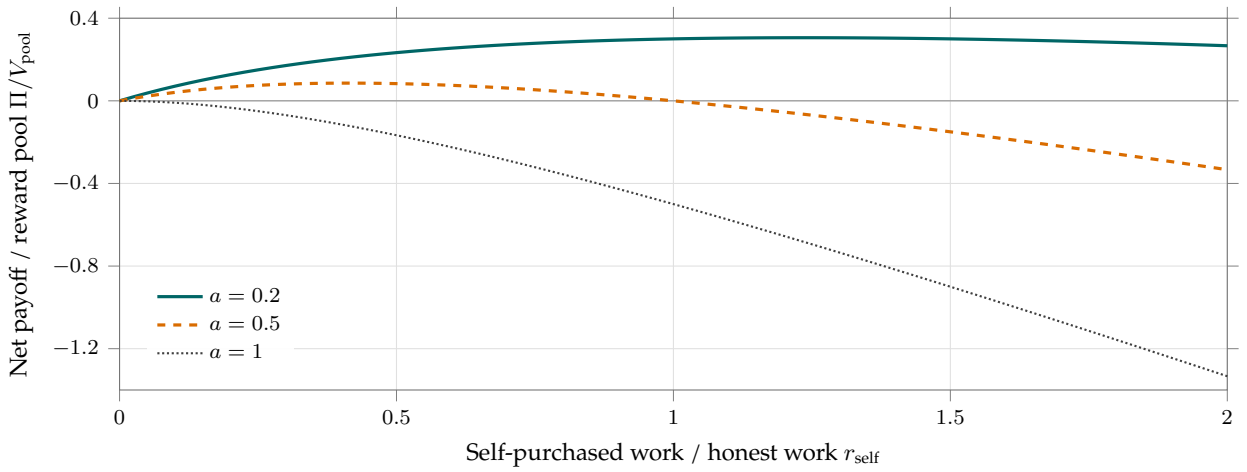


Figure 4: Simplified self-purchase incentive with fixed $V_{\text{pool}} > 0$, honest traffic $U_{\text{honest}} > 0$, no preexisting attacker credit, and no reward cap. Set $r_{\text{self}} = u_{\text{self}}/U_{\text{honest}}$ and $a = k_s U_{\text{honest}}/V_{\text{pool}}$; normalized gain is $\Pi/V_{\text{pool}} = r_{\text{self}}/(1 + r_{\text{self}}) - a r_{\text{self}}$. Curves vary the cost of buying work relative to the pool. They are sensitivity cases, not measured attack profitability.

an explicit destination for unused allocation. No numerical cap or implementation is selected here. Validator dividends, owner proceeds, and other rewards under common control must enter the combined attack budget; a miner-only calculation cannot establish resistance.

9.6 Calibration and activation decisions

Low demand makes reward concentration and fixed costs more important; high demand raises aggregate bills and can dilute rewards below serving cost. Calibration must test both regimes, including declining realizable token value, illiquid sales, resolver escalation in later *B*'s parsed workloads, retry spikes, provider repricing, delayed burns, and post-burn refunds. Use measured cost distributions rather than only averages, and treat free beta inference as a temporary subsidy.

Before enabling a paid-use pool, fix workload credit, author-versus-serving credit, f, release caps, zero-work pool treatment, and the native payout mapping. Choose M_{price} against measured quality, customer prices, and operating capacity; fund reserves before accepting liabilities. Update offers and future reward schedules through their published versioning rules. Rising burns or demand alone do not justify raising capacity beyond funded obligations and miners' willingness to serve.

10 Later transformation mechanism

B is a later mechanism for authorized revision toward a requested writer's voice, with preserved meaning and measured resistance to origin detectors. Its activation requires a separate scoring-authority decision, editorial validation, serving qualification, and reward allocation. The coordinator-led *A* pilot does not settle who adjudicates *B*'s semantic disputes. This section retains a candidate evaluation design; its formulas and service gates are not active launch rules.

10.1 The editing task

A benchmark supplies source text, independent authorized target-author samples, and a brief fixing the requested style and preservation requirements. Sources may have human-only, model-only, or mixed production histories. Revision preserves source ancestry while extending the candidate's production history: a model revision of human-only text is mixed under the initial labeling policy. Intended uses include editing one's own or licensed writing and authorized detector research; a low detector score cannot cancel disclosure obligations attached to the eventual use. The private job record binds the authenticated requester, source and reference

identities, declared permission to submit and transform them, and relevant license or permission records where applicable. These records document the authorization basis; a declaration alone cannot establish ownership.

Matching a writer includes lexical, grammatical, and rhetorical habits in the requested register. The brief distinguishes unwanted filler and repetition from deliberate informality and idiosyncrasy. It may request a different tone from the source. Without such a direction, source tone supplies the default. Shortening, simplification, and stronger rhetoric count as improvements only when they serve the brief without losing information or increasing certainty.

For the main intended customer workflow, the target is the private writer profile built from the user's samples (Section 3.1). The shared English CFG is the proposed pipeline's edit representation: it divides the source into word, phrase, and clause units, types each proposed edit, and reparses candidates so a revision can be compared with its source. *A*'s continuous author-fit signal against that profile ranks successive revisions; grammar-level writer-style features are a candidate further input to that guidance, qualified on their own evidence. The categorical *Provided writer* match alone is too coarse to guide this search. No representation tested so far supplies that signal: rewrites toward a target writer scored near chance against rewrites toward other writers for encoder, stylometric, CFG, and fused profiles alike, and the kind of rewrite moved scores more than the target voice did (Section 14.1). The feedback therefore waits for the learned matcher's qualification and for rewrites conditioned on several of a writer's own samples, data that does not exist yet. Freeze the reference profile, *A* artifact, and calibration for the job so an apparent improvement cannot come from changing the target. A language model may propose changes within the registered procedure; the private ranker receives compatible source and candidate bundles, deltas, the writer profile, and declared task controls. The target-style input is the profile rather than raw reference writing. Source prose remains available to the authorized editing runtime, and trusted preprocessing still reads the samples. Neither samples nor profiles enter a miner-accessible request path. Candidate selection retains separate meaning, readability, and tone checks; a higher fit score cannot erase a revision's production history. Whether the grammar's typed edits and this profile support useful transformation remains an empirical question. Private serving and the provider/operator trust boundary are specified in Section 7.

The assembled document is the evaluation unit. Later qualification must cover multi-paragraph work and, for profiles advertised to support them, complete manuscripts and books. Context windows and local edits are implementation units. Evaluation retains every span and checks cross-section claims, terminology, attribution, and voice. A chapter or sentence score cannot certify the complete work. Each length class needs measured resource limits, failure behavior, and whole-document quality evidence.

10.2 A frozen comparison

Before generation, the evaluation authority freezes the service version, workload profile, task population, references, style evaluator, detector panel, source baselines, and funded evidence budget. The benchmark uses the same bounded pipeline offered to customers. Its profile limits internal attempts, context, output, search, and cost. Each invocation produces one final candidate or abstention; all attempts and recovery actions remain recorded. Cloudflare generation may vary between runs, so qualification measures variability without claiming identical-output replay.

The proposed panel uses three qualified, distinct *A* inference artifacts: the prior-round origin leader and two preselected members. Before activation, choose whether the leader is ranked by binary AI-involvement Brier score or three-class origin Brier score. That choice is separate from *A*'s quality anchor and reward baseline. Freeze reserves before issue; after issue, recovery executes the same judges. Miners cannot nominate the panel, benchmark inputs, or a replacement comparison. Too few competitive artifacts require separately reported, funded bootstrap comparisons.

Run the complete source/candidate matrix under the frozen parsing, resolution, projection, and execution contracts. Retain the strongest detector's result separately. Miner self-scores remain nonauthoritative. Distinct artifacts and UIDs do not establish independent ownership; collusion and candidate-triggered errors belong in qualification tests. *A* earns quality credit against independent production-history labels, so judging *B* harshly supplies no reward by itself. Hidden labels and unreleased review material remain with authorized evaluators; retired examples need authorization before release.

Accepted bytes. The English benchmark profile specifies UTF-8, NFC, LF-only endings, permitted scripts and code points, and pinned Unicode/security-data versions with test vectors [47,48]. It forbids a terminal LF, BOM, undeclared invisible or bidirectional controls, tabs, edge whitespace, repeated spaces, and excess blank lines. Predeclared literal spans receive only exact-copy whitespace exceptions; they cannot add a terminal LF. Sources, references, and briefs conform before issue. Other formatting requires an explicitly qualified profile.

The runtime may finalize an uncommitted draft inside its measured procedure. Signed text is never silently repaired. Receipts, commitments, reconstructed vectors, style evaluation, reviewer display, and authorized Pangram requests bind identical accepted bytes. Decode the provider's returned analyzed text and compare its UTF-8 bytes exactly; a changed copy is unusable evidence. The probe behind the terminal-LF restriction and its limitations are recorded in Section 12.2. Qualify supported report retrieval and retention before live use. Report URLs may expose text and stay out of public receipts and logs. Customer disclosure to Pangram requires separate permission.

10.3 Style, preservation, and detector improvement

The public style evaluator is frozen independently of the private ranker. Its contract binds the profile schema, feature meanings, reference pooling, parameters, executable, numerical operations, rounding, and resource bounds. For text t and reference set Refs, it returns a nonnegative integer distance $d_\theta(t, \text{Refs})$. Under the proposed personal-profile workflow, Refs binds the writer profile and its construction record; evaluating a candidate requires no raw reference prose. A qualified A-derived author-fit distance is the intended implementation. Its mapping to this integer distance and its calibration must be fixed and tested before use; a categorical match or an uncalibrated similarity cannot replace the evaluator. A nonempty calibration multiset $(c_1, \ldots, c_M)$ contains distances from development queries to independent same-author reference work; source families are separated from fitting and evaluation. Duplicate values remain. With $Q = 1{,}000{,}000$, define

$$q(t, \text{Refs}) = \left\lfloor Q \, \frac{2\#\{m : c_m > d_\theta(t, \text{Refs})\} + \#\{m : c_m = d_\theta(t, \text{Refs})\}}{2M} \right\rfloor. \tag{10}$$

Ties receive half credit. This rank supplies neither authorship probability nor semantic approval. Source x and candidate z share references and calibration, giving q_x, q_z. Positive style improvement uses remaining headroom:

$$V(x, z) = \begin{cases} 0, & q_x = Q, \\ \dfrac{\max(0, q_z - q_x)}{Q - q_x}, & q_x < Q. \end{cases} \tag{11}$$

Keep exact rational values through aggregation and check raw nonregression $q_z \geq q_x$ separately. A tie or already-maximal source earns no invented improvement. Before rewards depend on this evaluator, test adaptive searches that optimize it through reference copying, topic substitution, omissions, padding, or changes of meaning. Among semantically acceptable candidates, higher rank must also agree with independent judgments of author fit.

Readers inspect the complete source and candidate before seeing numerical scores. The frozen rubric has three fields:

- P, preservation of complete meaning: claims, entities, quantities, negation, uncertainty, citations, attribution, and argument relations. Review covers omissions from the checklist as well as listed facts. Unsupported additions and stronger factual certainty fail.
- R, readability without material regression, including requested cleanup of filler or formulaic phrasing. Brevity alone earns no approval.
- T, the requested tone and audience constraints, preserving deliberate voice features. This field still applies when source tone supplies the default.

Every field records pass, fail, or unresolved with reasons. Set $G = 1$ only after all required approvals; an established failure gives $G = 0$. Missing evidence leaves G unknown. An automated judge needs validation against separate readers before influencing rewards. Controls include unchanged sources, qualified human edits, fluent semantic errors, erased voice, and detector-passing revisions that retain unwanted habits. Instructions inside evaluated prose are data. Blinded reader studies must have enough power for the claims made about voice and quality.

Pangram's reported AI-plus-assisted fraction $F_r(t)$ differs from a document-level probability of AI involvement [49]. The proposal takes three Pangram observations, $r = 1, 2, 3$, of each source and of each candidate, which gives

$$\Delta_P(x, z) = \min_r F_r(x) - \max_r F_r(z). \tag{12}$$

Reopening one report does not create another observation. The coordinator binds report counts, request identities, repetition/retry rules, provider version, and usable evidence before generation. All required observations and failures remain in the ledger; neither miner nor operator may select favorable reports after seeing scores. Retry and provider-drift rules need qualification before activation. The operator funds shared source observations; miners fund their bounded candidate observations and evaluation inference as participation costs. Section 12.2 reports the limited repeatability evidence.

For panel member j, let $H_j(t)$ be its frozen AI-involvement probability and $\Delta_j = H_j(x) - H_j(z)$. Define $\Delta_S = \text{median}_{j=1}^{3} \Delta_j$, retaining every raw answer. The provisional joint detector improvement is

$$E(x, z) = \begin{cases} (\Delta_P + \Delta_S)/2, & \Delta_P \geq 0 \text{ and } \Delta_S \geq 0, \\ 0, & \text{otherwise.} \end{cases} \tag{13}$$

With complete evidence, the candidate utility is

$$U_B(x, z) = \begin{cases} G(V + E)/2, & q_z \geq q_x,\ \Delta_P \geq 0,\ \Delta_S \geq 0, \\ 0, & \text{otherwise.} \end{cases} \tag{14}$$

These weights remain a proposal requiring reader validation and an allocation decision. Each comparison measures both objectives. One component can improve while others hold steady; a clipped value cannot hide regression.

Strict detector success additionally requires every candidate Pangram fraction below 10%, a majority of panel members below their independently calibrated human-false-positive thresholds, and a pass at the prior-round leader's own threshold. It also requires the approvals and raw nonregression checks above. Incremental utility can be positive before this target is reached. Reports separate utility, author fit, each detector result, and strict success. A zero source Pangram baseline permits no candidate increase, although approved style improvement can still earn utility. Each candidate's label follows its complete production workflow, regardless of detector scores.

For detection customers, a successful evasion is a false negative by the affected *A* detector. Its reported limitations must include performance on held-out revisions from evaluated *B* pipelines. Such revisions can expose weaknesses for future training, but only fresh evaluation can establish an improvement in detection.

Reducing required Pangram checks later needs repeated held-out noninferiority studies, including new adversarial revisions and difficult human slices. Calibrate the fraction and probability outputs to the same independently labelled event on separate development data. Agreement with Pangram or a change of quality anchor is insufficient. A reduced-check policy must state how unchecked candidates are scored and how provider drift is detected.

10.4 Service and outcome accounting

Once *B* is active, the adversarial loop of Section 1 is to run as a mandatory training service. An *A* miner, the requester, submits authorized training text through the coordinator, and an assigned *B* miner, the provider, must return revisions of it, which the requester receives as training examples for its detector; each side's emission eligibility then depends on its service record below. The two roles form separate *A*-requester and *B*-provider service classes. No such duties apply to the *A*-only launch. Before issue, reserve shared evaluation resources, candidate-report funding, and the provider's accepted inference budget. The requester pays no extra service price; the provider absorbs its participation costs for potential emissions. Publication and qualification of *A* remain separate.

A durable ticket binds registration generations, service versions, task and capacity limits, ordered counterparties, and feasible response windows. Assignments consume permanent slots; callers cannot nominate peers, redraw

by declining, or turn retries into new quota. A valid admitted request activates provider responsibility. Fallback success preserves earlier failure. Unresolved validity stops dependent attempts. Keys, UID reuse, migrations, and restarts cannot silently reset counters or remove outstanding work. Coordinator outages preserve the cohort and evidence under the recovery protocol; elapsed wall time alone cannot erase an obligation.

For each applicable service class, retain $O_{\text{act}} = S + F + U + O_{\text{void}}$: successes, attributable failures, unresolved cases, and scoped voids. Unused reservations stay separate. The proposed availability rule requires $U = 0$, $N = S + F > 0$, and $10F \leq N$, plus qualification and a cleared deficit $D_e = \max(0, D_{e-1} + 10F - N)$. Recovery work belongs to the failed class; customer volume, voids, and unresolved work cannot repay that deficit. A compliant abstention can satisfy delivery while earning no quality improvement. No inactive class needs an artificial empty success.

Benchmark closure distinguishes missing or unscorable candidates, unresolved judgments, and scoped platform failures. Under the proposed accounting rule, these slots earn zero and keep their assigned denominator weight and actual status. One uncertain candidate cannot erase completed competitors. Shared treatment requires evidence of a common cause and applies uniformly to its declared scope. Semantic uncertainty alone does not prove an assigned-service failure. Provider revocation, deletion, exhausted billing, or unreserved concurrent use can establish provider fault only through the adopted attribution procedure.

Before *B* activates, choose its review authority, evidence access, incident deadlines, appeals, and correction rules. Other validators' approval or availability cannot gate review, closure, or service; their only protocol dependency is timely submission of the prescribed native weight rows. Reserve capacity for complete review and qualify these rules against outages and strategic withholding. Preserve signed decisions and attempt records through recovery; coordinator records cannot claw back native emissions already distributed.

11 Later mechanism C: grammar contributions for B

C would reward improvements to the shared English CFG and lexicon that later *B* uses for edit units and revision guidance (Section 10); *A* does not use the grammar. *C* starts inactive and activates by its own decision, which may precede *B*'s so the grammar grows before *B* uses it. Its objective is correct, reusable analysis of English as used, including informal constructions, unusual vocabulary, spelling errors, and embedded technical or foreign material. It must not equate prescriptive correctness with parseability or reward broad catch-all rules merely for accepting more text.

A candidate round accepts bounded public patches against one fixed parent profile. Contributions may add lexical forms and parts of speech, structural productions, or a more accurate analysis of text already covered. Examples must be authorized for that use. The coordinator reviews the linguistic claims, measures transfer to held-out documents, replays affected earlier analyses, and checks interactions among accepted proposals. A changed old parse deserves inspection; change alone is not a regression. Whole-document coverage, quality of the interpretation, remaining ambiguity, and resource cost are separate measurements.

The central coverage statistic is the fraction of requested documents with complete accepted analyses. A parser accepting every token is weaker evidence than an analysis whose structural decisions have been checked. Report both, retain failures in the requested population, and track initial coverage on new documents before any repair. Freeze resource limits so a gain cannot come solely from lifting a ceiling. Rule count, added bytes, dictionary size, and grammar size are not reward scores.

Accepted changes enter a subsequent shared grammar profile for *B*. *A* reads raw text and does not depend on it; a *B* comparison continues with its frozen grammar, projection, and selected derivations. Compact cached parses and production dependencies reduce repeated work, but new productions can affect documents outside the old dependency set. The acceptance protocol therefore needs both targeted replay and broader change detection. Changes in feature meaning require requalification; retaining a profile name cannot hide a new representation.

Each accepted contribution earns credit once. A durable contribution identity and credited recipient prevent a later release, continued use, or resubmission from earning a second award. Before activation, define overlapping-proposal credit, interaction failures, regressions, award size, settlement epoch, and when a credited contribution stops influencing native weights. Repeatedly leaving the same credited contribution in a live reward row would

defeat the one-time rule. A scored pilot without payouts can test those decisions before assigning a nonzero c in Equation 3.

The project already develops its grammar as research infrastructure. That work does not establish a live *C* protocol, and a *C* award would not itself establish improved transformation. *B*'s frozen comparison must test whether a grammar change improves edit quality enough to justify its parsing and resolution cost.

12 External comparisons and evidence renewal

Pangram begins as the external origin-quality anchor: the designated comparator for claims of competitive detection quality and the coordinator's assessment of readiness for paid service. It supplies a reference for those decisions; commercial activation still follows the workload-specific criteria in Section 15. The coordinator compares qualified *A* artifacts, the incumbent anchor, and Pangram on the same hidden, independently labeled population. Shared provider observations are funded once per comparison set; admitting another artifact need not multiply that bill. Freeze provider/model versions, repetition policy, population weights, operating points, and uncertainty analysis before exposing the set. Rotate private sources, track repeated use, and release material only after retirement and source authorization.

Pangram reports separate fractions classified as AI-generated, AI-assisted, and human, with segment confidence reported separately [49,50]. Its current dashboard explanation defines the fractions by character count; our share target uses final-word counts in recorded model-produced spans [51]. Both are distinct from *A*'s probability of any document-level involvement. Quantitative share comparisons need common span labels and counting units; the raw fractions are not interchangeable. Compare both against documented histories at independently calibrated operating points. Provider agreement supplies neither ground-truth labels nor reward credit. The reference model that serves as the reward baseline, the external quality anchor, and the later *B* panel leader have different roles. Replacing an anchor cannot change the reference in force or rescore an issued cohort.

12.1 Changing the quality anchor

A contender must beat the incumbent at declared human false-positive operating points, exceed a predeclared practical margin, and satisfy slice limits. Account for selection among multiple contenders, then require confirmation on fresh material after selection. Ties, missing observations, and inconclusive evidence leave the incumbent in place. Publish a signed version-bound decision and apply it only to a future cohort. Continue measuring Pangram after a subnet artifact becomes the anchor. The pilot's authority is the coordinator; no independent validator-execution quorum is implied. The October 30, 2026 sealed Test (Section 14) found the reference R_0 noninferior to Pangram 4.1 but not superior on balanced Brier, and its pre-registered recipe verdict failed on one slice. Under this rule Pangram remains the anchor.

Report source-family dependence and uncertainty, including weak historical-human proxies. A small comparison cannot establish a rare false-positive guarantee. Report coverage and failures alongside conditional accuracy. Outages preserve the comparison population and evidence; they do not justify silently closing with a smaller favorable sample. A later decision to reduce *B*'s external checks needs its own repeated held-out noninferiority study. An *A* anchor change alone cannot justify it.

12.2 Exact report identity and permitted disclosure

Bind every observation to the exact accepted text, provider version, settings, time, result identifier, and retained response bytes. Retrieval must use a permitted provider interface and verify that the displayed report corresponds to the submitted text. A working public report route in a past probe does not establish a permanent API or permission to automate an undocumented endpoint. The public-result probe found that the API and report removed a terminal LF; this motivates the later *B* text profile's restriction without establishing a general preprocessing contract. The report exposed the analyzed text through an undocumented public endpoint, with no provider signature or immutable model identity established. Report URLs therefore stay out of public receipts and logs [52].

Repeatability probes found consistent document fractions for three texts, with auxiliary variation and no observation near the strict 10% boundary. They establish neither general determinism nor threshold stability [53].

A customer-requested external check is a separate funded operation requiring disclosure authorization. It is not silently included in private inference or sent to a third-party provider as a default audit. Required later B benchmark reports use bounded prepaid miner budgets and an operator-controlled selection policy, as described in Section 10. One report retrieved repeatedly is still one observation. Missing or mismatched reports remain evidence gaps rather than successful detector passes.

13 Trust assumptions and adversarial behavior

The pilot's scoring authority is the SlopNinja coordinator, the operator's lead validator implemented entirely as a serverless service. Held-out inputs, labels, individual predictions, and exact scores stay in that service, except that its generation requests send held-out passages to hosted model providers under their own data policies (Section 5.2), an exposure the pilot accepts; miners, validators, and the public receive none of them. The coordinator derives the native weight row from quantized scores under frozen rules and signs it; every other validator verifies that signed row and commits it through native commit–reveal with its own key and journal. The coordinator later certifies its finalized row for audit; other validators do not wait for that certificate before committing. Anyone can recompute the row from the public quantized result, which checks the reward arithmetic without reproducing hidden scoring or establishing the truth of its labels; validators trust the coordinator's row as they trust its measurements. A dishonest coordinator could fabricate results, omit a qualified entrant, leak a benchmark, or favor an owner. Models published when scoring starts (removable only by a recorded legal removal, Section 3.4), committed populations, retained evidence, and signed corrections help detect some misconduct, but the pilot assumes honest scoring and curation. There is no validator-quorum security guarantee for those claims. The coordinator's validator hotkey and durable signing state are also within this serverless trust boundary. Other validators retain independent keys and recovery records.

Table 6: Pilot risks, controls, and remaining trust assumptions.

Risk	Control and remaining limit
Benchmark leakage or biased labels	Review provenance and overlaps, separate source families, restrict live labels, draw a fresh batch for every cohort, and score each passage of fresh stock as a pure-human negative at most once. Commit the population and policy before scoring. Release only quantized scores, drawn with seeded noise under a per-cohort privacy budget, and count every released value, the unnoised gate, public-copy, cluster, and promotion outcomes included, in a versioned bit budget. Readings shown while scoring runs have their own versioned budget, use disjoint segments of the batch fixed at the seal, and count in the same bit budget; showing a released score before the close changes neither budget. A planned gate feedback record gives failing entrants coarse gate classes, also counted. Released values still leak a bounded amount about the batch they describe, the budget adds across cohorts that reuse a passage, and coordinated probing across many UIDs remains possible. The public challenge counts carry an uncounted timing channel, to be closed before mainnet (Section 5.7). Secrecy and accurate curation remain assumptions.
Memorized examples or hidden triggers	Qualify on held-out production workflows, writers, and generators, with targeted-trigger tests. Entrant weights published when scoring starts permit inspection but do not prove the absence of a trigger.

Risk	Control and remaining limit
Recognition and lookup of human text	The human side is existing text, and a mixed document keeps its source's unreplaced words, so recognizing human text can separate origins without detecting AI involvement. Admission allows only the allow-listed graphs, of at most 160 million parameters, or 173.5 million in the span tower class, with the allow-listed tokenizer and schema-validated files, and excludes an explicit text index. On mainnet, sampling breadth over Common Crawl, keyed secret passage choice, and share caps hold the estimated gain to about 0.5% of headroom, or about 1.1% for an attacker who completes a capped stratum; pretraining exposure is untested, and a low-powered lookup canary audits promotions. These reduce lookup; none prevents it. On testnet the public stock is recognizable in full, and cohorts without a working outside entrant redraw documents already exposed, an accepted pilot limit (Sections 5.3 and 5.4). It reaches mainnet's scoring only through the reference testnet hands on, which mainnet scores on its own batches: a testnet lookup gain does not carry into mainnet's first bar, which can therefore sit lower than the testnet record suggests (Section 5.6). It also reaches the revenue pot's distribution, which credits testnet promotions (the row on the pot's credit below).
Copied models and common ownership	Keep artifacts private until their cohort's scoring starts. Detected copies of earlier published artifacts earn nothing: exact copies fail by content digest, and near copies, recalibrated ones included, fail by aligned distance on the batch and on a private probe (Section 5.10). A copy perturbed with deliberate noise still escapes, at a cost of about 3.5×10^{-4} to 4.8×10^{-4} of expected score, about a tenth of an unrounded band at 1,000 cases, and can earn a share of its band in a fallback cohort, or in a margin cohort when its own release noise lifts it to the margin cutoff, as it does for an entrant equal to R_k about 35% of the time at testnet's full roster; promotion tests the margin on exact scores, so release noise cannot promote it. Because a band's share ignores how many clusters it holds, a copy degraded further, or an entrant's weaker variant of its own model, can land alone in a lower band and take up to 45.9% of the pool, more than each cluster in a crowded band above it; its expected credit is not yet measured (Section 5.9). Weight soups of published artifacts are new models and earn normally. Within a cohort, near-duplicate entrants share one credit equally, and a cluster is placed by its earliest-admitted member's score, so extra near copies add no noise draw toward the cutoff, though a party's portion of a shared cluster still grows with the hotkeys it registers; under `reward-v2`, which testnet's current record runs, a cluster is still placed by its highest-ranked member. Copies farther apart than the near-copy distance draw independently. Registration cost and UID slots are the only brakes on these; different hashes and UIDs do not establish independent work or ownership.

Risk	Control and remaining limit
False reference advance	Eligibility and promotion use the same margin of about two paired standard errors of seed retrains. When ten entrants equal to the reference differ from it as seed retrains do, one passes the margin test in about 14% of cohorts, and, since its released score must also reach the margin cutoff, about 6% of cohorts promote; when they differ from it as much as the other measured recipes do, most cohorts promote, and each such entrant passes the margin test alone about a third of the time (Sections 5.6 and 5.9). The same noise can promote a worse model: alone, a structurally different entrant a full margin below the reference clears it about a fifth of the time. Fresh re-measurement each cohort, the easier pass against a worse reference, and the displaced model's role as fallback anchor limit the drift, but no rule bounds it; it is measured before launch. Two lookup-canary flags, or a failed strict false-positive tier, pass the promotion to the next qualifying retained cluster head (Section 5.6). An operator reference decision, the one change outside that rule, applies only to later cohorts, so it moves no earned credit; while the launch burn lasts, it can raise or lower the bar that ends it.
Operator escape paths	Abandonment, refusal to pay, restarts, reward-rule changes, and reference decisions are journaled, publish signed records with their reasons, and, when they decline a result, change reward inputs, or restart the record, need an offline operator-decision key (Section 6.6). They never reassign paid rewards or skip a cohort by time, but an operator can still use them to withhold pay from a finished cohort or to reset the baseline; the records make such choices visible, not impossible. A cohort abandoned after its scoring started leaves its published entrants outside the published-artifact index, so copies of them can earn later until a rule for them is settled before mainnet.
Prompt injection or malicious model data	Admit bounded data-driven models, validate formats, and deny executable plugins and generated tool calls. Untrusted documents cannot modify scoring rules or authorize disclosure.
Resource exhaustion	Limit input profiles, parsing/resolution work, memory, candidate counts, and retries. Preserve the full assigned population when reporting failures; dropping expensive examples can inflate apparent quality.
Miner access to customer content	Centrally execute A; for later B, use the qualified direct Cloudflare path without a miner endpoint or gateway selection. Provider and operator remain trusted with plaintext.
Public access to private B weights	Keep credentials and assets off public paths, and test unauthorized and unrelated-account access. Purchased outputs can still support imitation or extraction.
Human misidentification, unwanted identity linkage, or impersonation	Keep personal samples and writer profiles private; record provenance and permitted uses for public or licensed gallery references. Bind revision jobs to their authorization records. A profile match indicates resemblance, not authenticated identity or authorship, and these controls cannot prevent every misuse.
Account revocation or model replacement	Register versions, check identities and digests, and suspend drift or lost access. A response's adapter ID does not prove immutable historical weights or guaranteed capacity.

Risk	Control and remaining limit
Fabricated service or self-purchased traffic	Require linked delivery, burn, and attempt records; fix work-credit attribution and caps before activating paid-use emissions. Operator receipts remain operator assertions.
Unearned or gamed credit in the revenue pot's distribution	Only models promoted to the reference are credited, under an attribution rule and serving checks published before they are used, and every payout, credit, purchase, burn, and claim is listed with its transaction (Section 8.1). A credited model inherits testnet promotion's limits: its promotion can rest partly on lookup or on release noise, so credit can reach a model no better than the one it displaced. If the attribution rule lets the credited miners share the whole miners' half, a miner credited alone, even briefly, receives all of it; under the other division a miner receives only the fraction of the half that its credited revenue forms of all testnet-era revenue. A miner whose model is credited could also buy the products itself, or draw invalid ad traffic, to raise its credited revenue; ad traffic is watched and ads are switched off when invalid activity appears, and whether the attribution rule makes buying from oneself unprofitable is settled when that rule is published.
Selective failure or retries	Keep all assigned outcomes and a frozen retry rule. Separate model failures from infrastructure incidents. A later success cannot erase an earlier recorded failure or select the best repeated result.
Coordinator outage, censorship, or equivocation	Retain signed checkpoints and evidence, fence stale generations, and resume the same cohort after reconciliation. Validators keep committing the last signed row, since no reward-intent authorization expires; only a signed revocation withdraws it. This preserves recoverable work but cannot force a withholding operator to resume or publish a truthful result.
Lost or forged block notifications	Authenticate webhook ingress, persist reconciliation work, and verify finalized history through RPC. Deduplicate by block identity and backfill gaps through RPC; a one-minute watchdog polls public RPC when deliveries stall, including after the provider pauses a subscription, which an operator must resume. A webhook cannot authorize signing or establish finality; RPC availability and chain validation remain dependencies.
Validators withhold or delay weight submission	Cohorts, service, updates, and recovery continue without their cooperation. Stale or missing rows can still distort native consensus and emissions. Record chain delivery separately; preserved scores do not guarantee later payment.
Forged row record, authorization, or certificate, or a stale row	Verify the row record, its authorization, and the revocation set under pinned keys, and that the row names the held or a later target, before committing. Audit the coordinator's later certificate against its authority and finalized row. A mismatch found by another validator or by recomputation never gates local work; only a signed revocation or a newer signed row changes what it commits.
Compromised release or interrupted migration	Verify signed release metadata and artifacts, fence old processes, and resume ordered migrations from durable checkpoints. The release authority remains trusted; a signature does not prove safe code. Preserve journals and reconcile before new signing.

Risk	Control and remaining limit
Colluding miners and validators	Test common-owner reward paths, duplicate entries, selective failures, and native consensus effects. The signed row is published off chain before any reveal; commit–reveal cannot prevent off-chain copying or force validators to submit the prescribed row.

Hidden labels describe production history under the declared policy. Neither an origin probability nor an attribution match establishes that history for a new document. Reference copying, topic overlap, and editing workflow can all produce apparent attribution quality without stable author identification. Independent reference/query works, withheld-identity cases, and source-family holdouts are therefore part of evaluation. Released benchmark examples retire from hidden testing; reuse permissions do not restore their secrecy.

Published A artifacts let an adversary optimize against the detector. Future B development deliberately uses that feedback, while preserving separate meaning, readability, and authorization requirements. A low score cannot certify human authorship, faithful revision, or compliance with a publisher's rules. External detectors are comparisons with their own version and measurement limits. Changing a panel, source population, or rubric after seeing candidate outputs invalidates a shared comparison.

Native consensus determines actual emissions from validator rows and chain state. Application records cannot force an atomic switch across validators, freeze native emissions during a coordinator pause, or reclaim rewards already distributed. Registration and hotkey identity must survive UID reuse in the off-chain ledger, so old credit never transfers to a new occupant. Deployment tests must cover stale weights, commit and reveal delays, finality, cohorts with no new target, zero-work usage pools, and common-owner miner/validator incentives. Timely submission of the coordinator's row by other validators influences the native reward outcome. Their availability, acknowledgments, and software versions impose no application progress gate. Unallocated credit and the share of a departed hotkey burn through the owner hotkey's UID, and the chain shrinks the subnet's TAO-side emission share by the burned fraction of miner incentive (Section 5.12).

Native commit windows and reveal obligations use the chain's clock, which continues through coordinator silence. Timelock auto-reveal cannot wait for a coordinator certificate or application acknowledgment. A missed commitment window moves eligible new delivery to a later valid window without deleting the scored cohort; existing obligations retain their original recovery rules and deadlines.

Durable recovery also has limits. Persisted records require capacity planning, backups, and tested restoration. A missing result stays unresolved; a timeout does not establish failed execution or justify replacing the cohort. Chain runtime upgrades never invalidate a cohort; an operation that a new runtime leaves without a type-correct encoding waits alone until live metadata admits it. Operational upgrades may preserve the frozen evaluation semantics. New scoring semantics require a future comparison or an explicit correction decision that accounts for weights already submitted.

SlopNinja signatures attribute statements to the operator; they are not Cloudflare signatures, proofs of model execution, or confidentiality evidence. Customer content protection relies on the qualified serving configuration and the provider and operator's conduct. Those dependencies, together with honest curation and coordinator-led scoring, define the pilot's trust boundary.

14 Existing evidence and implementation status

These dated checkpoints describe different experiments, each with its own Test population, identified below by date and size; Brier values from different populations are not directly comparable. The A baseline release candidate is a package whose document model is the first reference R_0, with a span head added later. R_0 was noninferior to Pangram 4.1 on a sealed, pre-registered Test, but the pre-registered verdict failed for its three-seed recipe. No result below qualifies a live subnet. Most source reports and some underlying inputs are access-restricted during subnet preparation, so the results cannot be independently reproduced from this paper alone. Earlier research checkpoints, which gate nothing at launch, appear in Appendix A.

14.1 Current A baseline

Release candidate, October 28, 2026. The document model R_0 is a ModernBERT-base encoder fully fine-tuned from V6, the earlier raw-text detector of Appendix A, with a style adversary and a generator-contrastive head. It has origin, share, and conditional attribution heads, and its weights are seed 17 of the selected recipe. Train holds 86,575 rows over 36 domains. Its positives come mostly from local open-weight generations, with hosted frontier generations, RAID paraphrase cohorts, and earlier positives. A single read of a selection split chose the recipe: its three-seed mean balanced involvement Brier was 0.0619 against 0.1820 for V6, uncalibrated. Calibration was fitted on 5,379 Calibration (CAL) documents only, as an origin temperature followed by a clipped isotonic map; the 1% human false-positive operating point is a calibrated involvement of at least 0.9091. The Rust FP64 replay matches the reference outputs within 7.3×10^{-7}. A mixed-int8 build failed its pre-registered no-quality-loss qualification, so serving uses f32 [54].

Span head, October 31, 2026. A pre-registered span protocol fitted four arms from the release lineage, with three seeds each: a token head on the frozen R_0 encoder, trained with and without spliced partial rewrites, and a joint fine-tune of encoder, document heads, and token head, with and without them [55]. The shipped configuration is R_0 plus the token head trained with the spliced rewrites on the frozen seed-17 fit. Its document outputs are bit-identical to R_0's on 227 replay requests, and its Rust FP64 replay matches the reference within 6.1×10^{-6} in token probability, against a 10^{-5} bound, with every span offset equal. After the share tilt of Section 3.2, the mean word probability equals the emitted document share within 6×10^{-16} in every evaluated document. On the 823 held-out spliced rewrites of the report half, scored once after the decoder and calibration maps were frozen, the calibrated head reaches word Brier 0.128 and character F1 0.410, with precision 0.862 and recall 0.402; span-level F1 is 0.245 and mean span IoU 0.306, and it decodes no span in 346 documents. Span metrics here are per-document means: F1 and recall count a document without a decoded span as zero, and precision covers only the 477 documents with one, so F1 is not the harmonic mean of the reported precision and recall. Character F1 is about 0.85 when most of a document is rewritten, 0.60 for a paragraph, 0.17 for one sentence, and 0.05 for a clause. These documents edit development humans that bounded the decoder's human limit, so the figures are somewhat optimistic. Retrospectively, on the consumed Test, character F1 is 0.478 on mixed documents and 0.021 on base-model continuations, and 156 of 6,132 humans carry a decoded span. The head adds about 0.14 seconds to a 2,048-token request, against about 27 seconds for the encoder on one CPU thread. The protocol's selected arm, the joint fine-tune with spliced rewrites, reached character F1 0.560, but its uncalibrated human-only Brier rose from 0.061 to 0.093, and calibrated it flagged 6 Test humans at the 0.1% operating point, where R_0 flags none, so it was not adopted.

Adopted span head S2c, November 1, 2026. A pre-registered follow-up fitted three arms from the same lineage and data, with three seeds each, to keep the joint fine-tune's span quality without its human-document shift [55]. Two joint arms failed the human gates. The selected arm, S2c, freezes the encoder and document heads and feeds the token head through a trained copy of encoder layers 18–21 and the final norm, about 20 million parameters. Its document outputs are byte-identical to R_0's at every seed. Its primary span metric, a tilted word Brier on 2,011 development mixed documents, is 0.1295 over three seeds, against 0.1412 for the shipped recipe and 0.1205 for the joint fine-tune: it beats the shipped recipe, by 0.0118 $[0.0104, 0.0131]$, but not the joint fine-tune, so the pre-registered outcome is partial. Before the tilt its word Brier, 0.109, equals the joint fine-tune's; tilting to R_0's document share costs it 0.020, against 0.012 for the joint fine-tune, which retrained its share head. After the tilt, mean word probability equals document share within 6×10^{-16} for every arm. No point of the original decoder grid met the 22-human limit; its declared extension chose a decoder that puts spans on 21 of 570 development humans at seed 17, and on 22 and 27 at seeds 29 and 41. On the 823 spliced rewrites of the report half, already scored for the other heads and evaluated once after the decoder and maps were frozen, seed 17 without span calibration maps reaches character F1 0.519, recall 0.532, and span F1 0.485, against 0.410, 0.401, and 0.241 for the shipped head on that basis, and decodes no span in 288 documents. Retrospectively, on the consumed Test, its mixed-document character F1 is 0.535, against 0.478 for the shipped head, and 210 of 6,132 humans carry a span, against 156, so it does not hold the shipped head's human rate there. Its Rust FP64 replay matches the reference within 4.4×10^{-6} in token probability with every offset equal, against the same 10^{-5} bound. The tower adds 18–20% of encoder CPU time, 5.3 seconds at 2,048 tokens on one thread. Its development human gates are optimistic, because the baseline's early stopping and the earlier decoder limit used those humans. SlopNinja adopted S2c on November 1, 2026 as the span head of its first span package. A calibrated S2c package with this

10-word decoder was exported, replayed, and served by the live scorer in operator probes from November 2, its predictions, sentences, and spans equal to the local run.

First span package under the 24-word floor, November 7 to 9, 2026. The span figures above count spans of every length. Spans under 24 words are no longer scored or decoded (Section 3.2), so the figures below, all under the floor, are not comparable with them. Under a floor evaluation whose protocol and decoder selection were frozen before any held-out output was read [56], S2c with a decoder whose shortest run is 24 words matched or slightly exceeded its 10-word decoder's character F1 on development documents, by $+0.003$, and on the consumed Test, by $+0.006$, lost on the spliced rewrites, 0.633 against 0.652, and put a span on about 27% fewer Test humans, 140 against 191, all as three-seed means. A retrain that masked shorter spans out of the span loss stayed within about one seed standard deviation of S2c everywhere and was not adopted. A 16-word floor gained only on the spliced rewrites, whose short edits it targets; on the Test its 16–23-word runs had precision 0.07 and its character F1 fell by 0.016, so the floor stays at 24 words.

The first span package combines S2c's seed-17 weights with that 24-word decoder and a span confidence map refitted on CAL under the floor. Its graph and tokenizer are byte-identical to the earlier S2c package's, its document outputs equal R_0's on the local host on 215 synthetic, 200 local, and 15 parity requests, its Rust FP64 replay matches within 2.1×10^{-6} in token probability, and its CPU latency and memory stay within the earlier package's own round-to-round spread. Retrospectively, on the 5,173 consumed-Test mixed documents with a gold span of at least 24 words, it reaches character F1 0.544, with precision 0.922 and recall 0.534, span precision 0.771 and recall 0.505, and recall of 0.361, 0.440, and 0.559 in the 24–49, 50–99, and 100-plus-word bands; 151 of 6,132 humans carry a span. On the 545 re-read spliced rewrites of the report half with such a span, its character F1 is 0.639 and its band recalls are 0.555, 0.803, and 0.894. The refitted span confidence is better calibrated than an identity map on the Test, with expected calibration error 0.010 against 0.014, but overconfident on the spliced rewrites, 0.056 against 0.016; its CAL fit has only 128 negative runs. These are one-seed figures without a bootstrap, and spans of 24–49 words remain the weakest band. The package passes the current artifact spec unchanged, though that spec cannot refuse a shorter decoder minimum at admission.

On November 9 the package replaced the earlier S2c package as SlopNinja's span-serving package, after operator probes on the live evaluation scorer emitted the same spans as the local run, one 26-word span on one probe text and none on the other, with exactly the same offsets and labels. Live scorer hosts fall into two arithmetic classes: one reproduces the local host's outputs bit for bit, and the other differs in the last bits. Every probe of the package ran on the second class, where its document outputs equalled R_0's, and its sentence outputs the earlier S2c package's, bit for bit, as recorded on that class, and lay within 8.4×10^{-7} of the local run, the same spread that R_0 and the earlier package show between the classes. Live probe checks are therefore class-aware: a probe must be bit-exact to outputs already recorded on its own class, or to the local run on the class that matches it, with labels and offsets exact on every class. The probe reply names no host, so the class is identified by those matches.

Sealed competitive Test, October 30, 2026. R_0 and Pangram 4.1 were evaluated once on a sealed Test under a pre-registered protocol [57]. The Test has 14,904 documents (10,495 primary) in 5,030 source groups, including 3,286 primary human documents and 826 positives from twelve hosted frontier models. Its human documents are whole-paragraph excerpts of 150 to 450 words, so its results describe passages of that length, not longer documents. A scored every document with no execution failure. Pangram was observed on a pre-registered random subsample of 2,950 primary documents, one of them missing; the paired comparison uses a Horvitz–Thompson estimate of the full primary value. Both detectors' outputs map to involvement through the same CAL-only clipped isotonic method, so Pangram's mapped fraction is not its native confidence.

Measure	A (R_0)	Pangram 4.1
Balanced involvement Brier [95% CI]	0.0518 [0.0491, 0.0545]	0.0551 [0.0499, 0.0600]
Paired difference, A minus Pangram	-0.0049 [-0.0103, $+0.0005$]	
Human FPR at its own CAL 1% point	1.31% (43 of 3,286)	2.35% (35 of 1,490)
TPR at that point	0.861	0.801
Frontier slice, paired difference	-0.0039 [-0.0134, $+0.0052$]	
OLMo-2 slice (Test-only generator)	-0.0042 [-0.0137, $+0.0041$]	
Legal-opinions slice	$+0.0247$ [-0.0009, $+0.0557$]	

Table 7: Sealed Test, R_0 (seed 17) against Pangram 4.1. A's Brier covers the full primary Test. Pangram's Brier and every paired difference are Horvitz–Thompson estimates from Pangram's subsample, so the paired row need not equal the difference of the two Brier values. Negative differences favor A.

The pre-registered competitive criterion, called Tier C, required four conditions for both the three-seed recipe and the artifact R_0: an upper 95% bound of at most 0.010 on the Brier difference; a one-sided 95% upper bound of at most 2.5% on human FPR at the CAL 1% point for every seed; a gap of at most 0.030 on every slice the criterion named; and a completed result for every primary document. R_0 passed all four on its own, with an upper bound of $+0.0005$, at most 43 human flags against a limit of 67, a largest slice gap of 0.0247, and no failures. The recipe's legal-opinions gap was 0.0324, so it missed the slice condition and the pre-registered verdict is *fail*. Legal opinions are the weak slice; Wikipedia and Wikipedia talk favor A by 0.030 and 0.038. Against the frozen earlier adapted A (Appendix A), R_0's paired difference is -0.0734 [-0.0765, -0.0702], and its human FPR at the 0.5 cutoff falls from 21.9% to 6.2%; that separate improvement criterion passed.

The result is noninferiority, not superiority: the Brier interval includes zero. At the diagnostic 0.5 cutoff R_0 still flags 6.2% of human documents. Its mixed-document share error is 15.5 percentage points, above the 10-point diagnostic target. AI attribution top-1 accuracy is 0.35 overall and 0.16 on frontier voices held out from training. The Test is now consumed and cannot guide further fitting; the sizing, mix, and copy-policy analyses below use it for statistics only.

Container serving, October 29, 2026. f32 ONNX serving of R_0 on Cloudflare Containers passed parity with R_0's FP64 replay: the largest output difference was 4.7×10^{-6}, confidence was identical, labels matched, and outputs were bitwise identical across three instance sizes, all within that replay gate. Warm 2,048-token latency inside the Container was 7.9, 4.7, and 3.2 seconds at the median on the standard-2, -3, and -4 sizes; documents of about 300 tokens took 0.40–0.90 seconds. From zero instances the model was ready after a median 9.4 seconds. Busy-time cost was \$0.23–\$0.34 per 1,000 documents at 2,048 tokens and \$0.027–\$0.050 at about 300 tokens; an idle instance costs \$0.057–\$0.113 per hour, and one session peaks at 0.8 GB. The test used synthetic text on one CPU model, and its synthetic inputs saturated the calibration map. Distributional parity on evaluation-like text inside the provider boundary remains to be checked. Workers AI cannot host this encoder [37].

Cohort sizing, October 30, 2026. A power and cost analysis for daily cohorts used the consumed Test for statistics only. Its paired standard errors appear in Section 5.2. Scoring costs about 3.4×10^{-5} dollars per document including model loads. On testnet, with 1,000 cases and 32 entrants, generation and scoring cost about \$57 for a month of daily cohorts, including fixed fees; 64 entrants cost about \$91. These figures exclude the canary's audits (Section 5.4) and predate the fingerprint probe, which roughly doubles each executed entrant's scoring at 1,000 cases (Section 5.10). On mainnet, with 4,000 cases, one cohort of 20 to 256 entrants costs about \$6.45–\$38.26 with batch-API generation, which does not yet fit the daily cadence (Section 6), or \$2.72 more at standard prices. Evaluation documents average 307 tokens and never exceed 751. On testnet on October 30, a scoring instance's cold path, including streaming the artifact from R2, chiefly its 601 MB graph file, took 26–41 seconds, and a 32-document request took 7–10 seconds on the standard-4 size. Throughput beyond two parallel instances was not measured then; the reward rehearsal below later scored a cohort in 35 lanes. Projected from those two instances, the fan-out lanes of Section 6.2 would score a 1,000-case cohort in about 100 seconds for rosters of up to 75 artifacts, the most whose 200-document lanes fit the account's default limit of 1,500 concurrent vCPU, or 375

standard-4 instances [29], but the cold-start tail and aggregate R2 reads at hundreds of instances are unmeasured. The costs assume entrants the size of R_0 and receipts written in chunks.

Cohort mix and context bleed, October 30, 2026. A design analysis compared batch mixes on the consumed Test, for statistics only, with 2,000 stratified bootstrap replicates over its 54 domain×origin cells. Under self-weighting, the seed-pair standard error of q is 3.50% of headroom at 1,000 cases and 1.72% at 4,000 for 20/55/25, against 3.58% and 1.80% for 40/40/20. Against R_0, the paired standard errors of three seeds each of two other recipes, and of V6, are 12.8–18.9% of headroom at 1,000 cases under 20/55/25. The reference's headroom is about 0.062 under both mixes, but human-only documents carry 21% of its loss under 20/55/25 and 43% under 40/40/20. An independent recomputation confirmed the standard errors and the false-promotion rate, and corrected the human false-positive rate at the frozen 1% operating point to 1.33%, domain-weighted. Context bleed was measured on a smoke-test span head, selected and measured on an exposed Development iteration before the shipped head existed. Identical human words score a mean AI probability of 0.098 alone and 0.259 inside their mixed child, a difference of $+0.161$ $[0.152, 0.170]$; the token flag rate rises from 5.7% to 20.0%, and 93% of segments score higher in the child. At sentence level, 2.5% of human sentences are flagged in pure-human documents against 16.6% in mixed documents, and 33% next to an AI sentence. The excess falls from $+0.21$ within five tokens of an AI span to $+0.06$ beyond 100 tokens, in all seven domains measured. These optimistic figures come from one head; evidence across models is missing.

Copy-policy measurements, October 30, 2026. Near-copy thresholds and prices were measured on the consumed Test, for statistics only, from the calibrated predictions of three recipes with three seeds each and of V6. On 2,000 random 40/40/20 batches at each of six sizes from 256 to 4,000 documents, no independent pair fell within $d = 0.025$: the closest, two seeds of the weakest recipe, came no closer than 0.034 at 1,024 documents, and seeds of the launch recipe no closer than 0.059. Temperature, bias, and the best affine recalibration of R_0 sit at distance zero, and exact, recalibrated, and lightly perturbed copies all failed the gate. The cheapest escape, noise added to R_0's ranking after its best recalibration, was measured twice, by the implementer and by an independent verifier, each over 128 noise realizations with 200 batches per realization and size; the pooled figures appear in Section 5.9. Fifty-fifty output averages of R_0 with an independently trained seed scored 0.19 and 0.29 of a 1,000-case band above R_0 and escaped every batch; such averages need two encoders and cannot be submitted. Weight soups, and the credit of copies degraded beyond the cheapest escape, were not measured. Protocol fixtures exercise these rules natively and on wasm32 with byte-identical golden reports.

Public human stock, October 30, 2026. Counted from the sealed Test's reservation records, the unassigned evaluation-reserved human stock is about 240,000 expected documents under that Test's excerpt rules and per-group caps, all dated public text treated as historical-human proxies. About 175,000 lie in the eleven domains added for the sealed Test, 95% of them in four conversational and blog domains; Wikipedia holds about 51,000, FineWeb about 10,000, and each other earlier source fewer than 1,200. Seven of the eighteen Test domains retain fewer than 350 documents, and five others between about 600 and 4,300. These are expected yields from reservation metadata; no live pool has been drawn or opened. The stock holds about 3.4 million sentences: a sentence Bloom filter of it at a 1% false-positive rate takes 4.1 MB, 64-bit sentence hashes 27.5 MB, and its raw text 327 MB. On a 40/40/20 batch, recognizing all of one domain's human-only and mixed documents would gain 11.9% of the reference's headroom for Wikipedia and 6.6–9.0% for each of six other domains.

Common Crawl human pilot, October 30, 2026. A pilot sampler downloaded 178 Common Crawl WET files, 24.45 GB holding 8,245,186 records, from the pre-cutoff frame of 89 crawls and 237.84 billion captures [58]. It found no digest or parse errors, and 50 of 50 byte-range re-fetches reproduced their records. After screens and deduplication against the existing corpus, 416,492 excerpts were admitted, 17,031 per GB, with a median of 204 words; pages carry 5.53 valid passages on average. Admitted yield per GB ranged from 3,601 for news and 2,816 for essays down to 37–39 for speech and fiction and 66 for email and business text, 84% of which were press releases. R_0 flagged 155 of 4,954 sampled documents at its 1% Calibration operating point, 3.13% [2.68, 3.65], or 2.20% weighted to the pool, against 1.31% of the sealed Test's humans. The rate shows no trend with crawl year ($z = 0.19$); later crawls exceed the earliest by $+0.47$ points $[-0.51, +1.52]$. Fiction is flagged at 11.6% in every year from 2013, a detector gap rather than contamination. Of 45 flagged documents reviewed, 38 read as human-written, four were machine-made or machine-processed, one was a templated page, two were translated web novels, and none read like generated prose of the GPT-2 or GPT-3 era. Pangram 4.1 labelled all 302 documents

it observed human, an upper bound of 1.25%. Neither detector's sensitivity to machine text from 2019–2022 has been measured, so these rates bound only what they detect. A hashed n-gram probe trained on humans alone separates 2013–2016 text from 2020–2022 text at AUC 0.688 [0.672, 0.703], or 0.672 with digits masked, through topical cues; on the Test it separates model-only documents from same-domain humans at AUC 0.72 and mixed documents at 0.56. R_0's human scores show no drift with year (AUC 0.496) and no correlation with the probe ($r = -0.01$). Lookup measurements put the frame at about 64 billion first-seen URLs and give the capacity-bound gains and share caps of Section 5.4; a 1,000-anchor canary detects a close fine-tune's lookup with 80% power only at a gain of about 20–23% of headroom. Data access costs nothing.

Common Crawl stock qualification, October 31, 2026. Follow-up measurements on the pilot pool fixed the launch rules [58]. Three rounds of hand-labelled domain review, each a fresh draw of 450 documents that withheld those read before, raised pooled usable precision from 49% under the first domain rules to 72% [64, 79] under the third, both weighted by launch shares. By domain it ranges from 62% for legal text to 84% for speech and email, and machine text fell to 7 of 450. The density rule's caps on the third rules' frames set the launch shares of Section 5.3. Under the third rules speech yields about 24 excerpts per GB of WET text; at its 2.3% share it is the scarcest domain, sets the volume, and the other domains fill from the same files. Even 584,000 documents a year, twice the human negatives that mainnet batches consume, then take about 0.56 TB of WET text at nominal counts, 0.08% of the frame's WET text, and about 0.67 TB as usable documents at speech's 84% precision. The lookup canary's audit draws, about 7.3 million documents a year at one audit a day (Section 5.4), would take about 7.0 TB at nominal counts and 8.4 TB at that precision, 1.0–1.2% of the frame. A Common Crawl-parented mock cohort of 400 human-only, 947 mixed, and 480 model-only documents from three local generators, reweighted to 20/55/25 and built under the cohort generation profile, passes both gates: the era-only adversary gains 2.9% [1.5, 5.04] of the reference's headroom and the layout-and-typography adversary 1.6% [0.3, 4.1]. As diagnostics, the character-class block gains 6.9%, above the gates' 5% bar and the 1,000-case margin of 5.1% of headroom given below, which is why Section 5.3 states it as a limit; the era probe separates parents from dated mirrors at AUC 0.714, and mean word length alone gains 16.7%, a lexical signal that the detector itself reads; the length window brings word count's own gain to 0.2%. On this batch the headroom is 0.112, and the seed-pair standard error of q is 2.53%, 1.79%, and 1.26% of headroom at 1,000, 2,000, and 4,000 cases, giving δ of 0.051, 0.036, and 0.025. On mixed documents whose response was only normalised, R_0's Brier rose from 0.089 to 0.143: the reference leans on generator formatting, which is why the next training round retrains and recalibrates it on normalised, length-matched positives. A membership-twin set of 698 pages first captured after the cutoff, matched by host and month to 525 frame excerpts, shows no difference for R_0 between the two arms (AUC 0.464). Three close honest models scored on it give an honest-model offset of 0.0073, with a systematic part of 0.0035, too imprecise to resolve the 0.0025 the canary design targets, so a 10,000-twin bank is being built. In the red team, two small encoders that share ModernBERT's tokenizer and pretraining data containing Common Crawl showed no membership signal in their masked-language-model loss, and a linear membership head on R_0's encoder recovered 8.5–14.7% of the pages it was trained on at a 0.1% false-positive rate. The test on ModernBERT-base's own pretrained weights, the narrow strata at full size, and a stronger attacker remain.

Writer profiles, October 31, 2026. A frozen protocol compared writer profiles from the native CFG exporter, from R_0's encoder, and from a stylometric control of 203 function words and the 2,000 most frequent character 3-grams, alone and in score fusions, on 630 evaluation and 474 training writers from six public domains whose own documents neither system had seen [59]. Profiles were unlearned: an average of normalized, standardized reference vectors, scored by cosine. With five references, pooled verification AUC was 0.743 for CFG, 0.778 for the encoder, 0.828 for stylometry, and 0.846 for the encoder fused with stylometry. CFG fell below the encoder by -0.034 [-0.047, -0.022]. Fused with the encoder it added $+0.027$ [$+0.022$, $+0.032$], and added to the encoder plus stylometry, in a follow-up on the same writers, only $+0.0015$ [$+0.0004$, $+0.0025$]. Operating points were weak for every profile: the strongest still passed 42% of impostor trials at 90% of genuine ones, and no threshold fitted on training writers held its false-accept target on withheld writers. For author fit, the nearest available data were hosted-model rewrites in a voice derived from one passage of the target writer. Every profile ranked them near chance, with mean per-target AUC 0.53–0.55, and the kind of rewrite moved scores more than the target did. All profiles used unlearned matchers, so these tests do not measure the learned matcher the design requires. CFG writer vectors therefore do not enter writer profiles; the grammar's planned role is B's edit representation (Section 10).

14.2 Implementation status

Implementation boundary. The native Rust adapter in this repository pins `rtao fa83052d` (November 5, 2026) [32]. It uses durable journals, signer-free restart reconciliation, persistent weight/reveal scheduling, semantic guards, and explicit pause/resume. Its journal uses schema 3 and migrates schema-1 and schema-2 journals on open, refusing any it cannot carry over with the original bytes retained. Nothing compares a runtime spec or transaction version: signed operations resolve by name in live metadata, and only an operation without a type-correct encoding is deferred. At this pin `rtao` composes weight calls, and reads the storage of its weight preflight, scheduler, and reveal watcher, through compiled descriptors while their live shapes match and by name otherwise, and it follows the chain's stored commit–reveal version. What it can read neither way is recorded as a deferral; for a storage item, finalized observation, reveal tracking, recovery, and both weight operations then stop together until the chain or `rtao` changes. Two such stops are known risks for a later runtime: metadata larger than the coordinator can store in one value, about 1 MB, and a changed layout of the epoch-index keys, which the evaluation service still reads through a compiled layout. `rtao` supports journaled shield dispatch from per-runtime evidence, but the adapter submits unshielded only, and no runtime range is declared validated, which no SlopNinja component requires. Adapter, validator, miner, and protocol tests passed with Clippy on Linux at this pin, without keys or transactions. The native validator's live testnet commitments ran at the earlier pin `b5553a76` (the "Testnet weight commitments" and "Testnet reward rehearsal" paragraphs below) and at this one (the "Testnet record restart" paragraph).

The production native validator and the serverless coordinator, which is itself the lead validator, remain unfinished, as does finalized-RPC recovery qualification. The shared protocol crate implements the `reward-v2` calculation over `quant-v1` quantized scores with the per-cohort bit budget, the public-copy gate and near-copy relation, private submission records, the launch and reward-intent authorizations, which never expire, with their signed revocations, the coordinator's signed row record, the cohort phase checks, and the check of a row against the target its holder has active, as exact computation that reproduces byte for byte on native and wasm32 targets [20]. Since the testnet record restart, testnet cohorts run `reward-v2` with quantized scores, take on-chain digest commitments and direct R2 uploads under the current artifact spec from every registered hotkey, and publish every admitted artifact when scoring starts. The coordinator's evaluation service builds probe versions and fingerprints, keeps the published-artifact index, measures the gates, and signs packages; the coordinator commits the resulting rows, and the native validator fetches the signed rows from a mirror of the coordinator's published layout and commits them. Generation that takes half of a cohort's text from frontier models is implemented but not yet enabled, so testnet batches are generated by open models on Workers AI. Not yet implemented are signed corrections; the lookup canary's statistic, since testnet has no anchors and its flags come only from drills; scoring the canary's audit draws for promotion candidates only (today every executed entrant scores them); rebuilding the published-artifact index instead of trusting the coordinator's copy; binding that index by digest, since every active-target bundle embeds the whole index, which outgrows the coordinator's single stored value at about 5,000 published artifacts, about 130 full testnet cohorts; the signed automatic updater with its resumable migrations (Section 6.10), so participants update by hand; the successor of `reward-v2`, which places a near-copy cluster by its earliest-admitted member, while `reward-v2`, which the protocol crate implements and testnet runs, places it by its highest-ranked member (Section 5.9); the paid detection API and the pot of Section 8.1, with its attribution rule, its serving checks for the reigning reference, and its claim contract; and the gate feedback record of Section 5.7. The testnet exposed-pool reuse rule of Section 5.3 is built and, as of November 11, awaits installation. Detailed-receipt deletion runs, with the one gap noted in Section 6.8. Legal removals (Section 3.4) have no dedicated tool: the operator performs each by hand and records it publicly; none has been needed.

Built, but as of November 11 neither deployed nor released, are the parts below. None changes what testnet runs until it is deployed, and the span gate, the pre-finalized scores, and the segment readings change nothing until a cohort template names them. Two operator paths already run in an earlier, deployed form without the operator-decision key: the record restart, as used on November 6 (Section 14.2), and the abandonment of a cohort stopped by a fault, for that fault. The validators' side of a restart is released: `testnet-v0.2.0`, published on November 11 as the first signed release with validator binaries, accepts a restart's launch authorization without a manual reset and refuses rows from before its floor (Section 6.6).

- The operator decisions of Section 6.6 with the verification of the operator-decision key, except the rotation of the authorization key at re-registration and some supporting checks and tools, among them checks of the span-equivalent reference decision of Section 5.6. Until the key is configured, every decision that needs

it is refused, except the switch to burn, which then runs unsigned, as the record restart of November 6 did, so a missing key never blocks a restart.
- The span gate in report mode, with artifact spec v5. In a cohort whose template names the gate, the evaluation service evaluates every entrant's spans against the anchor and signs the per-entrant record of Section 5.11, which is not yet published; enforcing the gate is not yet implemented. No cohort has yet evaluated spans.
- The pre-finalized scores and segment readings of Section 5.8, and the refusal of subnormal weights at admission (Section 15.2).
- The service behind the public detection page of Section 8.1. The page itself opens only after its parity, abuse, and wording checks.

Testnet cohorts still run in a separate evaluation service. Native signing, commit–reveal, and recovery were exercised on testnet (the "Testnet weight commitments" paragraph below). The coordinator's own commit and reveal have also run live there through the Workers chain path, but its signing is not yet qualified.

The coordinator additionally requires Workers-compatible transport, chain encoding and signing, a Durable Object intent/attempt journal, and recovery for its own commitments and reveals. Native `rtao` tests do not qualify those components or establish Wasm support. `rtao-rpc` block webhooks are live for Finney and testnet; the authenticated receiver, durable cursor recovery, and the watchdog's public-RPC fallback remain integration work. A subscription that the provider pauses after exhausting its retries leaves the coordinator in fallback, which the watchdog reports for an operator alert; it is not yet resumed automatically. Neither a qualified webhook receiver nor a serverless signing path is claimed as deployed evidence here.

Research utilities include corpus/evaluation tools, a public Pangram-report reader, and an HPKE receipt prototype whose trusted assignments come from its caller. That prototype establishes neither a deployed key registry nor the customer protocol [60]. No trained grammar-based B ranker exists, and no experiment has yet validated joint preservation, author-fit, and evasion performance. A production origin service and a commercially validated transformation service both remain to be demonstrated.

Testnet weight commitments, October 30, 2026. On testnet subnet 601, the native validator signed timelocked weight commitments (commit–reveal version 4) with a registered validator hotkey, for a coordinator-signed fixture package derived under a testnet-only reward rule rather than `reward-v1` [61]. The first commitment, in epoch 7, used the target's operation identity, and each re-commitment its own per-tempo identity. Each operation for epochs 7 through 11 holds one signed attempt, which finalized. A drill withheld the first signed attempt of the epoch-12 operation, which expired unexecuted; the same operation was signed once more, with the same nonce, in epoch 14, and that second attempt finalized. After every reveal the validator's on-chain weights equalled the package's integer row. Fault drills crashed the process at each journal phase boundary, killed it after the node acknowledged a submission, dropped acknowledgments behind HTTP 429 responses and behind a failed endpoint, and withheld a signed submission until its era expired. Every restart recovered from the journal, and finalized read-backs found no duplicate commitment. The journal ended with seven operations, none unresolved: the six committed ones, which hold the seven signed attempts (six finalized and one expired unexecuted), and an epoch-15 re-commitment that was pending and not yet signed. An independent verification passed. The drills exposed `rtao` behaviors now reported upstream: a reconnection that meets an HTTP 429 leaves its session failed, so the application restarts with fresh connections, and each commitment makes 26 to 28 requests within its 30-second window from nonce lease to pool admission, above the public endpoints' guidance of 20 requests a minute. Reveal tracking replays every finalized block at about 14 requests a block. Miner incentive and emissions were not part of this qualification.

Testnet reward rehearsal, October 31 to November 1, 2026. Rehearsal cohorts on subnet 601 ran `reward-v1` end to end under signed manifests with $K = 4$ tempos, 1,000 cases mixed 20/55/25, $\sigma = 0.035$ and hence $\delta = 0.07$, one 1,024-document probe version, the rewrite-source limits of Section 5.2, and provisional gate thresholds [62]. Human documents came only from exposed pools. The reference was R_0 with its calibration flattened halfway to 0.5, a rank-preserving change that R_0 itself could beat by the margin; this record is separate from the one testnet opens at R_0 (Section 5.6). Entrants were variants of R_0: exact copies, rank-preserving recalibrations, noise added to head weights, a soup of head weights, and artifacts that fail every document or never load. Generation took about 6.5 minutes a cohort, and scoring a cohort of seven entrants 4.6 minutes in 35 lanes; in one cohort 28 of 30 scorer starts failed once and were retried. With the reference alone, a cohort produced no target and the launch

burn row stayed active. In a cohort of seven entrants, R_0 cleared the margin as the only cluster head above the cutoff, a canary drill flagged it twice, so nobody was promoted, and its cluster received the whole row. Exact copies of published artifacts were never executed. Recalibrated copies and a head-weight soup within the near-copy distance failed the public-copy gate, on the batch or, against an earlier artifact's fingerprint, on the probe; a noisy variant that cleared the batch comparison failed against the reference's fingerprint on the probe; and artifacts without predictions scored zero and failed the share and loose false-positive gates. Two later cohorts, one after review fixes, produced no target and carried that target forward. An offline verifier checked every package and closure, and the native validator verified every bundle it used, including all 14,344 receipts of the seven-entrant cohort. The verifier also reproduced every row digest that the two signers reported, from the package and a registration snapshot at the same block. The coordinator committed the launch burn row at eight consecutive epochs and the cohort target from its activation, and the native validator, which then abstained from all-burn rows, committed the target from activation and again under a carrying package's authorization; every reveal read back equal to the committed row. After the fixes both signers kept their own record of the active target, refused the launch authorization after activation, and re-derived each tempo's row from that tempo's snapshot, and a plan naming a fresh pool was refused. No live row paid two UIDs in these rehearsals, because no available artifact beat the reference by the margin while staying clear of every published one: the max-upscaled form of such a row, promotion, the fallback cutoff, and canary re-audits were then covered by tests alone, and the fault drills of the "Testnet weight commitments" paragraph have not been repeated under `reward-v1`.

Testnet record restart and open cohorts, November 6 to 9, 2026. On November 6 a rehearsal cohort of the testnet record then running, whose reference was R_0, promoted a test miner's entry, one of SlopNinja's own experimental models, which cleared the margin; its target, a row paying that one UID, was never committed. The operator restarted testnet's record from R_0 by decision instead of keeping that promotion. With signing paused, the coordinator set a restart floor, kept the target it set aside in its record, and committed the `reward-v2` burn row from the floor under a new launch authorization. A fresh record then began at R_0 under `reward-v2`, `quant-v1`, and the current artifact spec, with a probe build as its first cohort. The native validator, rebuilt at the current `rtao` pin and reset by hand, verified that launch authorization and committed and revealed the burn row, each reveal equal to the committed row. The restart also needed code changes; the operator decisions of Section 6.6 specify journaled paths that need neither. On November 7 admission opened to every registered hotkey, and within minutes an outside participant, registered from a coldkey that is not the operator's, committed and uploaded an artifact. The first standing cohort of the new record, with $K = 20$ tempos, 16 of them for admission, 1,000 cases, and provisional gate thresholds, then ran end to end unattended. Its eight entrants, seven operator test miners entering distinct versions of SlopNinja's own models and the outside entrant, each committed on chain and uploaded straight to R2; every admitted artifact was published when scoring started; all eight were scored; one test miner passed every gate, and the other seven, the outside entrant among them, each failed at least one. Nobody was promoted, and the cohort closed and published its results on the public status page [63], which also lists each running cohort's intake. A second standing cohort, again with an outside entrant among eight, finished scoring on November 9. Both drew exposed documents only (Section 5.3), so their scores show that artifacts run and are scored, not how they perform on unseen text. A drill that deletes an admitted object to exercise upload repair has not yet run in a live cohort, and no live row has yet paid two UIDs.

15 Development and launch criteria

Launch begins with A alone. Testnet runs for several months before the mainnet subnet is registered, and the rehearsals below run there. A rights-cleared offline comparison comes before live rewards; a coordinator-scored network rehearsal comes before paid service. Neither completing a cohort nor qualifying for rewards authorizes sales. The coordinator opens a bounded commercial workload only after recording its quality evidence, supported document classes, capacity, and operating budget. B transformation and the C grammar contributions that support it each require a later activation decision.

15.1 Model qualification

A's input contract is raw text (Section 3.3), and the reference R_0 was compared with Pangram on a sealed, pre-registered Test (Section 14). It was noninferior as a single artifact, but its recipe missed the legal-opinions slice

limit. Later reference changes follow Section 5.6: the cohort promotion rule, plus signed operator reference decisions with recorded reasons, among them the span-equivalent decision that passes R_0's position to SlopNinja's first span package and a planned replacement of the testnet's R_0, announced in advance, by the next reference SlopNinja trains, only if it passes every required gate; external comparisons continue under Section 12. Within each new architecture, compare the joint model with origin-only and attribution-only controls using matched encoder capacity, data, and training budgets. Measure whether attribution supervision improves origin detection and whether origin supervision improves attribution, including cases where either task is harmed.

Keep each source work and its generated or edited descendants together across Train, Development, Calibration, and Test, and freeze gallery policy, model versions, and thresholds before opening final evaluation data. The qualification set needs independent reference/query works, withheld writers, unseen generator families and versions, mixed editing histories, and cross-topic controls. Human proxies and documented human-only histories remain separate strata. Report AI-involvement Brier score, calibration, sensitivity at declared human false-positive operating points, and source-family uncertainty. Report attribution and open-set rejection separately, including unseen generators, ambiguous listed-model cases, and mixed work with a supported human match. Qualify both conditional fallback categories on the documented contributor roles, including mixed work with neither identity in the gallery. Assess absent roles through involvement probabilities, without treating a conditional fallback as a claim that the role is present. Include low AI confidence with supported named-model matches to test that attribution remains visible. A model failure stays in requested-population coverage; paired accuracy on successful documents cannot hide it. Include cold starts, tail latency, memory, and total delivered-document cost. Any change to A's input contract, including grammar features, needs a new paired comparison with raw-text models, including its added cost, before admission.

Before mainnet live rewards, re-measure the era-only gate of Section 5.3 on normalised, length-matched positives from the next training round, with a cohort large enough that its upper bound falls below 5%. Score each quarterly pool sample with all three detectors, finish the red team, including the test on ModernBERT-base's own pretrained weights, and re-derive σ, δ, w, and the headroom on a Common Crawl-parented batch. Record each pool's measured domain precision and size its draw by it. Measure R_0's false-positive rate and Brier on freshly written anchors against same-domain Common Crawl humans, first at 300 anchors. When the reference testnet will hand on differs from R_0, report the same measurements, its Common Crawl-parented batch score, and a canary audit for it.

For the later personal-profile workflow, the next qualification is a learned matcher over encoder and stylometric profiles, trained on fresh authors, with its acceptance thresholds measured on withheld writers and on real style-transfer data. Evaluate enrollment from independent samples and *Provided writer* matching on separate query works across sample counts, lengths, topics, registers, and outliers, comparing the proposed pooling contract with qualified alternatives. Test cross-customer isolation and fallback when the supplied profile fails the matching rule. Before B uses A's author-fit feedback, verify on rewrites conditioned on several of a writer's own samples that iterative gains against the fixed profile agree with blinded judgments of voice while preserving meaning. Keep adaptive reference-copying and metric-gaming controls from Section 10.3. These tests qualify a later product workflow, not an additional A-only launch requirement.

Qualify document-level AI share on controlled partial rewrites with measured final-span fractions, including zero and complete replacement, dispersed edits, and small rewritten regions in long documents. Keep all versions of each source in one split. Report source-balanced share error, failures, uncertainty, and transfer to held-out generators and editing procedures alongside binary-label confidence calibration. Neither a calibrated involvement score nor accurate pure-human/pure-model classification establishes fractional accuracy. Measure span outputs under the 24-word floor on the sealed batches of closed testnet cohorts, fix the span gate's margins from those measurements, and confirm them on live cohorts that report the gate without enforcing it before the standing template enforces it; measure again before enforcing on a changed cohort population, mainnet's included. Artifact spec v5 adds the 24-word decoder minimum and the token-level head requirement, so that admission refuses other artifacts, and needs a miner client release. Measure the span package's calibration on fresh documents with more negative runs than its Calibration fit holds (Section 14.1). Word-level explanations are a later qualification and do not delay launch; their tests must establish explanation faithfulness as specified in Section 3.2.

15.2 Protocol checks before launch

The first coordinator-scored tournament freezes entrants and a fresh hidden batch, records every assigned outcome, and publishes the public quantized result. The serverless coordinator derives and signs the native weight row, and Rust validators, the coordinator included, commit it through commit–reveal in parallel. Anyone can audit the coordinator's certificate and finalized row afterward. Hidden labels, individual predictions, and exact scores remain with the coordinator, and no hidden input goes to miners or other validators. The tournament must include both reference models, malformed output, targeted triggers, an embedded text index, an artifact outside the allowed graphs, with another tokenizer, or with a file that fails its role's schema or has no declared role, outputs that change with a document's batch-mates, missing uploads, uploads that do not open their commitment or open one made for another cohort, chain, or hotkey, exact, recalibrated, perturbed, and deliberately degraded copies of published artifacts, including copies that match only on the probe, weight soups of published models, crashes that do and do not reproduce in isolation, and selective execution failures. Measure, under the frozen rule and on simulated rosters of honest entrants that resemble seed retrains or differ structurally, the expected credit of a degraded copy and of a weaker variant aimed below a crowded band, the distribution of honest entrants' pay across cohorts, the fallback cutoff's pay rates for entrants equal to R_k and to R_{k-1}, and the drift of the reference's true quality across promotions (Sections 5.6 and 5.9). Commitment order requires a valid opening and qualification; it is not proof of authorship. Verify that no submitted artifact is retrievable by another participant before its cohort's scoring starts, that every admitted artifact is published when scoring starts and every opening when the cohort closes, that no upload byte passes through a SlopNinja Worker, and that R2 stores nothing for an upload whose bytes or length differ from the declared ones. Canonical-byte controls cover Unicode normalization, invisible characters, whitespace, malformed encodings, and mismatches between submitted, scored, and externally analyzed text. No representation-only change earns utility. The full reserved workload determines the tempo targets; a small protocol test cannot establish production throughput. Rehearsals may use $K = 4$ manifests to exercise deletion and recovery within hours.

Rehearse the Workers and Durable Object state machine by interrupting dispatch, result persistence, scoring, weight submission, and transition completion. Expire queue messages and task claims, duplicate deliveries, drop block deliveries, return results from superseded workers, interrupt migrations, and restore from durable exports. Rehearse above the testnet roster cap: at least 64 scored artifacts in parallel lanes, measuring cold-start and artifact-load distributions, rate-limit behavior, lane loss and reconciliation, and pause and resume during scoring. Authoritative records must reconstruct unfinished work after an arbitrarily long pause. Results need one canonical outcome per task under the frozen attempt policy. A transition issues no new challenges; a recovered coordinator resumes the same cohort. Slow work can move advancement to a later tempo without changing evaluation semantics or discarding observations. Interrupt coordinator checkpoints across several tempos and verify that remaining application deadline budgets survive without retroactively charging the silent intervals. Coordinator work is fenced by authorization and checkpoint generation, never by elapsed time: a superseded or revoked work authorization must stop new coordinator work, never validators' commitments, while existing chain and customer obligations retain their separate deadlines.

Rehearse every operator decision of Section 6.6 on testnet from broken and stale states: abandon queued, running, and faulted cohorts; refuse to pay a finished cohort by each route; switch to burn and open a fresh record while a cohort is held, with validators following the new launch authorization without a manual reset; change the reward rule through a fresh record; set references by decision; and re-register the subnet on the same chain with a rotated authorization key. Verify that each decision is journaled and published with its reason, that the high-stakes ones refuse a missing or wrong operator-decision signature, that a lost operator-decision key can be rotated without it, that no cohort is skipped by time, no number reused, and no commitment signed twice, and that rows from before a restart floor are refused.

Qualify `rtao-rpc` webhook delivery. Reject unauthorized notifications; duplicate and reorder valid ones, lose deliveries across several tempos, and interrupt the handler between durable recording and acknowledgment. Verify RPC backfill from the saved finalized height and hash, fork handling for best-head events, and recovery without advancing past unprocessed finalized effects. Stop deliveries and verify that the one-minute watchdog enters fallback after the threshold, recovers the saved work from public RPC in bounded ticks, flags cursor lag while deliveries continue, and returns to webhooks when they resume. Let a subscription exhaust its retries and pause, and verify that the watchdog carries the coordinator and raises the operator alert until the subscription is resumed. Measure actual tempo-boundary observation lag with webhooks enabled and disabled, including

provider polling, scheduler delay, and RPC catch-up. No webhook can authorize signing, reset active-time budgets, or create a replacement cohort. These tests must run without a resident chain subscriber or polling server.

Native-chain rehearsal verifies the recognized owner's execution path for required hyperparameter changes under the actual ownership or lease arrangement, including enabling native weight commit–reveal. Every validator checks the effective setting at startup and before signing. An unexpectedly disabled flag must pause new commitments, with no fallback to direct submissions. Measure commitment inclusion, reveal intervals, finality, rate limits, normalization, quantization, bonds, registration changes, UID reuse, stale RPC views, and cohorts with no new target, including the subnet's emission share while miner credit burns. Miss a commitment window and verify later valid delivery without loss of the cohort. Exercise the native timelocked reveal path; automatic reveal cannot be withheld pending a coordinator check. Re-commit the active target at every tempo under per-tempo operation identities across a whole cohort, then silence the coordinator for longer than the renew-by hint and verify that validators keep committing the last signed row; issue a signed revocation and verify that new commitments of the withdrawn target stop while submitted obligations recover. Restart validators with uncertain commitments and pending reveals. Upgrade the chain runtime between cohort creation, commitment, reveal, and finality, including an upgrade that removes or changes a signed call: work continues, and only an operation without a type-correct encoding defers until live metadata admits it. Journal recovery must precede new signing, while existing native obligations retain their original deadlines. The native validator's testnet qualification (the "Testnet weight commitments" paragraph in Section 14.2) covered commitment, per-tempo re-commitment, and recovery under a fixture package, and the `reward-v1` rehearsal (the "Testnet reward rehearsal" paragraph in Section 14.2) covered verification, commitment, and a carried target under `reward-v1` packages for a row paying one UID, and the record restart (Section 14.2) covered the native validator's commitment and reveal of the `reward-v2` burn row under a restart's launch authorization. Still required are a live row paying several UIDs, the fault drills under the coordinator's signed `reward-v2` rows, native validators' commitment of an all-burn row under a cohort target whose credited miners have all left, qualification of the coordinator's own signing, and an immunity period above the first-payout lag. Successful scheduling alone does not prove finalized execution or cohort settlement. Verify that an old UID's credit cannot transfer silently to a new registrant. Subtensor can continue paying native emissions during a coordinator outage, so stale-weight behavior needs an explicit tested policy.

Test tampered row records, authorizations, revocations, and certificates, invalid score-signing authority or coordinator hotkey bindings, stale revisions, mismatched finalized rows, runtime changes (work continues; only an operation without a type-correct encoding defers), UID reuse, and owner-hotkey rotation, including to an unregistered hotkey and during coordinator silence. Recompute the row with an independent implementation on Linux and macOS from the same public result and registration snapshot, including shuffled input order, ties, zero entries, rounding boundaries, and cohorts with no new target, and compare it with the Workers coordinator's signed row. Their canonical integer rows must match exactly, and validators on both platforms must commit the signed row unchanged except that the burn entry and the units of a registration their own snapshot no longer holds go to the UID of their own snapshot's owner hotkey.

Verify the `reward-v2`, successor, and `quant-v1` calculations against hand-worked fixtures: the grid width, offset, and noise draws, including one shared draw for identical copies under several hotkeys, the withheld output of the minimum-success gate, the bit budget at plan validation, seal, and publication, the rounded margin cutoff and promotion on the exact margin, the fallback cutoff with and without a predecessor reference, a cohort with no new target, with and without a previous target, a perfect reference, the public-copy gate on the batch and on the probe, the canonical fingerprint of identical content, cluster formation and placement under each rule, equal splits, the bits of the span-gate and gate feedback records, tie groups, band boundaries on the grid, a lone cluster below a crowded band, largest-remainder units with equal remainders in roster order, a departed hotkey's burn, a deferred burn row, a reference-model failure, a promotion that passes to the next retained cluster head, the max-upscaled committed row, and allocations and caps. Exercise the launch authorization before the first manifest, a first cohort with no target, a probe-version replacement, a signed revocation and a later re-authorization, and the per-tempo active-target check. Each validator uses its own key, nonce, and journal, and commits without waiting for a coordinator reveal. A chain conversion change requires an authenticated compatible delivery revision, preserving the frozen scores. Publish a new score revision before its package is ready and verify that validators keep committing the previous authorized target within its identity and native-window limits. Preserve pending commitments when a successor target or authorized coordinator hotkey rotation arrives; a new target cannot reset old obligations or cause a stale row to overwrite its successor. An absent or divergent coordinator certificate

must not block healthy validators; pause only locally affected work when the evidence exposes a defect in its own target, authority, or context.

Verify, inside the coordinator before it installs a target, the R2 package's index, count, and receipt digests for successful, failed, retried, and pending calls without revealing prompts, responses, or labels, and confirm that no receipt reaches the public host or a validator. After cohorts $N+1$ and $N+2$ complete, delete N's detailed receipts and index, including managed copies, while preserving compact score inputs, calculated rows, row records, certificates, and recovery state, and, for a cohort that produced a target T_N, only after T_N has been superseded (a cohort that produced none waits only for $N+1$ and $N+2$). Missed tempos and coordinator silence must not trigger deletion, and the artifacts of R_k and R_{k-1} must remain hosted. Recover a long-offline validator from its journal and the current signed row; deleted receipt objects must not be needed to reconcile existing submissions.

Set every other validator to silent, offline, refusing, or an old release. The healthy coordinator must still finish evaluation, publish results, complete the transition at a permitted boundary, begin another cohort, and serve admitted customer jobs. Its deployment and recovery must also work without validator acknowledgments. Track absent chain submissions separately; this test does not establish that native payouts match the prescribed rows. Delay coordinator delivery and certificate publication as well, and verify that cohorts still advance. Restore validators with uncertain submissions, then verify independent catch-up without replaying stale targets, deleting obligations, or duplicating credit. Separately rehearse migration from an earlier direct-submission configuration and recovery across commit–reveal versions. Resolve uncertain transactions and preserve existing commitments under their original rules; changing the flag, updating software, or publishing a new row cannot erase those obligations.

Settle participant update opt-out/deferral policy and staggered rollout sequencing before launch, within the compatibility and independent-progress requirements of Section 6.10. Exercise signed Rust updates after many missed releases on Linux and macOS. Interrupt every migration step, retain its checkpoint, and resume the complete ordered path while preserving keys, journals, and pending operations. Test invalid signatures, release-key rotation, stale release metadata, missing migration paths, and accidental old-process startup after migration. Updated participants must reconcile before new signing, while other participants and the serverless coordinator continue independently. Rehearse coordinator schema migrations with unfinished cohorts and enforce generation fencing. Test the single Rust supervisor's handoff, exclusive journal and signer ownership, and schema-compatible rollback without losing newer transaction evidence.

The cohort length, batch size and mix, human stock, authorization rule, reward rule, score quantization, copy policy, parameter bounds, and the treatment of a cohort with no new target are fixed above, except that mainnet's privacy budget, batch size, or budget split must change first: at 4,000 cases and 256 entrants the current values make rewards mostly noise, and the bit budget refuses the plan. Qualify the `live-v1` budget of Section 5.8 at mainnet's batch size and roster together with the quantization budget. Before mainnet, also close the timing channel that slow arithmetic on chosen inputs could open in the published challenge counts, by flushing subnormal values to zero after requalifying every served output or by coarsening the published progress times; refusing subnormal weights at admission removes only its simplest lever. Also settle before mainnet what the entrants of a cohort abandoned after its scoring started keep (Section 6.6): whether the abandonment first lets the publication of their artifacts finish, and whether their contents join the published-artifact index at the abandonment, so that copies of them earn nothing, or the entrants carry over to the next cohort without committing again. Before live rewards, settle active-time accounting, retry and canonical-attempt rules, gate thresholds, including both false-positive tiers, correction authority, the criteria of the signed review that clears a blocked candidate, and shared-failure treatment, choosing them on development evidence. Mainnet batches of 4,000 cases may use batch model APIs with no stated completion time; generating ahead of a cohort would need its own authorized phase, which is not yet specified. There is no validator-adoption threshold or membership denominator in cohort advancement. The production application must bind signed results to the actual weight calls and retain reveal evidence beyond a scheduler's diagnostic ring. Qualify Linux/macOS participants and the coordinator's Workers chain path separately; native `rtao` support does not establish Wasm compatibility. The serverless path must qualify transport, encoding, signing, nonce ownership, durable attempt records, finalized reconciliation, and commitment/reveal recovery. Interrupt signing and submission across deployments and Durable Object migrations, then verify that only the current signing generation can create a new attempt and that uncertain earlier effects are reconciled. The coordinator must perform all lead-validator duties without a separate native signer or daemon.

15.3 Commercial activation

A paid *A* pilot exercises centrally served published artifacts, durable job acceptance, private delivery, idempotent receipts, and separate customer cutoffs. Customer delivery and refund duties survive a paused evaluation cohort. Qualify Coinbase Onramp's native-TAO route, available card methods, wallet requirements, transfer holds, and any additional Solana entry route. Verify burn authority under the actual ownership or lease arrangement. Test duplicate provider events, late finality, failed conversion, burn allocation, and original-payment refunds, and qualify the merchant-of-record processor for the detection API. Rehearse the revenue pot's multisignature custody, monthly statement, and tranche mechanics on testnet with test funds before its first deposit, and its registration payment and claim contract before they are used (Section 8.1); reconcile its published balance with its payout records and its credited revenue with its attribution rule, and verify the registration and each purchase, burn, and claim against finalized chain state. Burned principal is unavailable for refunds or operations, so operating funds and reserves must exist separately.

Measure actual pipeline costs before fixing the pricing multiplier or promising a price below Pangram. The exploratory multiplier of 25 and the detection price range of $0.01–$0.03 require representative document lengths, Container utilization and idle time, retries, and matched competitor volumes. Public-artifact serving and hosted *A* evaluation costs belong to SlopNinja. Miners fund their own training; later *B* miners also absorb bounded evaluation inference and required candidate-report costs. Shared benchmark infrastructure and external comparisons need an operator budget. Free `rtao-rpc` access does not remove chain or compute costs.

Paid-use emissions are separately optional. A nonzero share needs contributor attribution, ensemble treatment, caps, self-purchase controls, and unused-pool handling. Rehearse a common owner receiving miner rewards and validator dividends while purchasing its own traffic. Alpha burning does not by itself establish cost recovery or prevent manipulation. Measure usable coverage, false-positive disputes, repeat use, turnaround, willingness to pay, and operator cost per delivered job before expanding sales.

15.4 Later transformation and grammar mechanisms

Before *B* activates, choose benchmark and incident authority, panel and report-selection rules, service obligations, and a mapping into the chain's supported native mechanisms. Qualify the Cloudflare base/adapter and numeric ranker on the offered workload. Complete the content-privacy comparison with gateway logging enabled, a routed positive control, and a direct request without gateway selection. Check owner controls, private-weight denial to unrelated callers, model replacement, retention, and response isolation. Test adapter-controlled output-length leakage through owner-visible usage as described in Section 7.3. Synthetic jobs must survive restarts and duplicate delivery without exposing content to miners or creating duplicate credit. The provider boundary needs independent review of the actual serving path and account controls.

Transformation then needs joint author-fit, preservation, readability, tone, and detector comparisons, including blinded reader review and adaptive attacks on the public style metric. Test held-out authors, registers, source families, and later detectors with matched search budgets. Measure disagreements and preservation failures alongside score gains. Chapter and book claims require assembled-document evaluation of long-range consistency. A small protocol pilot cannot establish reader indistinguishability, safe editing, or profitable miner-paid inference.

C activates by its own decision, which may precede *B*'s so the grammar *B* will use can grow first. Before it activates, define admissible contributions, correctness and regression checks, overlap attribution, award sizes, and settlement/weight-expiry rules. Credit accepted reusable improvements once. Larger grammar files alone are not a benefit; evaluate held-out coverage, interpretation quality, ambiguity, and runtime cost, and measure each change's effect on *B*'s edit units and coverage of its intended workload. Shared grammar changes must preserve the frozen representations of running *B* comparisons. The contribution allocation starts inactive.

16 Related subnets, services, and research

The comparisons below use project documentation and papers reviewed between September 9 and 17, 2026, except for Umi (SN78), whose source was reviewed on October 30, 2026. Subnet numbers and service descriptions

are the projects' own claims; this survey is neither a current chain audit nor an independent performance evaluation.

16.1 Bittensor precedents

It's AI describes SN32 as a human-versus-AI text-classification subnet, with miners answering detection requests. BitMind GAS describes detector and generator competition for images, video, and audio, including generation rewards for fooling detectors. These establish precedents for detection and adversarial incentives [64–67]. Macrocosmos Finetuning's public model submissions provide a precedent for rewarding downloadable weights [68]. Umi (SN78) scores hidden cohorts against an incumbent re-run on the same suite, pays geometric weights across score bands, and keeps its previous row live until a successor activates; the October source review found these mechanisms working, alongside exact runtime-version pins that held weight writes after at least one chain upgrade and were relaxed several times [19]. SlopNinja's `reward-v2` ports the band mechanism, computes it from quantized scores, and avoids such pins. Its initial *A* competition adds a shared origin-and-attribution contract with span outputs. Whether these choices improve detection remains an experimental question.

16.2 Serving and privacy

Chutes, Targon, and Privatemode document confidential-serving approaches based on encryption, attestation, and protected execution [69–74]. SlopNinja trusts Cloudflare and the coordinator to protect content. The later *B* design seeks to keep request and response text from miners while keeping weights from the public. It does not promise secrecy from Cloudflare or the SlopNinja operator. Provider access controls and the exact request path therefore need qualification; citing confidential infrastructure does not establish this boundary.

16.3 Existing writing services

Pangram and GPTZero are relevant external origin comparators. Their output categories must be mapped to each test item's documented production history; neither agreement between detectors nor a low score establishes human authorship [50, 75]. Pangram remains the initial outside quality anchor. Price comparisons also require matched document sizes and request volumes.

Grammarly's personal voice profiles, Wordtune's editorial controls, and QuillBot's paraphrase modes are comparators for later *B* work [76–78]. Undetectable AI advertises detector evasion [79]. Those offerings establish product categories, not preservation or bypass performance on our material. Comparisons require the same authorized sources, briefs, and independent reader judgments.

16.4 Adoption and demand

Google's SynthID text watermarking, the Authors Guild's Human Authored certification, and Amazon KDP's distinction between generated and assisted content illustrate different approaches to origin disclosure [80–82]. Keyed watermark verification, production-origin inference, author attribution, and prose quality remain different claims. These developments motivate customer trials; they establish neither paid demand nor a sustainable price for SlopNinja.

16.5 Research foundations

Gero et al. use function-word frequencies and syntactic constructions for style control. StyleRemix supplies interpretable style axes and LoRA modules for authorship obfuscation [83, 84]. PAN/ELOQUENT 2024 joined detection with generation and obfuscation; DIPPER studied paraphrase attacks and retrieval-based defence [85, 86]. These results motivate source-family separation, evolving evaluation sets, and explicit preservation checks for any later rewriting service.

The proposed contribution is a protocol combining published origin-and-attribution models, recoverable daily evaluation cohorts, and separately activated transformation and grammar-contribution incentives, the latter built on a shared English grammar for *B*'s edits. Each component has precedents. Neither their combination nor this survey establishes novelty, quality, or commercial viability without direct comparison.

A Earlier research checkpoints

These checkpoints precede R_0 and gate nothing at launch. R_0 was fine-tuned from the V6 detector below, and the adaptation results trace that path; the grammar and CFG-vector results explain why A uses no CFG inputs and record the grammar's state for later B work.

Origin detector baseline, September 13, 2026. Our V6 baseline is a raw-text ModernBERT-base experiment fine-tuned for three production-origin classes. It accepts up to 2,048 tokens and has an offline CPU reference runner. The frozen primary corpus has 5,463 records drawn from 797 source families. For every record we retain the source rights, full generation ancestry, and split assignment [87]. Development data selected the checkpoint. We then fitted the temperature on Calibration and froze the decision cutoffs before looking at Test. V6's stricter and looser cutoffs are approximately 0.84365 and 0.42874; V5's are 0.90529 and 0.88845. A flag requires $P(\text{model_only}) + P(\text{mixed})$ strictly above the respective full-precision cutoff. Calibration targeted 1% and 5% human false-positive rates, respectively; these targets are not population guarantees. The 0.5 boundary below is a separate diagnostic operating point. Its results provided the raw-text comparator for the CFG-only experiments below.

The second-rewrite Test view, labeled R2 in the detector release, pairs 112 unchanged historical human-source passages with 112 model-only passages that each underwent two successive text rewrites: a Qwen anti-ai rewrite followed by a Mistral fix-slop rewrite. On these 224 passages from 112 source families, V6 reaches 95.1% accuracy at the 0.5 boundary; V5 reached 88.4%. The predeclared binary log loss fell from 0.2653 to 0.1007. A paired bootstrap by source family gives a 95% interval of $[-0.2229, -0.1075]$ for that difference. Mixed-origin passages are absent from these rows, and repeated views from successive revision stages are dependent observations. In a separate diagnostic over all 117 assigned human sources, the stricter frozen cutoff reduced false positives from 7 for V5 to 3 for V6, an observed V6 rate of 2.6% against the 1% Calibration target. At the looser cutoff they increased from 9 to 12, an observed V6 rate of 10.3% against the 5% target. Both observed V6 rates exceeded their Calibration targets. Three-class classification and mixed-origin cases are still material weaknesses [88].

A September 15 probe added one leading ASCII space to each of 126 Train records that V6 had already seen. The mean absolute change in combined model/mixed probability was 7.80 percentage points. In the largest model-only swing, the score fell from 96.73% to 1.30%. That is evidence of a formatting shortcut. It says nothing about held-out robustness or transfer to Pangram. The proposed text profile must prevent this exact representation change from earning later B utility. Paired formatting controls also remain necessary for A qualification [89].

We also ran a fresh paired comparison with Pangram. It contains 162 passages from 18 source families: 18 human passages, 108 distinct model drafts or revisions, and 36 direct model edits of human text. Using the declared boundary of at least 10% combined AI and AI-assisted content, Pangram flagged all 108 model-only passages and 34 of the 36 mixed-origin passages. It flagged none of the 18 human passages. At its stricter cutoff, V6 found 98 of 108 model-only and 29 of 36 mixed-origin passages, again with no observed human flags. At the looser cutoff it found 102 of 108 and 33 of 36, while flagging 2 of the 18 human passages.

These two scores answer different questions: Pangram reports a content fraction, whereas V6 reports a probability about the document's production origin. We have not matched their population false-positive rates. The 162 passages are also clustered within 18 source families, so they are not 162 independent trials. This sample cannot support a Pangram-parity claim, a rare error-rate guarantee, or unrestricted genre coverage. Pangram stays the outside quality anchor. V6 is a research baseline and is not yet a qualified subnet detector [88].

Native grammar checkpoint, October 5, 2026. The earlier author experiments combined word occurrences with grammatical features extracted from spaCy annotations. The current Rust path binds exact document bytes to reviewed sentence units, packed grammar forests, append-only ambiguity decisions and frozen lexical and grammatical coordinates. A compact cache records the source span, grammar identity, resource envelope, grammar-item dependencies and any selected tree. A grammar change can require replay beyond the old production-dependency set when it introduces new alternatives. An unchanged selected tree carries forward only when it maps exactly onto the new forest. This is the proposed representation for later B edits; A does not use it. The author-retrieval results below come from the earlier feature system.

At this checkpoint, the native grammar campaign had completed 205 of its first 500 documents. Of these, 151 tracked-public documents contributed 1,696 exact candidate units to a compact cache; 54 local-private proofs

covering 1,065 units remain outside Git. A document counts only when every candidate has a complete root, a complete lexical scan, no unknown token and no parser limit. The v368 replay of 1,657 previously tracked public units retained all 1,217 saved selections and every terminal status. It changed 305 packed forests only by adding alternatives; 92 changed, unselected ambiguities remained open for review rather than receiving the first available derivation. In the nine-document Wave 136 batch, only 47 of 148 candidate units parsed completely before repair. Reusable additions brought all 148 to complete roots without unknown tokens or resource limits, while a replay left all 1,657 previously tracked units complete. That batch added 39 public units to the prior 1,657, giving 1,696 at this checkpoint; its other 109 units belong to the private proofs. These figures measure source-bound parser coverage and replay behavior. Complete roots do not certify the selected interpretation or measure detector accuracy [90].

CFG-vector preparation, October 22, 2026. A Rust exporter produced whole-document structural features for 70 of 80 requested documents: 20 historical human proxies, 27 model drafts, and 23 mixed edits. Ten documents remained unavailable because 58 candidate spans lacked retained ambiguity selections. Twenty source families retained both human and model documents. The full requested population remains in the report, so these exclusions cannot disappear from a coverage denominator.

Export from existing cache trees took 9.97 seconds and peaked at 1,263,268 KiB RSS with no swap, under a one-CPU, 4 GiB limit. This excludes first-time parsing and ambiguity resolution. All 80 documents were previously Development and exposed to grammar work. Their caches span four parser engines and 899 grammar identities across six grammar/profile prefixes. The export preserves those identities rather than claiming one frozen representation. Lexical and combined inputs remain unavailable. No model was fitted, no attribution targets were certified, and no clean holdout result was produced [91].

Supervised shared CFG model, October 26, 2026. A Development-selected grammatical-leaf model uses a shared $2062 \to 64 \to 32$ encoder, a three-name model head and a four-name writer head. Train contains 229 queries, including 16 measured fractional-share targets; eight independent reference rows are excluded from fitting. Features, weights and Calibration mappings were frozen before Test. On 102 Test documents from 15 source families, including 21 human-labeled documents, joint seeds 17, 29 and 41 have involvement Brier 0.125825, 0.129129 and 0.134119, versus 0.095745 for the Calibration-fitted Pangram fraction-to-involvement mapping. That mapping is not Pangram's native involvement probability. Pangram is the paired raw-text comparator here; the historical V6 measurements use a different cohort. Human false-positive rates at 0.5 are 66.67%, 86.67% and 66.67%; share MAE is 30.18–34.72 percentage points. Conditional known-human top-1 is only 28.13–31.25%, and rejection of withheld identities is unstable. Matched origin-only and attribution-only controls retain explicit untrained-head limits. Historical human proxies, grammar exposure, source/generator confounds and the same withheld OLMo model across Calibration, Development and Test prevent broader generalization claims. No rare false-positive guarantee or Pangram parity follows. All 450 non-Test and 102 Test inputs have complete byte accounting with zero execution failures, but 35 and 34 unresolved tokens remain. Frozen projection matched all 450 non-Test numeric vectors exactly, and three unlabelled synthetic documents passed the native extraction-to-prediction path. A Wasm library build passed; neither that build nor small native Rust numeric inference qualifies parser memory or full Workers execution. Full per-seed evidence, artifact bindings and commands are recorded in the current repository report `grammar/crates/grammar-a-baseline/SUPERVISED_BASELINE.md`, linked from the continuing CFG-vector viability experiment [91].

A paired whitespace-collapse diagnostic retained the words and targets of all 102 Test documents while changing whitespace in 71. With joint seed 17 and all calibration frozen, Brier was 0.125631 versus 0.125825 originally; human FPR remained 66.67% and share MAE was 30.1074 percentage points. Mapped Pangram Brier was 0.095504, a paired gap of 0.030127. Equal-document absolute confidence changes averaged 0.1252 percentage points, with p95 0.3576 and maximum 4.2182; four exceeded one point and one crossed 50% confidence. Absolute share changes averaged 0.1123 points, with p95 0.3636 and maximum 4.2592; three exceeded one point. No conditional profile names changed, including outputs for absent roles; these comparisons are not false-attribution counts. There was no retraining. This single perturbation does not establish general formatting robustness.

Recovered V6 and shared text/CFG pilot, October 26, 2026. The unchanged V6 release was recovered and its original reference checks passed under a separately identified Linux x86-64 CPU profile. On the same consumed 102-document Test population, frozen V6 involvement Brier, with equal weight for each of the 15 source

families, is 0.13959 overall, 0.02283 on the 57 human/model-only documents, and 0.23824 on the 45 mixed documents. Its historical 95.1% accuracy excluded mixed examples and is not the result of this matched comparison. Source-family exposure strata and all unsupported-input checks are retained; the release has no trained share or attribution heads.

A subsequent bounded Development experiment freezes V6's 768-dimensional text representation and the native CFG's 2,062 numeric coordinates, with a shared $2830 \to 64 \to 32$ learned encoder feeding all current A heads. The 21 fits cover text-only, CFG-only and combined inputs, matched single-task controls, and seeds 17, 29 and 41. On 106 exposed Development documents, joint text-only Brier is 0.05177–0.05834; combined is 0.04842–0.05659; CFG-only is 0.12573–0.13122; mapped Pangram is 0.07705. Combined-minus-text Brier averages -0.00150, with a paired family interval of $[-0.01052, +0.00600]$. Combined human false positives remain 21.9–28.1% at 0.5. Conditional generator ranking improves over text-only, but writer retrieval remains near chance and open-set decisions are unstable. Mixed-share errors are much larger in sparse fractional-share ranges than the aggregate MAE suggests.

These checkpoints were selected on Development, including 50 documents exposed to V6 selection or Calibration. No new Test was opened and no new Pangram calls were needed. Thus lower Development Brier establishes neither Pangram parity nor a reliable incremental CFG benefit. Every numeric head passed Rust reference replay, but the complete ModernBERT pipeline was then unqualified for Rust and Cloudflare serving. The public 552-record cohort retains exact text, licenses, production spans and fixed splits; fitted profiles remain private. Full evidence and reproduction commands are in `docs/a-v6-cfg-fusion.md`.

A fresh-family comparison completed 39 shared-model fits on 929 non-Test records, with two known generators, a Calibration-only generator, independent writer works, and 96 measured fractional Train targets. The public snapshot contains 937 records including separate reference excerpts. Equal-family, equal-origin Development involvement Brier is 0.04103–0.04626 for text, 0.04474–0.04762 for combined, and 0.13809 for unchanged V6. Combined-minus-text averages $+0.00313$, with a family interval $[-0.00720, +0.01262]$. Combined share MAE improves to 10.04–11.10 percentage points versus text's 12.66–13.41, and observed human false positives fall to 3.1–6.3% versus 6.3–9.4%, at cutoff 0.5. There are only 32 human excerpts from 16 Development works; these are not rare-error guarantees. Aggregate CFG alone remains weak, and a fixed four-bin ordered-count probe underperforms its global control. Human attribution and open-set decisions also remain weak. See `docs/a-balanced-baseline.md` for all seeds, single-task tradeoffs, coverage and runtime costs.

Six further fits initialize a shared residual encoder to reproduce frozen V6's origin predictions, then train origin, share and attribution heads together. All 929 initialization checks pass within 10^{-6} absolute probability, and all six fitted heads pass Rust numeric replay. Text-only residual Brier is 0.03664–0.03910 and share MAE 8.27–8.73 points; adding CFG gives 0.04287–0.04555 and 8.81–9.27. Text-only is the current Development selection. Only 215 queries from 12 works train those selected models, with the upstream encoder frozen. Six matched fits then expand Train to 1,264 queries from 108 works, including 669 fractional-share targets. Development Brier worsens to 0.07354–0.08332 for text and 0.07044–0.07211 for text/CFG; share MAE is 10.40–11.66 points. The earlier text residual remains selected. The added historical V6 Train sources also change domain mixture and rewrite histories, so this is not a pure sample-count test. All examples and observations are retained, and no extra Pangram calls or new Test reads are needed. One fixed 50/50 original/added cohort weighting recovers text Brier to 0.04517–0.04830, while the earlier model remains selected. More independent works, writer coverage and balanced source mixtures are still needed; encoder adaptation and tree-aware CFG inputs remain untested. These count-feature experiments do not establish CFG's ceiling.

All 702 fresh Pangram observations were retained as permanent exact-text evidence. Its content fraction has 5.23-point error against the controlled replacement-share convention, better than these new A heads. It does not separately provide our share and involvement-confidence targets; any Calibration-derived confidence mapping is our proxy, not native Pangram confidence or a training label. A CAL-only monotone isotonic proxy gives balanced Brier 0.01896, better than every new text/combined seed. That diagnostic was added after inspecting Development; it is not independent Test evidence. No Pangram-parity claim follows. Historical human proxies and unknown pretrained exposure remain explicit. Test was unopened during the function-preserving V6 warm-start and Train-expansion comparisons. A local ModernBERT Rust CPU prototype passes the original reference and bounded synthetic checks; serverless serving was then unqualified. Head-only latency excludes feature extraction. Later sentence and paragraph scores require separate qualification.

Reserved 414-query Test, October 26, 2026. After all selection and Calibration were frozen, sixteen joint configurations with three fixed seeds were evaluated on 414 accepted queries from sixteen reserved works. Two of 416 requested queries failed during bounded generation; all accepted documents were supported by every candidate and frozen V6 without truncation. Four separate withheld-writer references complete an eight-writer gallery. Human examples remain historical proxies. The human/model/mixed counts are 32/94/288. All 414 additional Pangram observations and their raw exchanges are permanently retained, using the unchanged CAL-only isotonic mapping.

The preselected text residual has mean balanced involvement Brier 0.07661, versus 0.12434 for frozen V6 and 0.01537 for mapped Pangram. The selected-minus- Pangram gap is $+0.06124$, with paired family interval $[+0.02852, +0.10131]$. The selected-minus-V6 interval, $[-0.09590, +0.00578]$, includes zero. Its human false-positive rate at 0.5 is 23.96% averaged across seeds, versus 12.5% for V6 and zero observed flags for Pangram. Thus better mixed-inclusive detection than V6 comes with worse human flags; 32 human excerpts cannot establish rare error rates. Mean share MAE is 9.04 percentage points. Pangram's raw content fraction has 4.81-point diagnostic error against the replacement-span target, but that fraction is neither our involvement probability nor span ground truth. The original binary V6 evidence remains a separate population and task.

The full-data text candidate trained on 3,629 queries from 333 works reaches 0.05937 Test Brier, with 0.06625 for its matched CFG fusion. It cannot replace the preselected artifact on these results. The different Development ordering shows the limits of small selection populations. Aggregate grammatical-leaf CFG has 0.14692 Brier; the four-bin ordered probe has 0.17452. These do not establish a CFG-only detector or test full ordered-tree representations. Attribution, open-set behavior and Pangram parity remain unqualified. All seeds, family-aware uncertainty, missing support and output metrics are retained in `docs/a-final-test.md` and its aggregate evidence.

The selected exact-text-to-A pipeline passes native Linux Rust reference checks, with about 605 MB of encoder/head weights, 1.67 GiB peak RSS and a roughly thirteen-second 2,048-token forward on one CPU. First-time Test CFG extraction took 15m27s; V6 extraction took 12m28s. Cached features and sub-millisecond numeric heads do not remove those costs. Workers/Wasm and Cloudflare Container execution were then unqualified. No deployment or grammar expansion was performed.

Broader data and final-block adaptation, October 26, 2026. A separate bounded experiment trains on 4,157 queries from 357 works and selects on 439 Development queries, with 637 separate Calibration queries. Six fixed runs compare adaptation of V6's final encoder block with a matched frozen encoder, using one shared A representation and seeds 17, 29 and 41. CFG coordinates are zero in both conditions. Historical human proxies and measured replacement-span shares remain distinct from authenticated production histories. Previous Test populations are excluded from fitting and selection.

On 636 newly reserved Test queries from twenty works, selected adaptation has mean family/origin-balanced involvement Brier 0.07255 versus 0.07755 for its matched frozen control. The paired difference is -0.00500, with source-family interval $[-0.00935, -0.00148]$. With equal family weight but the observed origin mixture, Brier is 0.10610 versus 0.10593, with an interval for the difference spanning zero. Original frozen V6 scores 0.16827 balanced and 0.37594 under the observed mixture; previous A scores 0.09070 and 0.07054. Adaptation therefore improves mixed-inclusive detection over V6 and modestly improves the matched balanced criterion, but does not uniformly improve on previous A.

Adapted human false positives at 0.5 are 5–7.5% across seeds, versus 32.5% for previous A and zero for V6, on only forty human-proxy excerpts. Adapted share MAE is 11.55–12.43 percentage points overall and 14.88–16.05 on the 476 mixed-history documents. Previous A has lower overall share error, 10.03–11.17 points. These results establish neither rare-error guarantees nor useful general attribution. All candidates support the retained queries; four generated duplicate/no-change queries remain explicit upstream exclusions.

All 1,076 new CAL/Development Pangram responses are preserved. Development balanced Brier is 0.04382 for the CAL-mapped provider versus 0.07809 for selected adaptation; their paired gap is $+0.03427$ with interval $[+0.02408, +0.04569]$. Test collection is complete: all 636 queries have version 4.1 results: 607 from bulk requests, all successful, and 29 earlier results reused. Every request, including one rejected earlier, and every raw response remain evidence. Using the unchanged CAL mapping, provider Test Brier is 0.04803 balanced and 0.07348 under the observed mixture, versus 0.07255 and 0.10610 for adapted A. The paired balanced gap is $+0.02452$, interval

$[+0.00406, +0.04101]$; the family-weighted gap is $+0.03263$, interval $[+0.01600, +0.04877]$. Pangram flags one of forty human proxies at 0.5, versus two or three for adapted A. These results favor the comparator and do not establish parity or rare-error guarantees. Provider fractions are neither native involvement probabilities nor share labels. No Test result changes selection or Calibration.

A separate native Linux Rust/MKL profile passes all six calibrated runs on 44 fixed CUDA-reference cases each, retaining five explicitly unsupported inputs and earlier numerical failures. One-CPU 2,048-token encoder latency is about 19 seconds, heads about 0.13 milliseconds, and peak process RSS about 1.31 GiB. The roughly 623 MB parent/overlay/head artifacts and native-library profile do not fit or qualify Workers/Wasm; Container execution was then unqualified. Full results, scope, fixed-comparator routes and recovery commands are in `docs/a-encoder-adaptation.md`. Persisted CFG vectors and compressed ambiguous-forest archives support later reuse in *B* work, without changing this experiment's learned inputs. No deployment or grammar expansion was performed.

Earlier author experiments. For the table below, accuracy is first averaged within each author, then equally across authors, then across three seeds. The query counts are unique inputs scored in every seed. Consequently, the author-macro percentages are not pooled fractions of those query totals. The protocols are reported in [92,93].

Experiment	Author-macro top-1	Scope
Blog author retrieval	56.30%	600 evaluation authors in six 100-author galleries; 1,206 queries
Global Voices transfer	73.81% combined; 77.98% lexical-only	Fourteen provisionally qualified authors; 58 queries; topic/editorial confounds
Compact document projection	32.82% Blog; 25.00% Global Voices	Development-selected linear encoder; previously evaluated test corpora
Selected-feature ablation	40.83% Blog; 29.17% Global Voices	Conditional diagnostic retaining the frozen scorer's pooling and calibration

For Global Voices, we removed grammar terms from the frozen Blog-trained scorer without refitting the model or renormalizing the terms left behind. Accuracy on this sample rose by 4.17 percentage points, the opposite of grammar's contribution on the Blog data. We do not know why. There is no matched-topic control, and the experiment cannot separate register, editorial practice, and extraction effects. It argues for testing feature weights across registers; it does not show that grammar is generally useless [93].

The projection experiment also puts the parameter count in context. Its 3,557 learned coordinate weights are not a sufficient fixed input vocabulary, since the full model keeps additional word and grammar contributions that were not selected. Neither retrieval nor a feature ablation demonstrates safe editing or detector evasion. We saw the danger directly in another diagnostic: replacing *may* with *must* moved the passage closer to its target author while changing the claim. Preservation therefore needs an independent gate. See [92] for the projection study and [94] for the counterexamples.

References

[1] sam0x17. rtao: Durable native work and transaction recovery. https://github.com/sam0x17/rtao/blob/aa8627ba8f3691ec1ecd14b11b612b0f86479e6c/docs/recovery/durable-work.md. Repository revision aa8627ba; accessed October 23, 2026.

[2] Cloudflare. Workers: Rust language support. https://developers.cloudflare.com/workers/languages/rust/. Accessed October 23, 2026.

[3] Cloudflare. Access Durable Objects Storage. https://developers.cloudflare.com/durable-objects/best-practices/access-durable-objects-storage/. Accessed October 23, 2026.

[4] sam0x17. rtao-rpc: Worker specification. `sam0x17/rtao-rpc`, `docs/specs/worker.md`. Repository revision 53615d67; accessed October 23, 2026. Repository access restricted.

[5] Cloudflare. Workers: Fetch handler. https://developers.cloudflare.com/workers/runtime-apis/handlers/fetch/. Accessed October 23, 2026.

[6] Cloudflare. Workers: Cron Triggers. https://developers.cloudflare.com/workers/configuration/cron-triggers/. Accessed October 23, 2026.

[7] Cloudflare. Queues: Delivery guarantees. https://developers.cloudflare.com/queues/reference/delivery-guarantees/. Accessed October 23, 2026.

[8] SlopNinja Research. Training an A artifact from public material. https://github.com/sam0x17/slop-ninja/blob/main/docs/testnet/training-guide.md, 2026. Testnet training recipe, tested end to end on Linux x86_64 with a smoke run.

[9] Answer.AI and LightOn. ModernBERT-base model card. https://huggingface.co/answerdotai/ModernBERT-base. Accessed September 9, 2026.

[10] SlopNinja Research. CFG-vector viability experiment. https://github.com/sam0x17/slop-ninja/blob/main/docs/cfg-vector-viability.md, 2026. Supervised shared CFG model on 102 Test documents, October 26, 2026.

[11] SlopNinja Research. Frozen V6 features with a shared A head. https://github.com/sam0x17/slop-ninja/blob/main/docs/a-v6-cfg-fusion.md, 2026. Development-selected text, CFG and combined fits, October 26, 2026.

[12] SlopNinja Research. A artifact spec, versions 1 to 5. https://github.com/sam0x17/slop-ninja/blob/main/docs/testnet/artifact-spec.md, 2026. Testnet admission checks as implemented; spec v3 with the span tower class, November 2, 2026; spec v5 with the span requirements, built and not yet named by any cohort.

[13] Cloudflare. R2 object storage. https://developers.cloudflare.com/r2/. Accessed October 23, 2026.

[14] Global Voices. Global Voices attribution policy. https://globalvoices.org/about/global-voices-attribution-policy/. Accessed September 9, 2026.

[15] PLOS. Licenses and copyright. https://journals.plos.org/plosone/s/licenses-and-copyright. Accessed September 9, 2026.

[16] SlopNinja. Miner training sources and acquisition plan. https://github.com/sam0x17/slop-ninja/blob/main/docs/miner-training-sources.md. Repository revision 338804f.

[17] Bittensor. Chain consensus. https://www.bittensor.com/docs/concepts/chain-consensus. Accessed September 10, 2026.

[18] Bittensor. Emissions and Yuma Consensus. https://www.bittensor.com/docs/concepts/emissions. Accessed October 30, 2026.

[19] Umi-BitSign. Umi (SN78): cohort model award and geometric rewards. https://github.com/Umi-BitSign/umi/blob/04406d726d9499a920c39a4886022d3f94263a1b/src/umi/competition_geometric_rewards.py. Repository revision 04406d72; band assignment in src/umi/competition_cohort_model_award.py; source review, October 30, 2026.

[20] SlopNinja Research. Shared subnet protocol crate: reward-v2, quant-v1, private submission, authorizations and the copy policy. https://github.com/sam0x17/slop-ninja/blob/main/subnet/crates/protocol/README.md, 2026. November 9, 2026, with the coordinator's signed row record; native and wasm32 golden reports reproduce byte for byte.

[21] Cloudflare. Workers AI: Limits. https://developers.cloudflare.com/workers-ai/platform/limits/. Accessed October 30, 2026.

[22] Common Crawl. Common Crawl: Get started. https://commoncrawl.org/get-started. Accessed October 30, 2026.

[23] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of Language Models: Part 3.3, Knowledge Capacity Scaling Laws. arXiv:2404.05405, 2024. https://arxiv.org/abs/2404.05405.

[24] John X. Morris, Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar. How much do language models memorize? arXiv:2505.24832, 2025. https://arxiv.org/abs/2505.24832.

[25] Bittensor. Subnet mechanisms. https://www.bittensor.com/docs/guides/subnets#mechanisms. Accessed October 30, 2026.

[26] Bittensor. Emission Weights. https://www.bittensor.com/docs/concepts/emission-weights. Accessed October 31, 2026.

[27] Bittensor. Tempo: blocks per epoch. https://www.bittensor.com/docs/hyperparameters/tempo. Accessed October 23, 2026.

[28] Bittensor. Immunity period. https://www.bittensor.com/docs/hyperparameters/immunity-period. Accessed October 30, 2026.

[29] Cloudflare. Containers: Limits and instance types. https://developers.cloudflare.com/containers/platform/limits/. Accessed October 30, 2026.

[30] Bittensor. commit_reveal_weights_enabled. https://www.bittensor.com/docs/hyperparameters/commit-reveal-weights-enabled. Accessed October 23, 2026. Owner-settable native weight submission mode.

[31] Bittensor. Validating: Commit-reveal. https://www.bittensor.com/docs/guides/validating#commit-reveal. Accessed September 9, 2026.

[32] SlopNinja Research. Native subnet chain integration. https://github.com/sam0x17/slop-ninja/blob/main/subnet/README.md, 2026. rtao pin fa83052d and its limits, recorded November 5, 2026, with earlier pins.

[33] Cloudflare. Workers AI: Data usage. https://developers.cloudflare.com/workers-ai/platform/data-usage/. Updated October 16, 2026; accessed October 23, 2026.

[34] Cloudflare. Workers AI: REST API. https://developers.cloudflare.com/workers-ai/get-started/rest-api/. Accessed October 23, 2026.

[35] Cloudflare. AI Gateway: REST API. https://developers.cloudflare.com/ai-gateway/usage/rest-api/. Accessed October 23, 2026.

[36] SlopNinja. Cloudflare direct-inference checks and reported owner-session inspection. https://github.com/sam0x17/slop-ninja/blob/main/docs/research-notes.md#cloudflare-private-lora-access-verification-2026-10-22. October 22, 2026. Separates API observations from the operator-supplied browser report; owner-enabled content logging remains untested.

[37] SlopNinja Research. f32 serving of A on Cloudflare Containers. https://github.com/sam0x17/slop-ninja/blob/main/docs/a-container-serving.md, 2026. Measured October 29, 2026; synthetic text only.

[38] Cloudflare. Workers AI pricing. https://developers.cloudflare.com/workers-ai/platform/pricing/. Accessed October 23, 2026.

[39] Cloudflare. Containers pricing. https://developers.cloudflare.com/containers/platform/pricing/. Accessed October 23, 2026.

[40] Cloudflare. Workers AI: Using LoRA adapters. https://developers.cloudflare.com/workers-ai/features/fine-tunes/loras/. Accessed October 23, 2026; beta limits and pricing are subject to change.

[41] Pangram. AI Detection API: Developer pricing. https://www.pangram.com/solutions/api. Accessed October 23, 2026; standalone prepaid API rates.

[42] Pangram. Bulk API. https://docs.pangram.com/api-reference/bulk-api. Accessed October 23, 2026.

[43] TAO.com. Help with Buying TAO. https://support.tao.com/support/troubleshooting-and-help/help-with-buying-tao. Accessed October 23, 2026; documents TAO.com's Coinbase Onramp integration.

[44] Bittensor. Stake-burn transaction. https://www.bittensor.com/docs/tx/stake-burn. Accessed October 23, 2026.

[45] Bittensor. Subtensor alpha burn and recycle implementation. https://www.bittensor.com/code/pallets/subtensor/src/staking/recycle_alpha.rs. Code view revision 25b3866e13; accessed October 23, 2026.

[46] Model Context Protocol. The 2026-07-28 Specification. https://blog.modelcontextprotocol.io/posts/2026-07-28/, 2026. Published July 28; accessed September 17, 2026.

[47] Unicode Consortium. Unicode Standard Annex #15: Unicode Normalization Forms. https://www.unicode.org/reports/tr15/, 2026. Unicode 18.0.0, Revision 58, August 12; accessed September 17, 2026.

[48] Unicode Consortium. Unicode Technical Standard #39: Unicode Security Mechanisms. https://www.unicode.org/reports/tr39/, 2026. Unicode 18.0.0, Revision 34, August 27; accessed September 17, 2026.

[49] Pangram. AI detection API reference. https://docs.pangram.com/api-reference/ai-detection. Accessed September 9, 2026.

[50] Pangram. Pangram 4 model card. https://www.pangram.com/research/model-card/pangram-4. Accessed September 9, 2026.

[51] Pangram. What should I do if a student submits AI work? https://www.pangram.com/knowledge-hub/what-should-i-do-if-a-student-submits-ai-work. Reviewed October 9, 2026; accessed October 23, 2026. Defines character-weighted shares and separate segment confidence.

[52] SlopNinja. Pangram public-result probe. https://github.com/sam0x17/slop-ninja/blob/main/docs/pangram-public-results.md. Repository revision 338804f.

[53] SlopNinja. Pangram repeatability probe. https://github.com/sam0x17/slop-ninja/blob/main/docs/pangram-repeatability.md. Repository revision 338804f.

[54] SlopNinja Research. A baseline release candidate. https://github.com/sam0x17/slop-ninja/blob/main/docs/a-baseline-release.md, 2026. October 28, 2026; research repository.

[55] SlopNinja Research. A span localization results. https://github.com/sam0x17/slop-ninja/blob/main/docs/a-spans-results.md, 2026. Protocol spans-v1.1, measured October 30–31, 2026, with the shipped configuration of October 31; follow-up arm S2, measured October 31 to November 1, 2026. Research repository.

[56] SlopNinja Research. A span localization under the 24-word floor and the first span package, 2026. Floor evaluation, masked retrain, and 16-word comparison of November 7, 2026; package checks of November 8–9, 2026. Research report, access restricted during subnet preparation.

[57] SlopNinja Research. Competitive A evaluation protocol. https://github.com/sam0x17/slop-ninja/blob/main/docs/a-competitive-protocol.md, 2026. Pre-registered Tier C and Tier I criteria; results of the October 30, 2026 sealed Test are in an access-restricted report.

[58] SlopNinja Research. Common Crawl human slice: pilot sampler, contamination, era and lookup measurements. https://github.com/sam0x17/slop-ninja/blob/main/grammar/crates/grammar-cc-human/README.md, 2026. Pilot v1 of October 30, 2026, and the domain precision, share-table, twin and era-gate measurements of October 31; texts and per-document results are private.

[59] SlopNinja Research. CFG writer vectors versus A encoder profiles: results. https://github.com/sam0x17/slop-ninja/blob/main/docs/cfg-vs-encoder-results.md, 2026. Frozen protocol cfg-vs-encoder-20261031 and its version 2 follow-up, October 31, 2026.

[60] SlopNinja. Encrypted receipt implementation. https://github.com/sam0x17/slop-ninja/blob/main/docs/receipt-envelope.md. Repository revision 338804f.

[61] SlopNinja Research. Testnet M2: native weight commitments on subnet 601. https://github.com/sam0x17/slop-ninja/blob/main/subnet/crates/validator/TESTNET-M2.md, 2026. Live commitments and fault drills of October 30, 2026.

[62] SlopNinja Research. Testnet coordinator runbook: `reward-v1` rehearsal and review fixes. https://github.com/sam0x17/slop-ninja/blob/main/subnet/coordinator/TESTNET.md, 2026. Rehearsal cohorts of October 31 to November 1, 2026, on subnet 601. Research repository.

[63] SlopNinja. Testnet status. https://slop.ninja/testnet/status/, 2026. Cohort intake and published outcomes on testnet subnet 601, accessed November 9, 2026.

[64] It's AI. Text detection subnet. https://github.com/It-s-AI/llm-detection. Accessed September 9, 2026.

[65] It's AI. Mining and model FAQ. https://github.com/It-s-AI/llm-detection/blob/main/docs/FAQ.md. Accessed September 9, 2026.

[66] BitMind. GAS: Generative Adversarial Subnet. https://github.com/BitMind-AI/bitmind-subnet. Accessed September 9, 2026.

[67] BitMind. GAS generative mining guide. https://github.com/BitMind-AI/bitmind-subnet/blob/main/docs/Generative-Mining.md. Accessed September 9, 2026.

[68] Macrocosmos. Finetuning Subnet: Incentive overview. https://github.com/macrocosm-os/finetuning. Accessed September 9, 2026.

[69] Chutes. The Chutes starter guide. https://chutes.ai/docs/guides/starter-guide. Accessed September 9, 2026.

[70] Chutes. Private AI inference: What it means and how Chutes makes it verifiable. https://chutes.ai/news/private-ai-inference-verifiable-confidential-llm-serving. Accessed September 9, 2026.

[71] Targon. Infrastructure. https://targon.com/infrastructure. Accessed September 9, 2026.

[72] Targon. Confidential compute technical paper. https://targon.com/releases/intel-whitepaper. Accessed September 9, 2026.

[73] Edgeless Systems. Privatemode: Remote attestation overview. https://docs.privatemode.ai/security/attestation/overview/. Accessed September 11, 2026.

[74] Edgeless Systems. Privatemode: Prompt cache security. https://docs.privatemode.ai/security/prompt-caching/. Accessed September 11, 2026.

[75] GPTZero. Technology and API frequently asked questions. https://gptzero.me/technology. Accessed September 9, 2026.

[76] Grammarly. Introducing voice features. https://support.grammarly.com/hc/en-us/articles/23153676821773-Introducing-voice-features. Accessed September 9, 2026.

[77] Wordtune. Wordtune Editor explained. https://support.wordtune.com/en/articles/8995735-wordtune-editor-explained. Accessed September 9, 2026.

[78] QuillBot. Paraphraser modes. https://help.quillbot.com/hc/en-us/articles/35854318883351-What-are-modes-in-the-Quillbot-Paraphraser-and-how-do-I-use-them. Accessed September 9, 2026.

[79] Undetectable AI. Humanize documents. https://help.undetectable.ai/en/article/humanize-documents-1nehls2/. Accessed September 9, 2026.

[80] Google DeepMind. Watermarking AI-generated text and video with SynthID. https://deepmind.google/blog/watermarking-ai-generated-text-and-video-with-synthid/, 2024. Published May 14; accessed September 10, 2026.

[81] Authors Guild. Authors Guild Launches Human Authored Certification to Preserve Authenticity in Literature. https://authorsguild.org/news/ag-launches-human-authored-certification-to-preserve-authenticity-in-literature/, 2025. Published January 29; accessed September 17, 2026.

[82] Amazon Kindle Direct Publishing. Content Guidelines. https://kdp.amazon.com/en_US/help/topic/G200672390. Accessed September 17, 2026.

[83] Katy Gero, Chris Kedzie, Jonathan Reeve, and Lydia Chilton. Low Level Linguistic Controls for Style Transfer and Content Preservation. In *Proceedings of the 12th International Conference on Natural Language Generation*, pages 208–218. Association for Computational Linguistics, 2019. https://aclanthology.org/W19-8628/.

[84] Jillian Fisher, Skyler Hallinan, Ximing Lu, Mitchell L. Gordon, Zaid Harchaoui, and Yejin Choi. StyleRemix: Interpretable Authorship Obfuscation via Distillation and Perturbation of Style Elements. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 4172–4206. Association for Computational Linguistics, 2024. https://aclanthology.org/2024.emnlp-main.241/.

[85] PAN and ELOQUENT. Voight-Kampff Generative AI Authorship Verification 2024. https://pan.webis.de/clef24/pan24-web/generated-content-analysis.html. Accessed September 9, 2026.

[86] Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John Wieting, and Mohit Iyyer. Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense. In *Advances in Neural Information Processing Systems*, 2023. https://arxiv.org/abs/2303.13408.

[87] SlopNinja Research. Revision detector V6 corpus. https://github.com/sam0x17/slop-ninja/blob/main/datasets/detector-v6/README.md, 2026. September 13. Research repository.

[88] SlopNinja Research. V6 detector evaluation. https://github.com/sam0x17/slop-ninja/blob/main/baseline/detector/releases/v0.6.0/RESULTS.md, 2026. September 13. Research repository.

[89] SlopNinja Research. One leading space exposes a V6 formatting weakness. https://github.com/sam0x17/slop-ninja/blob/main/baseline/detector/BOUNDARY_SPACE_RESULTS_V1.md, 2026. September 15. Research repository.

[90] SlopNinja Research. Native CFG campaign coverage and replay audit, 2026. October 5. Native grammar workspace, not yet included in the public repository.

[91] SlopNinja Research. CFG-vector preparation and viability experiment. https://github.com/sam0x17/slop-ninja/blob/main/docs/cfg-vector-viability.md, 2026. October 22 preparation checkpoint; research repository.

[92] SlopNinja. Author projection study. https://github.com/sam0x17/slop-ninja/blob/main/docs/author-projection.md. Repository revision 338804f.

[93] SlopNinja. Global Voices author transfer. https://github.com/sam0x17/slop-ninja/blob/main/docs/global-voices-transfer.md. Repository revision 338804f.

[94] SlopNinja. Edit utility matrix and preservation counterexamples. https://github.com/sam0x17/slop-ninja/blob/main/docs/edit-utility-matrix.md. Repository revision 338804f.